

M202 Adopter l'approche agile

1^{ère} édition

Support du Cours
& Travaux dirigés / pratiques

Table des matières

1 - Connaître les fondamentaux de la gestion de projet.....	3
1.1- Découvrir les Concepts de gestion de projet	3
1.1.1 - Concepts de gestion de projet	3
1.1.2 - Parties prenantes de projet.....	5
1.1.3 - Principaux rôles dans un projet informatique.....	7
1.1.4 - Caractéristiques de base d'un projet	7
1.1.5 - Contraintes dans la gestion d'un projet	8
1.2- Découvrir les différentes méthodes de gestion de projet	9
1.2.1 - Les méthodes de gestion de projet traditionnelles	9
1.2.2 - Les méthodes de gestion de projet agiles.....	12
1.2.3 - Cycle en V vs. Méthodes agiles	14
2- Planifier un projet	15
2.1- Analyser le cahier des charges	15
2.1.1 - Compréhension des besoins client.....	15
2.1.2 - Contexte du projet	18
2.1.3 - Périmètre du projet.....	18
2.1.4 - Détection des risques liés à la nature du projet.....	19
2.1.5 - Proposition des solutions possibles	20
2.2- Préparer le projet	22
2.2.1 - Répartition de l'ensemble des fonctionnalités en tâches.....	22
2.2.2 - Estimation de la durée de réalisation de chaque tâche	23
2.2.3 - Ordonnancement des tâches	24
2.2.4 - Chemin critique	25
2.2.5 - Echancier et la chronologie des tâches	25
2.2.6 - Affectation des ressources aux tâches	27
2.2.7 - Maîtrise des coûts	28
3- Adopter l'approche Agile dans la gestion de projet	29
3.1 - La méthode Agile Scrum	29
3.1.1 - Les trois piliers de scrum	29
3.1.2 - Processus de la méthode Scrum	29
3.1.3 - Rôles et responsabilités :.....	30
3.1.4 - Evénements Scrum :.....	31
3.1.5 - Artéfacts Scrum :	34
3.2 - L'outil de gestion de projet Agile (Scrum/Jira).....	35
3.2.1 - Présentation de Jira.....	35
3.2.2 - Installation et configuration d'un compte Jira	37

3.2.3 - Création de projet SCRUM	40
3.2.4 – Gérer votre projet.....	43
3.2.4.1 - Créer un backlog.....	43
3.2.4.2 - Créer un sprint :.....	44
3.2.4.3 - Créer une tâche :.....	45
3.2.4.4 - Ajouter une personne à votre projet (un collaborateur) :	46
3.2.4.5 - Affecter une tâche à une personne :.....	47
4- Outils de gestion de versions et de mesure de la qualité du code	49
4.1 - Les outils de gestion de versions (git/gitlab)	49
4.1.1 - Introduction.....	49
4.1.2 - Présentation de git et gitlab.....	49
4.1.3 - Fonctionnement de base de Git.....	50
4.1.4 - Manipulation des commandes de base de Git.....	51
4.1.5 - Manipulation des dépôts avec gitlab	54
4.1.6 - Collaboration en gitlab	60
4.1.7 - Manipulation des branches.....	60
4.1.8 - Fusion et gestion des conflits.....	62
4.2 - L'outil de mesure de la qualité du code (SonarQube)	64
4.2.1 - Notions des métriques de la qualité du code	64
4.2.2 - Présentation des outils existants de mesure de la qualité du code.....	69
4.2.3 - Présentation de SonarQube	69
4.2.4 - Installation de SonarQube.....	71
Travaux dirigés et pratiques – M202 Adopter l'approche agile	78
TD1 : Planifier un projet.....	78
Exercice 1 :	78
Exercice 2 :	78
TD2 : Adopter une méthode agile SCRUM.....	79
TP1 : Créer un projet SCRUM.....	81
TP2 : Créer un dépôt git sur GitLab.....	83
TP3 : Collaborer sur un dépôt gitlab.....	84
TP4 : Les branches et gestion de conflits	85
Références	87

1 - Connaître les fondamentaux de la gestion de projet

1.1- Découvrir les Concepts de gestion de projet

1.1.1 - Concepts de gestion de projet

Définition d'un projet

Le projet est une **série d'activités** conduites par un effort complexe, non répétitif et unique, **limité par des contraintes de temps, de budget, et de ressources**.

Gestion de projets

La **gestion de projet**, la conduite de projet, l'ingénierie de projet, ou encore le management de projet est **l'ensemble des activités visant à organiser le bon déroulement d'un projet et à en atteindre les objectifs** en temps et en heures selon les objectifs visés.

Elle consiste à appliquer les méthodes, techniques, et outils de gestion spécifiques aux différentes étapes du projet, de l'évaluation de l'opportunité jusqu'à l'achèvement du projet.

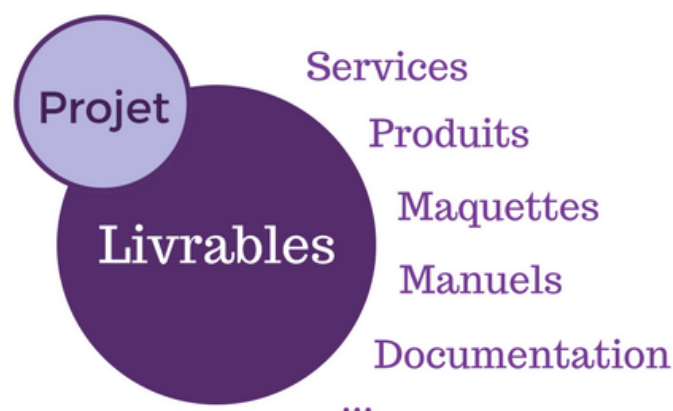
Ressources de projet

Il existe 3 sortes de ressources pour un projet :

- Les ressources humaines
Il s'agit tout simplement de l'ensemble des personnes qui vont intervenir sur le projet
- Les ressources financières
Elles font référence au budget global du projet.
- Les ressources matérielles
Elles concernent tout ce dont l'équipe projet va avoir besoin pour mener à bien le projet : les salles, les équipements informatiques, les logiciels...

Livrables de projet

La conduite d'un projet débouche sur un produit, un service, une nouvelle organisation, etc. Cette **finalité**, appelée "**livrable**", est **le résultat tangible d'une production réelle**, appréhendable, mesurable attendue par le client final. Un projet peut, bien sûr, avoir plusieurs livrables.



Charte de projet

La charte de projet, appelée aussi note de cadrage ou note de synthèse, est un **document qui confirme l'existence d'un projet en décrivant le projet et les résultats à atteindre**, et confère par la même occasion au gestionnaire du projet l'autorité d'utiliser des ressources organisationnelles pour mener à bien les activités du projet.

Une bonne charte contient obligatoirement les éléments suivants :

- **Présentation du projet**

Le document charte de projet commence par la **définition des objectifs** du projet et les **critères de mesure de succès** afin de confirmer l'atteinte des objectifs et résultats souhaités.

Cette synthèse doit contenir tous les renseignements dont ont besoin les principales parties prenantes pour **approuver le projet**.

- **Périmètre du projet**

Donner une **description générale des inclusions et exclusions du projet** (qu'est ce qui est dans le périmètre et qu'est ce qui ne l'est pas)

Vous pouvez également détailler les caractéristiques du produit ou du service à offrir ou le résultat à atteindre dans le cadre du projet.

- **Jalons**

Il ne s'agit pas de donner des dates précises, mais d'**indiquer les principaux jalons et deadlines du projet**. Notamment, les phases et étapes du projet, ainsi que les points de vérification et les portes d'approbation.

- **Livrables**

Définissez les principaux **produits, services ou résultats qui doivent être fournis** dans le cadre du projet en vue de réaliser le bénéfice attendu.

- **Budget**

Présentez le **budget global approuvé pour le projet** et une **estimation des coûts** pour les différentes ressources projet (humaines, matérielles et financières) qui sont nécessaires pour la réalisation du projet.

- **Risques**

La charte de projet serait incomplète sans **présenter une évaluation initiale des risques**, notamment les risques stratégiques qui ont été identifiés au début du projet. Pour chaque risque identifié, évaluer son niveau d'**impact** et sa **probabilité** de survenance (exemple : faible, moyen ou élevé).

- **Gouvernance, chef de projet, et parties prenantes**

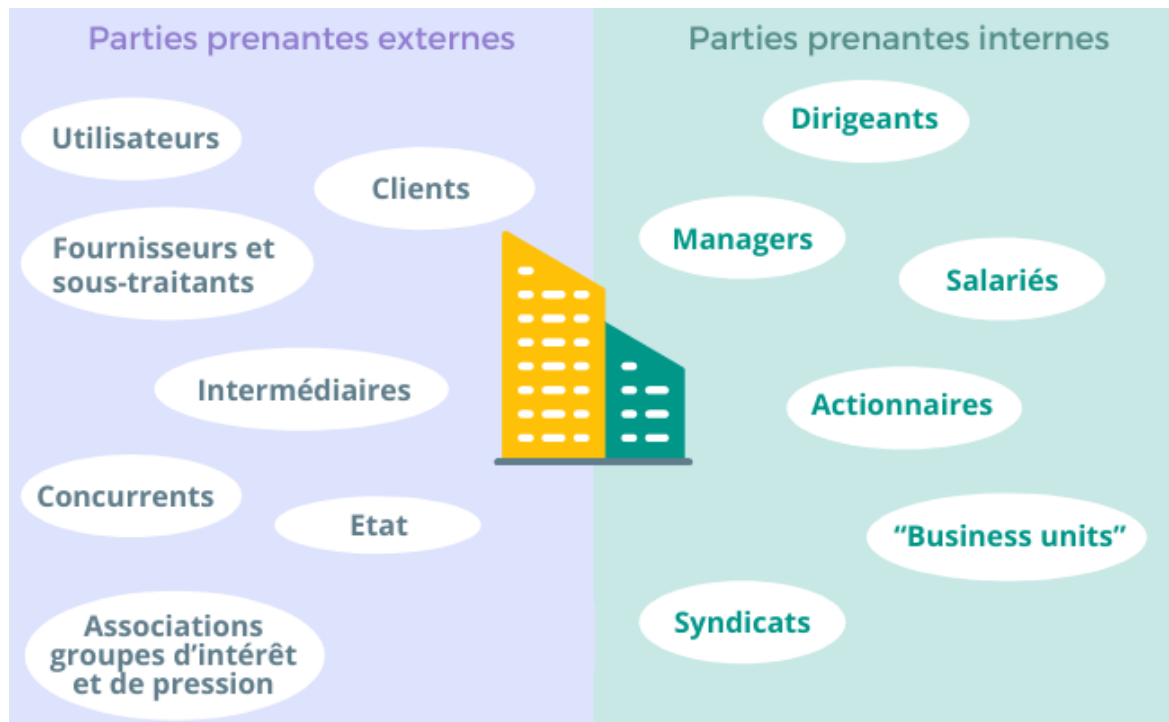
Décrivez **la façon avec laquelle le projet sera géré** et **définir les instances de gouvernance** qui vont être impliquées dans le processus d'approbation de la charte de projet.

1.1.2 - Parties prenantes de projet

Il s'agit des personnes physiques ou morales, groupes d'individus dans l'environnement de l'entreprise qui possèdent un intérêt dans son fonctionnement et ses résultats.

Elles sont impactées ou influencées directement ou indirectement par les décisions de l'entreprise et par ses activités.

Elles peuvent exercer elles-mêmes une influence plus ou moins forte sur la gouvernance et la stratégie de l'organisation. Pour certaines, elles peuvent même exercer un contre-pouvoir.



Les parties prenantes internes :

- **Dirigeants (dont le comité de direction)**
Le rôle de cette partie prenante est d'assurer la gouvernance de l'entreprise. Ils sont éloignés du management opérationnel quotidien, mais sont informés par des outils de pilotage.
- **Managers**
Chargés d'appliquer la stratégie décidée en amont. Ils sont influencés par les performances de l'entreprise, mais en sont également les responsables.
- **Salariés**
Ils sont essentiels à la bonne marche des affaires. Leur impact est collectivement majeur. Leurs attentes se partagent entre le volet financier (la rémunération) et, de plus en plus, par le non financier (épanouissement, reconnaissance, bien être au travail ...).
- **Actionnaires**
Propriétaire pour un actionnaire unique, ou bien ensemble d'actionnaires. Ils sont intéressés par la rentabilité et la valeur créée par l'entreprise.
- **"Business units"**
Services, départements, filiales qui ont leurs propres intérêts, leurs propres objectifs de rentabilité et de performance. Ils contribuent à la réussite de l'entité supérieure et

ont leurs propres exigences : moyens et ressources, autonomie, etc. Ils sont influencés par les objectifs assignés ainsi que les moyens alloués.

- **Syndicats**

Attentifs aux intérêts des salariés, les syndicats ont un rôle de contre-pouvoir au sein de la structure. Ils exercent une influence à divers niveaux.

Les parties prenantes externes :

- **Clients**

Partie prenante externe majeure, il s'agit des personnes physiques et des organisations qui achètent le produit (ou service). La qualité, le service rendu, la satisfaction de ses besoins.

- **Utilisateurs**

Ceux qui utilisent le produit. Ils peuvent être différents du client et participer à la prise de décision d'achat.

- **Fournisseurs et sous-traitants**

Ils contribuent à la performance de la chaîne de valeur de l'entreprise et à sa rentabilité (via le coût de revient). Leur performance en amont impacte l'aval (fiabilité des délais, qualité des composants livrés ou de la prestation fournie, etc.).

- **Intermédiaires**

C'est une vaste catégorie qui comprend les intervenants intermédiaires dans une filière comme les prescripteurs, grossistes, etc.

- **Concurrents**

Pas toujours référencés dans les parties prenantes (ils font généralement l'objet d'une analyse concurrentielle), les influences croisées entre les concurrents sont pourtant essentielles. Au-delà du côté compétitif, des échanges peuvent s'établir sur des sujets normatifs (ou autres) communs à l'ensemble des acteurs d'un marché.

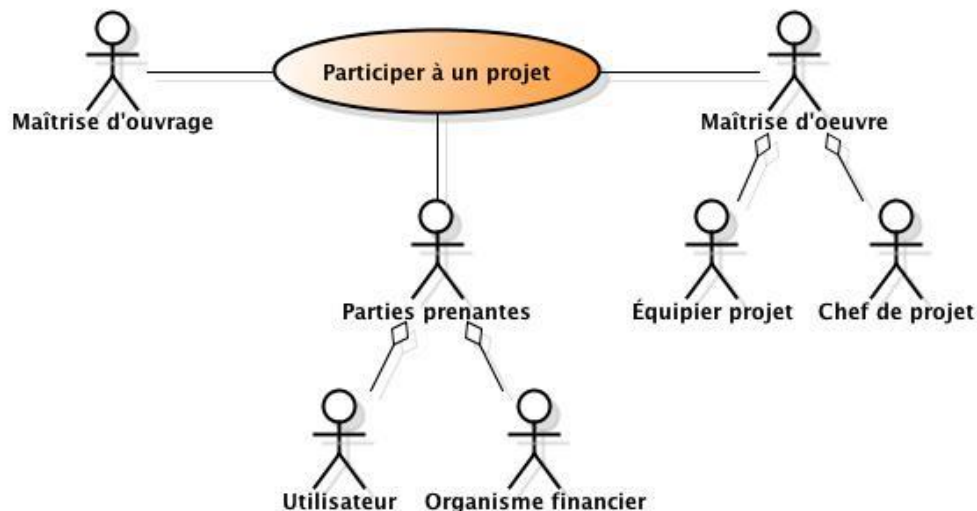
- **Etat**

Il s'agit d'une partie prenante majeure, car à travers son rôle de collecte des impôts et des taxes, mais aussi de fixation des règles du jeu des marchés par les lois et règlements (droit des affaires notamment). Il crée également les conditions d'exploitation interne de par le droit du travail.

- **Associations, groupes d'intérêt et de pression**

Ce sont divers groupes de consommateurs ou autres très présents dans les thématiques liées au développement durable (et dérèglement climatique).

1.1.3 - Principaux rôles dans un projet informatique



- **La maîtrise d'ouvrage (MOA)** : il s'agit du « client » du projet, soit celui qui en attend des résultats concrets.
- **La maîtrise d'oeuvre (MOE)** : il s'agit du « fournisseur » du projet, soit celui qui réalise l'ouvrage même.
- **Le chef de projet**, appelé aussi Project Manager, est responsable de l'équipe projet en charge de la préparation, de la réalisation et de la finalisation du projet.

1.1.4 - Caractéristiques de base d'un projet

Objectifs / Résultat ou produit attendu

Les projets ont des buts et objectifs clairement définis et exposés pour produire des résultats clairement définis.

Durée/Espace

Les projets se cadencent obligatoirement dans le temps : ils possèdent une date de début et une date de fin, et se déroulent dans un lieu et un contexte spécifiques.

Activités

Chaque projet est unique et nécessite la mise en place d'une organisation et d'un mode de pilotage spécifique.

Ressources

Il est indispensable de bien répartir les ressources, qu'elles soient humaines, financières ou matérielles, pour qu'un projet puisse aboutir.

1.1.5 - Contraintes dans la gestion d'un projet

Les contraintes de projet désignent les limites générales d'un projet, notamment les délais, les coûts et les risques. Il est important d'identifier les contraintes d'un projet, car elles ont des répercussions sur les performances de ce dernier.

Les trois contraintes de la gestion de projet, également connues comme le triangle d'or de la gestion de projet, sont : la portée, les coûts et les délais. Vous devrez équilibrer ces trois éléments pour chaque projet, même si cela peut s'avérer délicat puisqu'ils ont tous des répercussions les uns sur les autres.

Portée

La portée d'un projet correspond à son **ampleur en termes de qualité, de détails et de livrables**. Le délai et le budget dépendent de la portée du projet, car plus la portée s'étend, plus il faudra de temps et d'argent pour le mener à bien.

Coût

Les contraintes de coûts comprennent le budget du projet dans son ensemble et tout élément de valeur financière nécessaire à votre projet.

Délais

La gestion du temps est essentielle à la réussite du projet, et vous rencontrerez diverses contraintes de temps au cours de chacune des phases de ce dernier.

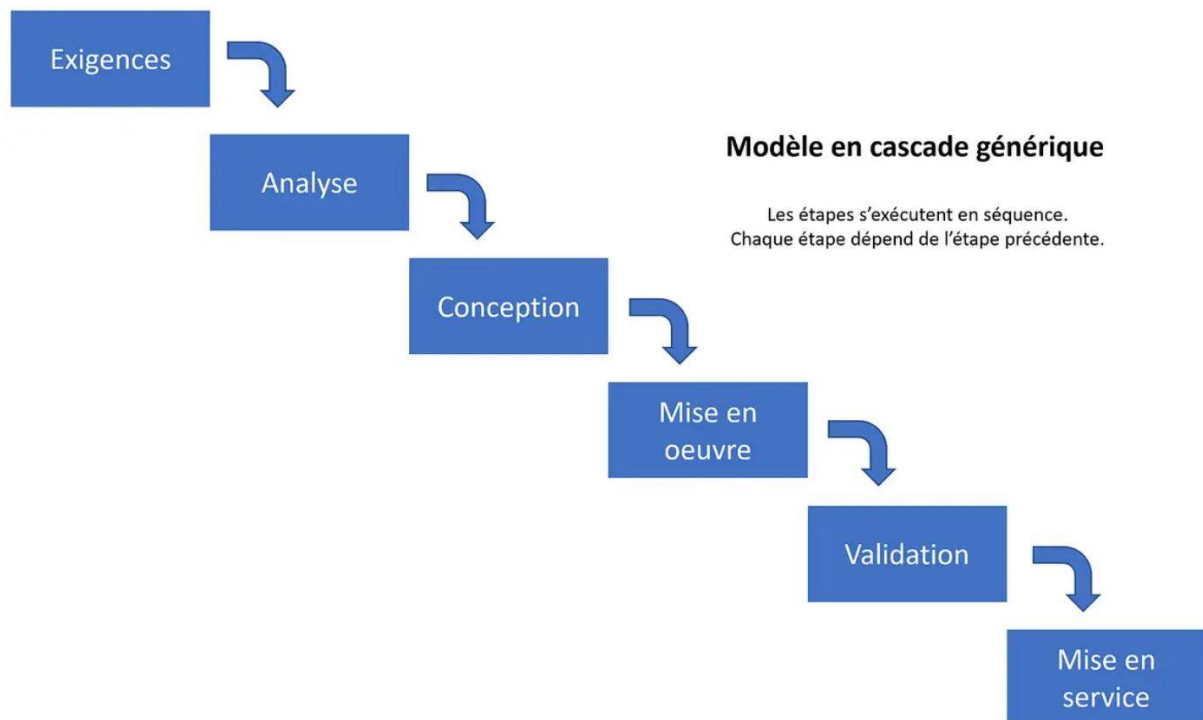
1.2- Découvrir les différentes méthodes de gestion de projet

1.2.1 - Les méthodes de gestion de projet traditionnelles

La méthode en cascade

Elle permet de simplifier la gestion du projet au travers d'un processus strict et séquentiel. Le modèle en cascade, appelé Waterfall en anglais, tel qu'appliqué aux projets, est une approche linéaire et séquentielle des différentes phases et activités du projet nécessaires à la livraison du ou des livrables.

Un modèle en cascade générique ressemble à ceci :



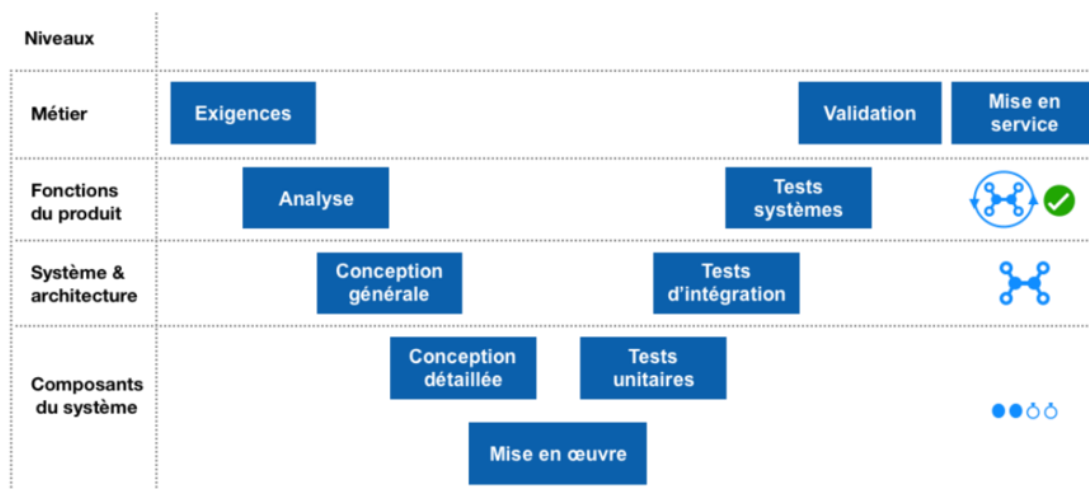
Le modèle générique comporte six étapes.

- **Étape des exigences** : définition et la description des exigences.
- **Analyse** : planification, analyse et spécification des besoins
Chaque projet logiciel commence par une phase d'analyse comprenant une étude de faisabilité et une définition des besoins. Les coûts, le rendement et la faisabilité du projet logiciel sont estimés lors de l'**étude de faisabilité**. Celle-ci permet de créer un cahier des charges (une description grossière des besoins), un plan de projet, une budgétisation du projet et, le cas échéant, un devis pour le client.
- **Conception** : conception et spécification du système
La phase de conception sert à l'élaboration d'un concept de résolution concret sur la base des besoins, des tâches et des stratégies déterminées au préalable. Au cours de cette phase, les développeurs élaborent l'**architecture logicielle** ainsi qu'un **plan de construction détaillé du logiciel** et se concentrent ainsi sur les éléments concrets tels que les interfaces, les frameworks ou les bibliothèques. Le résultat de la phase de conception inclut un document de conception avec un plan de construction logicielle, ainsi que des plans de test pour les différents éléments.

- **Mise en œuvre** : programmation et tests des modules
L'architecture logicielle élaborée pendant la phase de conception est réalisée lors de la **phase d'implémentation** qui comprend la **programmation du logiciel**, la **recherche d'erreurs** et les **tests de modules**. Lors de la phase d'implémentation, le projet de logiciel est transposé dans la langue de programmation souhaitée. Les différents composants logiciels sont développés séparément, contrôlés dans le cadre de **tests de modules** et intégrés étape par étape dans le produit global. Le résultat de la phase d'implémentation est un produit logiciel qui sera testé pour la première fois en tant que produit global lors de la phase suivante (test alpha).
- **Validation** : intégration du système, tests du système et de l'intégration
La phase de test comprend l'intégration du logiciel dans l'environnement cible souhaité. En règle générale, les produits logiciels sont tout d'abord livrés à une sélection d'utilisateurs finaux sous la forme d'une **version bêta** (bêta-tests). Il est alors déterminé si le logiciel répond aux besoins préalablement définis à l'aide des **tests d'acceptation** développés lors de la phase d'analyse. Un produit logiciel ayant passé avec succès les bêta-tests est prêt pour la mise à disposition.
- **Mise en service** : livraison, maintenance, amélioration

La méthode Cycle en V

Cette représentation associe les différentes étapes de développement du projet à une phase de validation concordante.



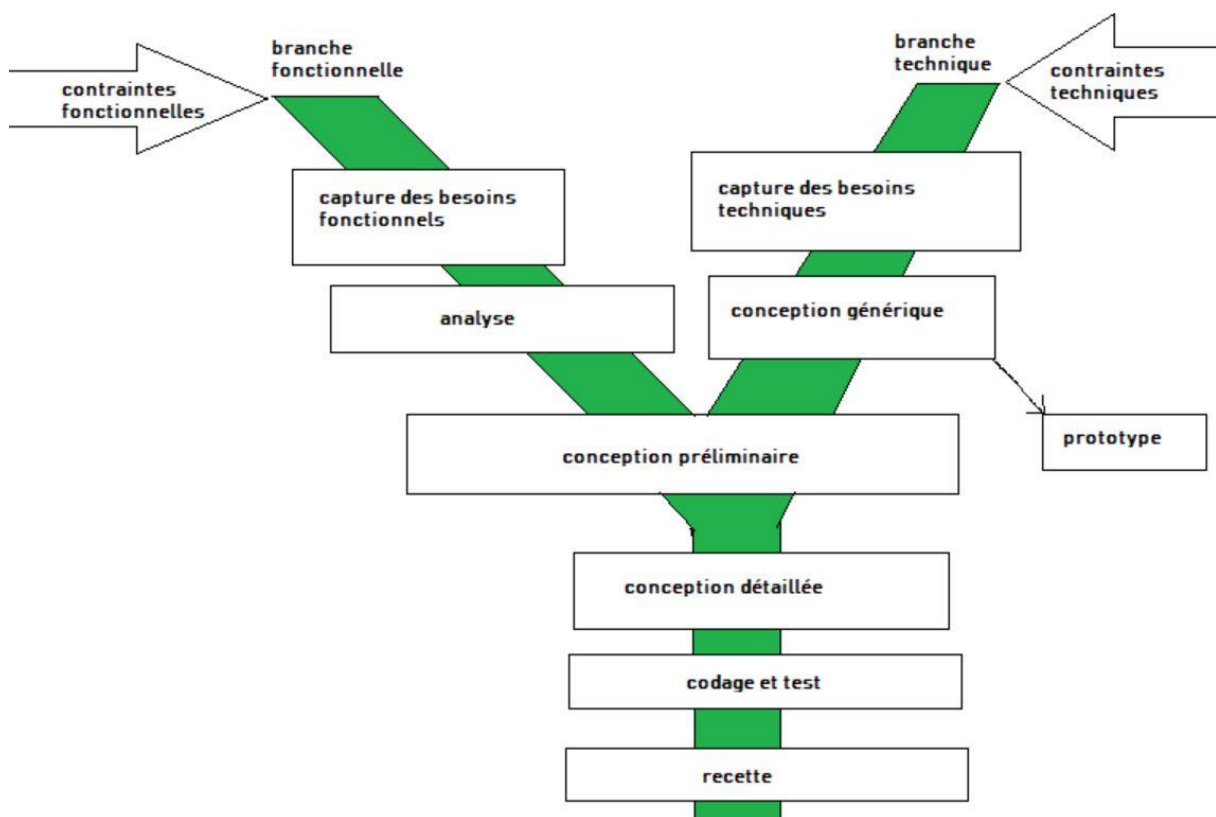
Les étapes du modèle sont alors :

- **Exigences** : les exigences font l'objet d'une expression des besoins. Le cas échéant, une étude de faisabilité peut être conduite avant d'engager les travaux.
- **Analyse** : il s'agit à partir de l'expression de besoin d'établir le cahier des charges fonctionnel ou les spécifications fonctionnelles
- **Conception générale**, aussi appelé conception architecturale ou conception préliminaire : il s'agit de concevoir le système qui doit répondre aux exigences et de définir son architecture, et en particulier les différents composants nécessaires ;
- **Conception détaillée** : il s'agit de concevoir chaque composant, et la manière dont ils contribuent à la réponse aux besoins;

- **Mise en œuvre** : il s'agit de réaliser chaque composant nécessaire. Pour les composants et systèmes logiciels, l'activité est essentiellement le codage ;
- **Test unitaire** : il s'agit de vérifier le bon fonctionnement et la conformité de chaque composant à sa conception détaillée ;
- **Test d'intégration** : il s'agit d'assembler le système à partir de tous ses composants, et de vérifier que le système dans son ensemble fonctionne conformément à sa conception générale ;
- **Test système** (anciennement « tests fonctionnels ») : vérification que le système est conforme aux exigences ;
- **Test d'acceptation** (également appelés « recette » dans le contexte de la sous-traitance) : validation du système par rapport à sa conformité aux besoins exprimés.

Le cycle en Y

2TUP est un processus unifié qui a pour but d'apporter une réponse aux contraintes de changement fonctionnelles et techniques qui s'imposent aux systèmes d'information. 2TUP propose un cycle de développement qui dissocie les aspects techniques des aspects fonctionnels. Il part du constat que toute évolution imposée au système d'information peut se décomposer et se traiter parallèlement, suivant un axe fonctionnel et un axe technique. Il distingue ainsi deux branches (fonctionnelle et technique) dont les résultats sont fusionnés pour réaliser le système. On obtient un processus de développement en Y.



Branche fonctionnelle ou « gauche »

Elle vise la capture des besoins fonctionnels et l'analyse des spécifications fonctionnelles de manière à déterminer ce que va réaliser le système en terme de métier. C'est ici, qu'on identifie et dégage toutes les fonctionnalités du système à réaliser.

Branche technique ou « droite »

Elle permet la capture des besoins non fonctionnels. Il s'agit essentiellement des contraintes que l'application doit prendre en compte comme par exemple les contraintes d'intégration, les contraintes de développement et les contraintes de performances.

Phase de réalisation

Cette phase est la fusion des deux précédentes et mène à la conception applicative et à la solution adaptée aux besoins des utilisateurs. Elle concerne les étapes de la conception préliminaire, la conception détaillée, le codage et les tests puis l'étape de recette.

1.2.2 - Les méthodes de gestion de projet agiles

La méthode Scrum

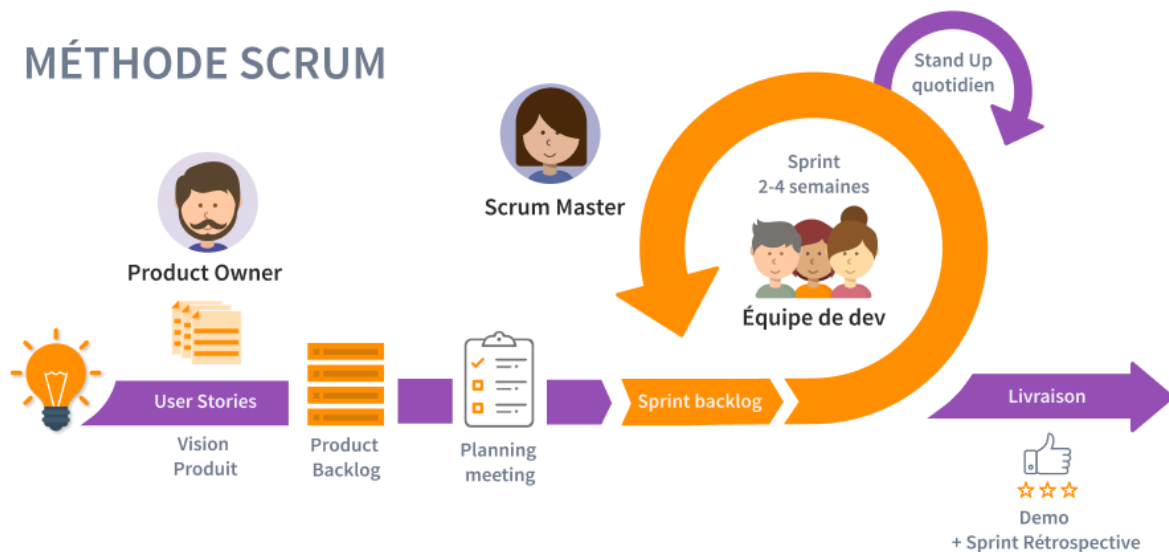
Scrum est un framework ou cadre de développement de produits complexes. Il est défini par ses créateurs comme un « **cadre de travail holistique itératif** qui se concentre sur les buts communs en livrant de manière productive et créative des produits de la plus grande valeur possible ». Scrum est considéré comme un groupe de pratiques répondant pour la plupart aux préconisations du manifeste agile.

La méthode Scrum repose sur une gestion de projet collaborative et un cycle de développement :

- **itératif** (répété plusieurs fois, de l'idée initiale à une version de plus en plus aboutie),
- **incrémental** (progressif, tâches après tâches),
- **adaptatif**.

Ainsi, les 3 piliers Scrum sont :

- la **transparence**, dans la communication et le suivi,
- **l'inspection** régulière pour détecter les écarts entre les objectifs et le travail réalisé,
- **l'adaptation**, pour un ajustement en permanence face aux contraintes.



Le **scrum** s'appuie sur :

- le **découpage d'un projet** en unités de temps courtes, nommées **sprints**, ceci afin d'éviter **l'effet tunnel**, dû à une planification tellement étendue dans le temps qu'on n'en voit pas le bout
- des **réunions régulières** pour faire le point

- **les rôles d'une équipe Scrum :**
 - une équipe technique (développeurs, architectes, designers, testeurs) ;
 - le **product owner**, qui a la vision du produit et s'assure de la bonne **traduction des attentes du client à l'équipe projet** ;
 - le **scrum master**, ou maître de mêlée, qui est le chef d'orchestre, le coordinateur de l'**équipe agile**, dont il fait partie intégrante.

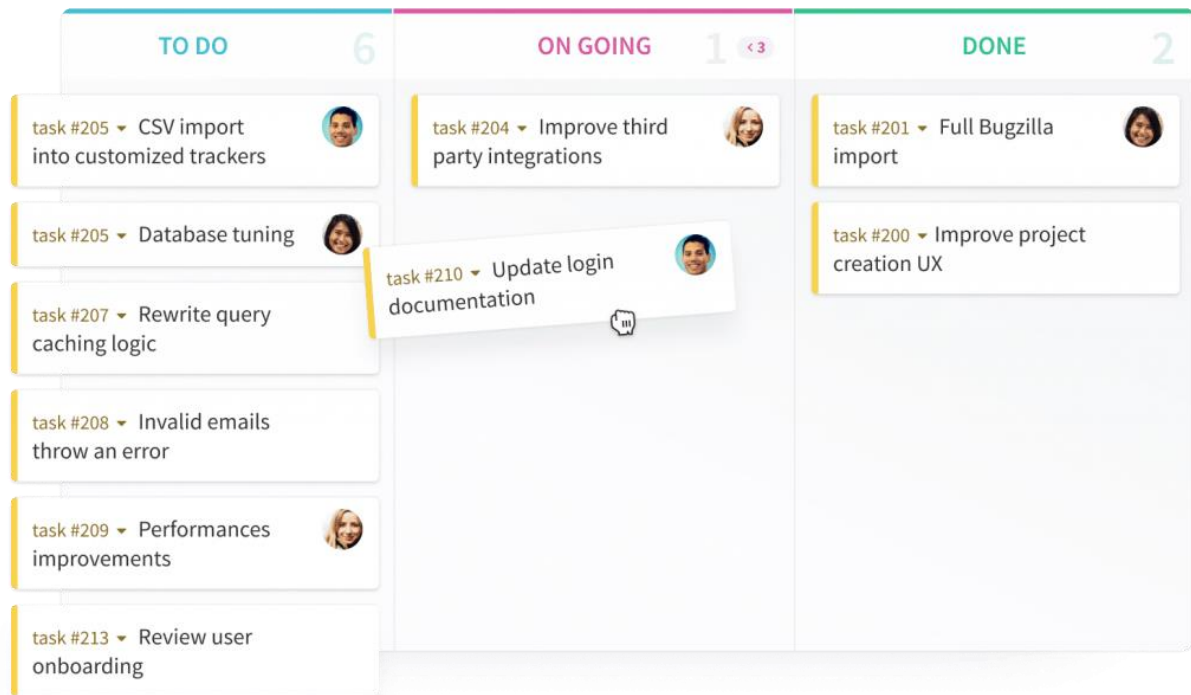
Glossaire

- **product owner (« directeur de produit ») :**
Personne ayant la responsabilité de produire et de maintenir à jour le carnet de produit. C'est lui qui détermine les priorités et qui prend les décisions d'orientation du projet ;
- **scrum master (« chef de mêlée ») :**
Membre de l'équipe dont l'objectif principal est de la protéger des perturbations extérieures. Il est complètement transparent pour la communication entre l'équipe et les clients et n'a aucun pouvoir hiérarchique sur l'équipe. C'est en revanche un facilitateur pour les problèmes non techniques de l'équipe ;
- **product backlog (« carnet du produit ») :**
Liste des fonctionnalités, des fonctions, des exigences, des améliorations et des correctifs qui sont nécessaires à l'évolution du produit ; celui-ci est dynamique sur tout le cycle de vie du produit.
- **sprint (« sprint ») :**
Nom d'une itération dans Scrum. Cette itération dure 1 mois maximum en théorie, mais en pratique entre 2 et 4 semaines. Pendant une itération, l'équipe doit développer la liste d'éléments du carnet de produit qui a été définie au début du sprint ;
- **sprint backlog (« carnet de sprint ») :**
Liste des tâches à accomplir pendant un sprint. Elles correspondent à la réalisation des éléments de carnet de produit affectés au sprint ;
- **daily scrum (« mêlée quotidienne ») :**
Réunion quotidienne de quinze minutes maximum pour faire le point sur ce qui a été fait depuis la dernière mêlée, ce qu'il est prévu de faire jusqu'à la prochaine et quels sont les obstacles rencontrés durant le travail ;

La méthode Kanban

La méthode du Kanban repose sur un système à flux tirés, qui démarre dès que le client passe une commande. Il s'agit donc de produire le produit uniquement quand la demande est faite côté consommateur.

L'approche Kanban permet de contrôler visuellement le flux de travail, et d'obtenir toutes les informations nécessaires pour éventuellement améliorer les processus par la suite. Cette méthode permet aussi de suspendre le processus de production si nécessaire afin d'apporter une solution à un problème bloquant ou en cas d'urgence.



1.2.3 - Cycle en V vs. Méthodes agiles

Lors de l'utilisation d'une approche "**traditionnelle**", le projet, ses fonctionnalités et sa finalité sont clairement définis à l'avance. Cette méthodologie repose sur l'utilisation d'un **processus strict**, la rédaction d'une **documentation détaillée** et une implication plus faible du client. Elle consiste à **définir l'ensemble des fonctionnalités** du projet, les spécifier de façon détaillée, les développer puis les tester avant validation et mise en service.

Le développement d'une application avec une **méthode dite agile** est très différent. L'utilisation d'une méthode agile implique une **methodologie** plus légère, des tâches plus petites, une **livraison rapide des incréments** et une communication permanente entre le donneur d'ordre et l'équipe de développement. Le maître mot est la **flexibilité**, tant en termes de planification que d'incrémentations des fonctionnalités.

On privilégiera plutôt la **méthode classique** lorsqu'on a une **idée précise du projet**, avec un **planning bien détaillé** et où on a anticipé tous les risques possibles.

Quant à la **méthode Agile**, on la choisira plutôt pour **les gros projets**, celle-ci permettant une meilleure adaptabilité, visibilité et gestion des risques. On privilégiera également la méthode Agile pour **les projets où il n'y a pas de documents détaillés**, ou quand vous sentez que votre client est indécis. Le **client pourra alors voir l'évolution du projet et l'adapter à ses besoins sans** pour autant vous obliger à recommencer tout le travail que vous avez fourni depuis le début.

2- Planifier un projet

2.1- Analyser le cahier des charges

Un cahier des charges (parfois abrégé en CDC) est un document qui doit être respecté lors de la conception d'un projet. C'est un document qui permet d'expliquer toutes les spécificités, les attentes et les contraintes d'un projet aux freelances ou aux agences.

2.1.1 - Compréhension des besoins client

L'analyse des besoins d'un client consiste à identifier les attentes et les désirs de celui-ci pour une offre de produit ou de service. Ce processus analytique doit faire partie de toute étude de marché. Concrètement, l'analyse des besoins client permet de développer les concepts produits, de créer des offres qui sont en adéquation avec les attentes des consommateurs et d'estimer la « Valeur Client » sur son marché.

Contextualiser le projet

Idéalement, tout projet commence avec l'expression d'un ou de plusieurs besoins (qui seront formulés en objectifs par la suite).

Selon qui expriment ce ou ces besoins et de quelle manière, vous (le chef de projet) allez devoir gérer des scénarios potentiellement très différents. La première différenciation à faire est celle des projets internes et externes.

Les projets internes

Dans certains cas, vous évoluerez au sein même de l'organisation qui a besoin du projet, en tant qu'employé. Vous êtes peut-être même à l'origine du projet dont vous avez identifié le besoin.

Auquel cas avant de crier victoire sachez que faire partie de l'organisation ne suffit pas à garantir la réussite du projet. D'autant plus si votre rôle premier dans cette organisation n'est pas de gérer ledit projet et que vous allez devoir faire ça en plus de vos autres responsabilités, il y a des risques pour que le projet tombe vite à l'eau ou pire, qu'il dérive indéfiniment.

Les projets externes

Dans d'autres cas, une organisation peut identifier un besoin mais ne pas lancer un projet interne pour autant. Cela peut être dû à un manque de ressources internes ou un éloignement du cœur de métier. Dans ce cas, cette organisation peut être commanditaire du projet et travailler avec une agence ou une autre forme de prestataire.

Là encore, plusieurs scénarios peuvent se présenter. Le commanditaire peut contacter directement des prestataires, auquel cas l'expression de besoins se fait à huis clos. Sinon le commanditaire peut décider de faire un **appel d'offres** ou un **avis de mise en compétition**. L'appel d'offres consiste à publier une expression de besoins sur un site spécialisé et inviter les prestataires à postuler pour mieux les étudier et sélectionner le meilleur.

Analyser et recueillir des besoins

Comme je vous le disais plus tôt, tout projet commence avec une expression ou l'identification d'un besoin et celle-ci peut être plus ou moins complète. Votre premier travail en tant que chef de projet, c'est donc d'analyser ce ou ces besoins, les compléter si nécessaire et les reformuler sous forme d'objectifs et de **livrables**.

Dans le cas du projet de l'hôtel Paradis, à première vue les besoins semblent relativement bien cernés bien que peu détaillés, et nous pouvons donc assez facilement les traduire en livrables :

Besoin	Livable(s) potentiel(s)
Développer une présence en ligne	Site vitrine, plan web marketing
Moderniser une image de marque	Plateforme de marque avec charte graphique et logo
Permettre aux clients de réserver en ligne	Système de réservation

Lors de vos échanges exploratoires, au-delà d'un relationnel irréprochable, vous allez porter une attention particulière à 4 choses :

- les besoins explicites du client
- les besoins implicites du client
- les livrables potentiels
- et sa grille de lecture supposée

Besoins explicites

Par "besoins explicites", comprenez les besoins exprimés clairement, sans ambiguïté et sur lesquels il y a un consensus. Pour savoir si c'est le cas, essayez de les reformuler et analysez la réaction de votre interlocuteur.

Dans notre exemple, le besoin de développer la présence en ligne de l'hôtel Paradis à travers la création d'un site web est un besoin assez explicite.

Besoins implicites

Si vous avez regardé la vidéo de ce chapitre, vous avez vu que la clientèle de l'hôtel est principalement étrangère, le site devra donc impérativement être multilingue. Ce besoin, en revanche, n'a pas été exprimé clairement par le commanditaire. On dira donc que c'est un besoin implicite et une contrainte qui devra être prise en compte dès le lancement du projet.

Besoins explicites	Besoins implicites	Livable(s) potentiel(s)
Développer la présence en ligne	Rendre le site accessible dans 4 langues : français, anglais, chinois et russe.	Site web (multilingue)
Moderniser l'image de marque	Conserver le nom et le logo de l'établissement.	Adaptation de la charte graphique
Système de réservation	Permettre au staff de l'hôtel de gérer ses prix et ses réservations en back-office.	Système de réservation (et de gestion)

Parfois certains besoins du commanditaire peuvent paraître contradictoires, comme ici, le souhait de moderniser son image de marque tout en conservant le logo de l'établissement. À vous de juger s'il est préférable de confronter le commanditaire à ses contradictions ou de trouver docilement un compromis.

Décrypter une grille de lecture

La grille de lecture du commanditaire est l'ensemble des critères à partir desquels il/elle va sélectionner l'agence candidate. Certains donnent plus d'importance à la compréhension de la problématique rencontrée ou au budget proposé alors que d'autres privilégient l'esthétique des livrables ou le relationnel avec le prestataire.

Que cette grille soit formalisée ou non pour votre commanditaire, il est essentiel que vous la compreniez pour pouvoir anticiper ses priorités et ainsi rédiger votre proposition commerciale en conséquence.

Pour ce faire, listez simplement les critères de sélection qui reviennent le plus souvent et donnez-leur un ordre de priorité d'après ce que vous pensez comprendre de votre commanditaire.

Voici un exemple de grille de lecture supposée :

Critère de sélection	Importance
Compréhension de la problématique	2
Esthétique	3
Solution technique	4
Méthodologie	5
Budget	1

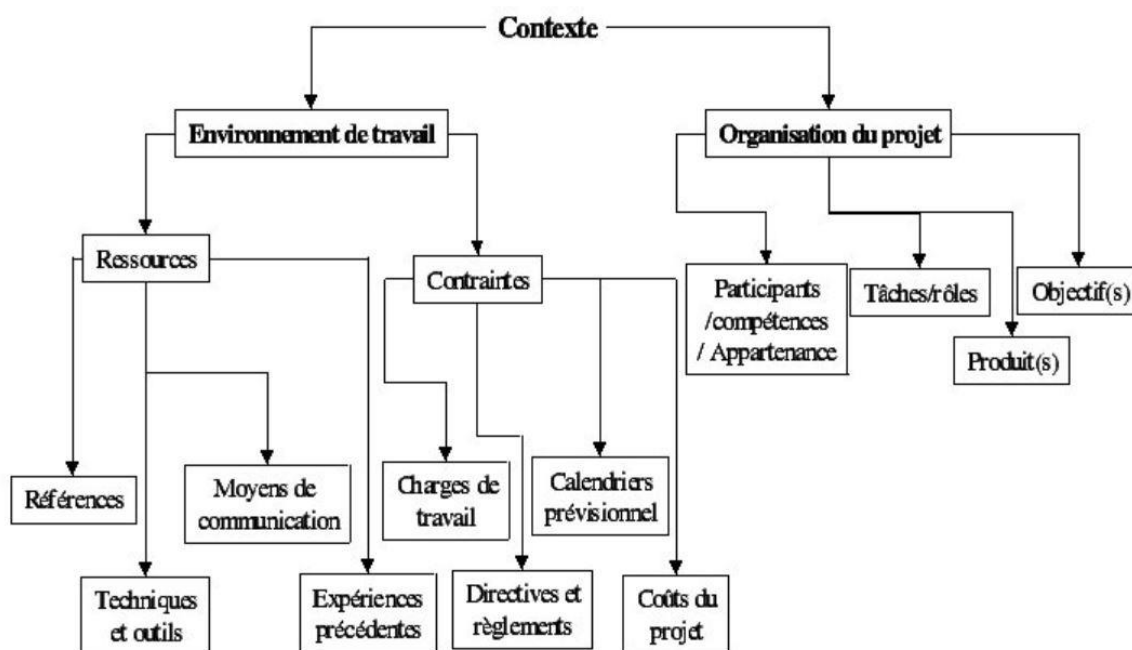
À la fin de ce premier échange vous devriez avoir suffisamment d'éléments pour passer à l'étape suivante : le cadrage du projet. Toujours en vue de formuler une proposition commerciale, vous allez consulter votre équipe pour élaborer une solution, étudier la faisabilité du projet et donc collecter les informations nécessaires à la planification et la budgétisation.

2.1.2 - Contexte du projet

Le contexte d'un projet correspond à l'ensemble des informations qui caractérisent un projet et lui donnent de la profondeur. Il peut s'agir de l'histoire et de l'origine du projet, d'informations sur le contexte réglementaire, culturel, économique, concurrentiel et social dans lequel évolue la société, ou encore de son environnement de travail.

En effet, un projet s'inscrit toujours dans un environnement social, économique et technique complexe, avec des éléments qui peuvent être inter-dépendants. Pour mettre toutes les chances de son côté, un bon chef de projet devra mener une analyse du contexte de projet, en plus de l'analyse du périmètre projet ainsi que de ses enjeux et de ses objectifs.

L'ensemble des éléments qui composent le contexte d'un projet doivent être notifiés dans la note de cadrage de projet, qui est le document de référence décrivant les tenants et aboutissants du projet.



2.1.3 - Périmètre du projet

Le périmètre du projet représente tous les éléments nécessaires à la réalisation d'un projet, notamment les tâches, les délais et les ressources. La gestion du périmètre du projet correspond donc au processus de supervision et de régulation de tous ces éléments afin que vous puissiez achever votre projet dans les délais et le budget impartis.

Le périmètre de projet décrit en détail les livrables du projet. Il permet aussi une compréhension globale et commune du périmètre par les parties prenantes. Il contient aussi les exclusions explicites qui aide à gérer les attentes et désirs des parties prenantes.

Il permet à l'équipe du projet d'effectuer une planification plus détaillée, guide son travail lors de l'exécution et fournit une référence de base pour évaluer si les demandes de modifications ou des exigences supplémentaires sont comprises ou non dans le périmètre du projet.

2.1.4 - Détection des risques liés à la nature du projet

La veille sécurité ou risk intelligence, même si elle varie selon le secteur d'activité, peut être définie par l'anticipation et la détection de tous les risques possibles pouvant affecter votre organisation et la création de plans d'urgence, de protocoles de réponse et de stratégies de communication.

Découvrons maintenant pourquoi il est essentiel de mettre en place une stratégie rigoureuse de risk intelligence :

- Acquérir une connaissance précise de votre environnement et comprendre où vous êtes vulnérable.
- Représenter ou définir clairement vos risques (ce processus peut également être appelé «cartographie des risques», et il comprend généralement la mise en place de vos risques sur un graphique).
- Surveiller de près vos risques (ou les facteurs qui pourraient se transformer en risques).
- Prendre des mesures stratégiques pour éviter les risques en trouvant des moyens de les prévenir ou en améliorant votre temps de réponse et votre efficacité lorsqu'un risque devient réalité.

Gestion et détection des risques :

1. Faites un brainstorming sur tous les **risques possibles** pouvant vous affecter, votre organisation, vos partenariats, votre secteur d'activité, votre pays, etc. Impliquez le plus grand nombre de collaborateurs d'autres business units car ils possèdent une excellente connaissance de votre environnement.
2. Une fois que vous avez identifié tous les risques possibles, déterminez comment ces risques pourraient vous affecter. Il faut associer à chaque risque des **typologies de risque**.
3. Évaluez tous les types de risques selon la **probabilité de survenance et la gravité**
4. Écrivez les différents types de scénarii afférents aux risques identifiés, en les graduant selon la facilité de mise en place, les **hypothèses** d'évolution-réaction favorable et défavorable.
5. Créez des plans pour atténuer ces risques et des plans pour répondre à ces risques dans le cas où vous n'êtes pas en mesure de les arrêter efficacement.
6. Maintenant, définissez vos paramètres de monitoring pour surveiller ces risques.

Risque				
Très probable				
Probable				
Peu probable				
Improbable				
Gravité	Faible	Moyenne	Grave	Très grave

2.1.5 - Proposition des solutions possibles

Résoudre un problème revient à s'appuyer sur une méthode et des outils adaptés. Nous vous proposons une démarche et une suggestion d'outils à utiliser pour chaque étape.

Les 5 étapes pour résoudre un problème

Définir le problème à traiter

La première étape est de bien intégrer les tenants et aboutissants de la problématique à résoudre. Et surtout de **ne confondre le problème avec ses symptômes**.

Il convient de poser le problème pour **bien comprendre toutes ses dimensions** et recueillir le maximum d'informations qui faciliteront la recherche de causes.

Selon l'ampleur du problème, **impliquez des collaborateurs de services différents** pour élargir le point de vue sur la question.

Identifier les causes

Voici une étape clé, la recherche des causes impactantes. Un point important est de **bien séparer la recherche de la sélection**. La première demande d'être exhaustif en listant toutes les causes possibles ayant une influence sur le problème. La seconde, plus analytique, a pour objectif d'identifier celles qui ont un poids suffisamment significatif pour être traitées.

Comme souvent, le diable est dans les détails. Au premier regard, l'origine du problème peut paraître évidente. Pourtant... **il faut toujours se méfier des évidences**.

Pour la recherche, dressez une liste en utilisant des méthodes comme le brainstorming ou le QQQQCP. Ne négligez rien. Encore une fois, **une cause d'apparence anodine peut très bien occuper un rôle central** dans la problématique traitée.

Pour la sélection, évaluez et classez chaque option. Le diagramme d'Ishikawa fait partie des outils tout indiqués pour cette tâche.

Trouver une solution

Comme précédemment, **cette étape se divise en 2 phases : la recherche et la sélection** à l'aide d'outils comme le brainstorming, puis en utilisant des méthodes d'aide à la décision, comme la matrice de décision.

Le choix de la solution prend en compte **différents critères** comme la facilité et la rapidité de mise en œuvre, le coût, les compétences devant être mobilisées, les risques d'effets de bord. Au final, **ce n'est pas forcément celle qui a plus d'impact qui sera retenue**. Il vaut parfois mieux retenir un ensemble de solutions secondaires faciles et rapides à mettre en œuvre plutôt qu'une seule, avec certes un fort impact, mais lourde et coûteuse à déployer.

Dans certains cas, la décision peut être validée par un test avant sa généralisation. L'expérimentation est toujours un excellent moyen pour vérifier la pertinence d'une décision.

Lancer les actions : mettre en œuvre la solution retenue

Une fois le choix arrêté, il est temps de passer à l'action. **Une bonne préparation maximise la qualité du déploiement**. Tout commence en fixant des objectifs pour s'assurer de l'efficacité des actions.

Suivre l'efficacité de la solution et de sa mise en œuvre

Le lancement d'actions ne peut exister sans un suivi précis à travers un tableau de bord.

Attention à **ne pas confondre l'efficacité des actions vis-à-vis des objectifs** qui leur ont été assignés - et qui garantissent leur bonne mise en œuvre - **avec l'efficacité de la solution à résoudre le problème initial** .

Dans le premier cas, vous évaluez la mise en œuvre d'une solution ; dans le second, la pertinence de son choix pour résoudre le problème.

2.2- Préparer le projet

2.2.1 - Répartition de l'ensemble des fonctionnalités en tâches

Une **mission** est un "super-objectif" qui va **nécessité d'être décomposé en tâches**, c'est-à-dire en actions qui seront réparties entre plusieurs coéquipiers, avec des délais imbriqués. Chaque **tâche** constitue un **objectif intermédiaire dont la réalisation devra être suivie**.

Une **tâche** est une **activité réalisée par un membre de l'équipe projet pour contribuer à la production d'une solution constitutive** du produit du projet.

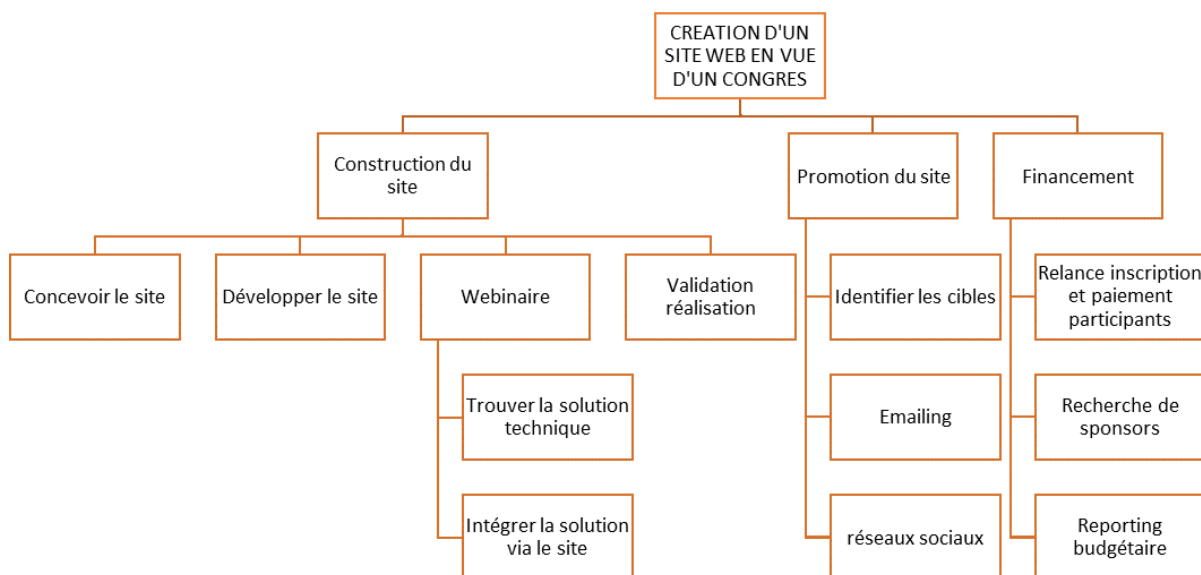
Dans un travail en équipe, il faut donc que les tâches soient réparties, voire déléguées pour certaines.

La répartition des tâches et le planning de réalisation sont donc fondamentaux. Quelle que soit l'ampleur de la mission, dès lors qu'elle mobilise plus de deux collaborateurs, il vous faut :

- établir l'organigramme des tâches ;
- définir les niveaux de responsabilités ;
- centraliser tous les documents et procédures d'accompagnement du projet ;
- mettre en place un suivi.

OT : l'Organigramme des Tâches

L'**organigramme des tâches** est la **décomposition arborescente de l'ensemble des travaux à réaliser dans le cadre du projet**. Les niveaux les plus bas dans l'arborescence sont appelés lots de travaux. Un lot de travaux définit un livrable, c'est-à-dire le produit de la tâche accomplie : par exemple la construction d'un site web, l'identification et le démarchage de sponsors, une étude de marché, la visite et l'audit de points de vente.



La Matrice des Responsabilités (RACI)

La **matrice des responsabilités** est un outil qui vous **permet de visualiser les 4 niveaux de responsabilité**. Soit votre outil collaboratif intègre ces niveaux, soit vous utilisez un tableur comme Excel intégré à votre plateforme collaborative. Voici les quatre niveaux :

- (R) Responsable : il est garant de la réalisation du lot de travaux. Il peut déléguer les tâches.
- (A) Acteur : il réalise le lot de travaux.
- (C) Consulté : c'est un expert, consulté pour la réalisation d'une tâche ou une validation technique.
- (I) Informé : c'est un coéquipier à qui l'on diffuse des documents, que l'on tient au courant de l'avancement, convie aux réunions, parce que le lot de travaux a un impact sur sa mission.



Responsable	Accountable	Consulted	Informed
<i>Celui qui réalise la tâche</i>	<i>Celui qui approuve la tâche</i>	<i>Celui qui est consulté</i>	<i>Celui qui doit être informé</i>
Qui ? Personne qui va exécuter la tâche : elle en est responsable.	Qui ? Personne qui va approuver la tâche : elle en est l'autorité.	Qui ? Personne qui va être consultée dans l'exécution de la tâche : elle est consultée.	Qui ? Personne qui sera informée lorsque la tâche est finie : elle est informée.
Sa mission : Réaliser la tâche qui lui a été attribué.	Sa mission : Veiller à l'exécution correcte de la tâche réalisée par le(s) responsable(s) et approuver l'activité faite.	Sa mission : Contribuer avec des conseils et opinions à ce que la tâche soit effectuée le plus efficacement possible.	Sa mission : Être tenue à jour sur les progrès réalisés, souvent à l'issue de la tâche ou du livrable.
Particularité : Il peut y avoir plusieurs responsables pour une même tâche, chacun fait une partie de la tâche.	Particularité : Une autorité par tâche.	Particularité : Il peut y avoir plusieurs personnes consultées et ce sont souvent des experts.	Particularité : Elle n'intervient pas activement dans la réalisation de la tâche.

2.2.2 - Estimation de la durée de réalisation de chaque tâche

Il est important, lors de la phase d'estimation, de faire le maximum pour obtenir des estimations réelles du travail nécessaire pour réaliser la tâche, et non pas une estimation grossière incluant une « sécurité ». Les marges (qui sont bien entendue nécessaires si on ne veut pas exploser le planning à tous les coups...) seront appliquées plus tard, de manière globale au projet.

Calcul de la durée (Méthode 1) :

La pratique répandue en management de projet est d'estimer trois durées pour chaque tâche ; Optimiste, Pessimiste et Plus probable.

La durée moyenne de la tâche est alors estimée à (Optimiste, + 4 x Plus Probable + Pessimiste) / 6.

Ce calcul est connu sous le nom d'équation PERT et fournit une moyenne pondérée.

Calcul de la durée (Méthode 2) :

Durée = (Travail / Capacité) + temps non travaillé

- La durée est le temps écoulé (en jours, en heures...) entre le début de la tâche et la fin de la tâche.
- Le travail représente la charge (en heures, en jours...) de travail nécessaire à la réalisation de la tâche par une seule personne occupée à 100% de son temps de travail sur la tâche.
- La capacité correspond au nombre de personnes affectées à la tâche.
- Le temps non travaillé est composé des temps de pause, des jours fermés (week-end, jours fériés, etc.), des nuits, etc.

2.2.3 - Ordonnancement des tâches

La théorie de l'ordonnancement est une branche de la recherche opérationnelle qui s'intéresse au calcul de dates d'exécution optimales de tâches. Pour cela, il est très souvent nécessaire d'affecter en même temps les ressources nécessaires à l'exécution de ces tâches.

L'ordonnancement se déroule en trois étapes :

- **La planification** : qui vise à déterminer les différentes opérations à réaliser, les dates correspondantes, et les moyens matériels et humains à y affecter.
- **L'exécution** : qui consiste à la mise en œuvre des différentes opérations définies dans la phase de planification.
- **Le contrôle** : qui consiste à effectuer une comparaison entre planification et exécution, soit au niveau des coûts, soit au niveau des dates de réalisation.

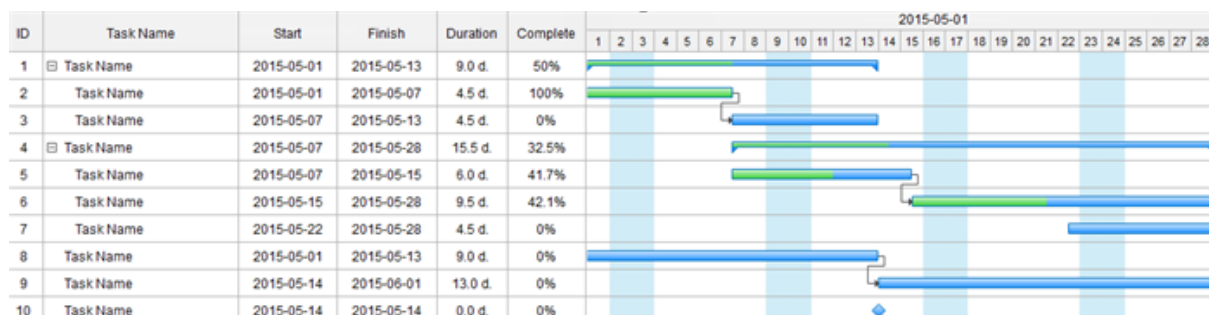
Les méthodes d'ordonnancement permettent d'élaborer un graphe qui représente l'ensemble des tâches composant le projet ainsi que les liens qui existent entre elles. Sur le graphe, apparaissent également la durée de chaque tâche, la date à laquelle elle peut débuter au plus tôt et au plus tard.

Il existe trois méthodes d'ordonnancement :

- le diagramme de Gantt
- la méthode MPM (Méthode des potentiels Métra)
- le PERT

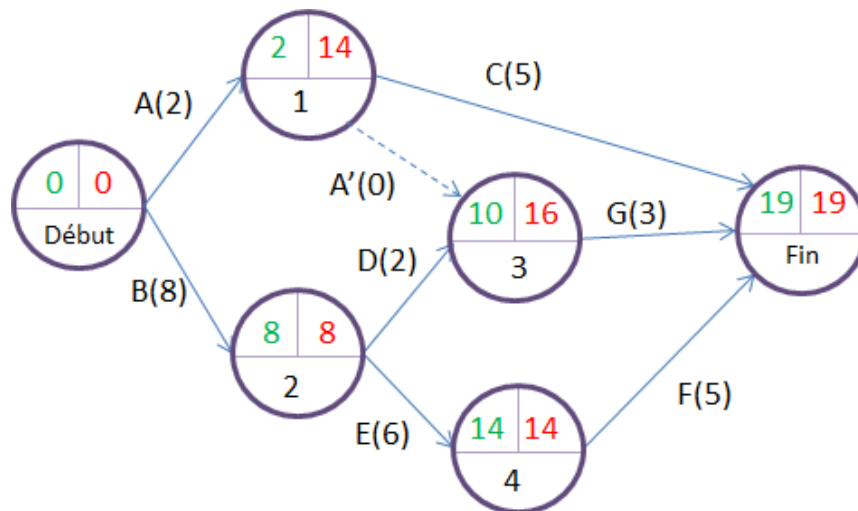
Le diagramme de Gantt

Le diagramme de Gantt est un **diagramme graphique pour la planification de projet**. Dans le diagramme de Gantt, les projets sont divisés en petites tâches et tâches concrètes au fil du temps. Dans le diagramme de Gantt, chaque activité (tâche) est représentée par une barre. La position et la longueur de la barre reflètent la date de début, la durée et la date de fin de l'activité. Les jalons sont représentés par des triangles, et les lignes liées aux tâches sont toujours indiquées par des flèches.



Le diagramme PERT

Avec le diagramme PERT, la relation d'ordre de traitement de chaque tâche dans un projet complexe est exprimée sous forme de réseau ou d'organigramme. Également, clarifiez le chemin (chemin critique) qui n'a pas assez de temps pour terminer la tâche et travaillez du début à la fin du projet et examinez la faisabilité du plan si le projet peut être achevé d'ici la fin prévue de la construction.



2.2.4 - Chemin critique

En gestion de projet, tout dépassement de la date de fin prévue peut entraîner des pénalités financières, mais également la dégradation des relations avec votre client. Le respect des délais est donc essentiel et la méthode du chemin critique (critical path method ou CPM en anglais) est un outil efficace pour y parvenir.

Le **chemin critique** désigne **l'ensemble des activités à accomplir afin que le projet soit terminé à la date définie**. La méthode du chemin critique **permet de savoir combien de temps prendra chaque tâche avant de finir le projet**. Vous utilisez ainsi cette information pour déterminer la date de fin du projet. Si une tâche prend plus de temps que prévu, alors la date de fin du projet est repoussée. La durée de certaines tâches peut être étendue sans pour autant affecter le projet.

Les tâches du chemin critique sont appelées « tâches critiques » car elles sont indispensables à la réussite du projet. En conséquence, elles ne doivent subir aucun retard, sinon l'intégralité du projet sera retardée. Leur marge de manœuvre est donc nulle.

2.2.5 - Echancier et la chronologie des tâches

Que vous travailliez sur un projet en cascade ou sur un projet agile, la conception d'une chronologie de projet initiale soulève des questions qui ne se posent dans aucune autre discipline.

D'une part, les projets en cascade se caractérisent le plus souvent par une structure et un calendrier peu modulables, et sont généralement construits autour d'estimations en amont d'excellente qualité. Les projets agiles, en revanche, dépendent de la souplesse et de phases de sprint parfois complexes : les cadres doivent donc essayer d'anticiper les inévitables

vagues de modifications. Quel que soit le type de projet qui vous concerne, la création d'une chronologie réalisable est une étape décisive lors la préparation des projets.

La liste de vérification ci-dessous peut aider les chefs de projet et les chefs d'entreprise à bien se préparer, que ce soit pour le développement de la première tâche d'un projet, la planification des différentes étapes de validation ou l'atténuation des risques du projet.

1. Comment diviser l'objectif fixé en tâches individuelles ?

La première étape de la construction d'une chronologie complète est la division du projet en plusieurs segments. Pour ce faire, commencez par la fin : à partir de votre objectif final, établissez les tâches individuelles qui devront être effectuées pour atteindre cet objectif. Commencez par établir les points clés, puis affinez la chronologie en plusieurs fois pour obtenir des tâches de plus en plus précises entre ces points clés.

En fonction de la méthode utilisée pour votre projet, ces étapes peuvent être groupées en séquences qui deviendront les sprints. La structure des projets en cascade sera principalement basée sur des points clés plus généraux.

2. Combien de temps attribuer à chaque tâche ?

Une fois que vos tâches sont intégralement planifiées, vous pouvez calculer une estimation du temps nécessaire à la réalisation de ces tâches. En fonction de la plateforme sur laquelle vous construisez votre chronologie, vous pouvez peut-être générer un diagramme de Gantt traditionnel, ce qui est positif. La « chronologie de gestion de projet » est souvent synonyme de « diagramme de Gantt », même si toutes les chronologies de projet ne sont pas nécessairement sous la forme d'un diagramme de Gantt.

Savoir à l'avance le temps que prendra chaque tâche peut également aider les chefs de projets agiles à estimer le nombre total de sprints nécessaires au projet. Même si la date de livraison de fin de projet est le facteur le plus important, assurez-vous de commencer à rassembler les estimations de temps de travail auprès des équipes responsables des différentes tâches. Cela vous aidera par la suite à comparer les besoins en matière de chronologie du projet avec sa mise en pratique réelle.

3. Quel est le livrable clé de chaque tâche ?

Les deux premières étapes concernaient principalement la création et l'anticipation de la chronologie du projet proprement dite. À partir de cette troisième étape, le processus concerne davantage les contrôles et la planification logistique. Il s'agit de matérialiser le calendrier, avec les éléments concrets à prendre en compte pour le mettre en œuvre.

Commencez par identifier les livrables principaux pour chaque tâche ou chaque sprint avec la même méthode qu'à l'étape 1, en affinant progressivement la chronologie pour identifier des livrables de plus en plus précis. Plus la liste de ces livrables est précise, plus vos équipes auront d'objectifs spécifiques à atteindre.

4. De quels éléments chaque tâche dépend-elle ?

S'il est une réalité universelle à laquelle tous les chefs de projet sont confrontés, c'est que chaque projet est invariablement amené à subir des modifications. Les ressources, les chronologies et les cahiers des charges sont souvent fluctuants, et les chefs de projets ne peuvent pas tout planifier à l'avance.

Toutefois, ils peuvent identifier des tâches ou des objectifs qui dépendent d'autres étapes du processus. De cette façon, toutes les parties connaissent les effets en aval des changements en amont.

La plupart des outils de gestion de projet permettent aux utilisateurs de lier une tâche à une autre. Ce faisant, il est plus facile de simuler les répercussions d'un retard sur le calendrier. Pour les projets en cascade, cette étape est particulièrement importante en raison de la rigidité des chronologies. Toutefois, pour les projets agiles, identifier les interdépendances peut aider les chefs de projets à mieux concrétiser le travail à fournir pour chaque sprint.

5. Combien de temps et quelles ressources sont disponibles pour mener à bien chaque tâche ?

Lors de l'étape 2, vous avez déterminé le temps nécessaire pour terminer les tâches principales de votre projet. À présent, vous devez utiliser ces estimations de temps et de ressources obtenues auprès de vos chefs d'équipe. Savoir que la réalisation d'une tâche donnée doit prendre 5 jours est une chose, mais gérer un chef qui ne peut pas fournir cinq jours de travail d'équipe représente une complication. Comparer ces points de données vous aidera à identifier les difficultés potentielles ou les sous-traitants extérieurs dont vous aurez besoin.

6. Qui est responsable de l'exécution de chaque tâche ?

L'affectation d'individus ou d'équipes à des tâches spécifiques fait partie du processus d'allocation des ressources disponibles. Cette étape ne se limite pas à une prise de notes avisée, car elle aide les intervenants et les dirigeants à avoir une vue d'ensemble du niveau hiérarchique de l'entreprise à mobiliser pour mener à bien le projet.

Il est préférable d'affecter le plus tôt possible les individus à leurs livrables respectifs. Les chefs d'équipes seront plus enclins à examiner l'estimation des ressources avec sérieux si ces ressources sont jointes à la documentation dont ils ont besoin. Cette étape permet également d'identifier les zones où les délégations sont particulièrement fragiles, que ce soit pour la méthode en cascade ou la méthode agile.

7. Qui est responsable de l'approbation de chaque tâche ou groupe de tâches ?

Cette étape pourrait être un point de l'étape 6, mais il existe des différences à prendre en compte lorsque vous décidez de qui effectue la tâche et qui approuve le livrable.

Dans bien des cas, le projet doit être approuvé par des intervenants qui sont détachés de la mise en œuvre quotidienne, ou du moins par des groupes multidisciplinaires qui doivent tout vérifier successivement. L'organisation de ces étapes d'approbation, qui peuvent être compliquées, est un point stratégique d'un grand nombre de projets. Les cycles de révision qui impliquent des équipes autres que les équipes de départ peuvent immobiliser le projet.

Les chronologies de projet constituent une partie cruciale de toute planification de projet. Elles permettent non seulement d'établir les spécificités de la mise en œuvre d'un projet ou d'une initiative produit, mais aussi d'effectuer des estimations sur l'intégralité du projet au moment de la construction. Cela contribue à améliorer la communication des éléments importants qui peuvent aider les intervenants à établir des attentes réalistes quant à ce qui peut ou ne peut pas être réalisé.

2.2.6 - Affectation des ressources aux tâches

Il s'agit du processus qui consiste à affecter et planifier les ressources disponibles de la manière la plus efficace et la plus économique possible. Les ressources sont indispensables aux projets, mais elles se font rares. Il appartient donc au chef de projet de planifier les ressources au bon moment en respectant le calendrier du projet.

2.2.7 - Maîtrise des coûts

La maîtrise des coûts consiste à superviser et à gérer les dépenses du projet et à se préparer aux risques financiers potentiels. Cette tâche est généralement du ressort du chef de projet. La maîtrise des coûts implique non seulement la gestion du budget, mais aussi la planification et la préparation aux risques potentiels. Les risques peuvent retarder les projets et parfois même entraîner des dépenses imprévues. La préparation à ces revers peut permettre à votre équipe d'économiser du temps et, potentiellement, de l'argent.

La maîtrise des coûts suppose une grande discipline et commence dès :

1. La phase de faisabilité du projet.

Dans un premier temps, la technique utilisée est une estimation analogique, c'est à dire une estimation à partir de projets analogues (combien coûte la construction d'une maison de 150 m² habitables ? Entre 150 K€ et 300 K€, soit 1000 à 2000 € le m²).

2. Dans la phase d'avant-projet,

Le projet est détaillé, des choix techniques sont arrêtés ou proposés, la méthode paramétrique sera utilisée (maison de deux niveau, avec sous sol, deux salles de bain, matériaux nobles, isolation renforcée, 6 pièces, trois salles de bain, deux WC, Le coût sera affiné avec un degré de précision plus grand (la maison coûtera entre 240 et 280 K€). A la fin de la phase d'avant projet, les derniers choix techniques doivent être confirmés (types d'équipements de la salle de bain et de la cuisine, nature des revêtements, ...).

3. Avant de démarrer le projet,

Le chef de projet construira le budget initial détaillé, méthode analytique, en s'appuyant sur des devis ou sur des estimations argumentées et précises. Ce budget servira de référence pour évaluer ultérieurement les dérives éventuelles lors du suivi du projet. Il s'agit d'une estimation contractuelle qui lie le chef de projet et le donneur d'ordre.

4. Tout au long de la réalisation

Le niveau des dépenses sera comparé au niveau prévu et quelques fois des actions correctives seront proposées (voir plus loin le suivi économique et financier).

3- Adopter l'approche Agile dans la gestion de projet

3.1 - La méthode Agile Scrum

3.1.1 - Les trois piliers de scrum

Scrum est un framework, cadre de travail de type organisationnelle le plus répandu à ce jour créée par Ken Schwaber. Elle se base sur 3 piliers essentiels : la transparence, l'inspection et l'adaptation. Elle se mélange très bien à d'autres méthodes agiles ou apparentées agiles comme kanban, extreme programming, lean startup, devops...

La transparence

Le Scrum **impose de partager tous les aspects liés au processus à l'ensemble des décideurs** et la vision globale des développements à l'ensemble des observateurs.

Pour ma part, je n'hésite pas à mettre un management visuel complet afin d'aider à une transparence complète.

L'inspection

Il est **essentiel de bien suivre l'avancement des développements et des objectifs de l'équipe**. Cette inspection se fait grâce au management visuel de façon transparente, se fait à la review où l'équipe fait un point en fin d'itération du travail accompli et se fait avec la daily (petite réunion d'équipe du matin pour aligner tout le monde sur l'avancement du sprint).

Adaptation

Le Scrum préconise d'adapter les processus et l'environnement de travail afin de proposer un contexte optimal à l'équipe. La cérémonie de la rétrospective en fin de sprint permet de définir des axes d'amélioration en équipe afin d'être dans une réelle démarche d'amélioration continue.

L'équipe scrum pourra tester de nouvelles pratiques qu'elle décidera d'adopter ou non selon les résultats ; elle n'hésitera pas également à supprimer les pratiques devenues obsolètes.

3.1.2 - Processus de la méthode Scrum

Ce framework agile fonctionne sur une approche **incrémentale** et **itérative**.

Les cycles de développements sont volontairement courts et itératifs ; nous sommes en grande majorité sur des itérations de deux semaines (certains en font sur 4 semaines) que nous appelons des sprints.

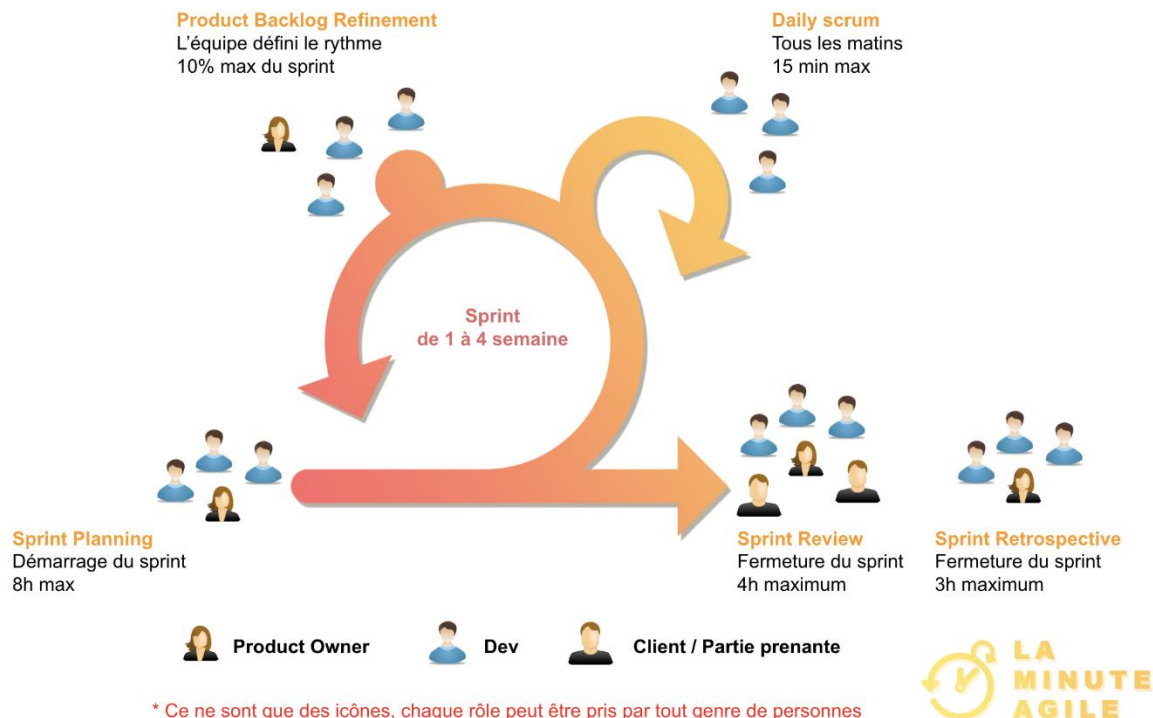
En Scrum, il est très important d'avoir des cycles de projet agile structurés avec des itérations courtes, rythmées et égales.



Ces itérations courtes sont essentielles pour se laisser la capacité d'adapter rapidement les processus voir le scope du projet ; nous profitons de ces itérations très courtes pour obtenir

un maximum de feedback des clients (à la fin de chaque itération) ce qui permet éventuellement d'incrémenter le produit de nouvelles évolutions.

Ces itérations sont représentées simplement par différentes cérémonies que nous verrons plus en détail à la suite de l'article : **sprint planning** qui démarre l'itération, **daily stand-up** tous les matins, **sprint review** et **rétrospectives** qui ferment le sprint.



3.1.3 - Rôles et responsabilités :

Une équipe Scrum comprend un **Product Owner**, une **équipe de développement** (Development Team) et un **Scrum Master**.

Les équipes Scrum (Scrum Teams) sont auto-organisées et pluridisciplinaires. Les équipes auto-organisées choisissent la meilleure façon d'accomplir leur travail, au lieu d'être dirigées par des personnes externes à l'équipe. Les équipes pluridisciplinaires ont toutes les compétences nécessaires pour effectuer le travail sans dépendre d'autres personnes n'appartenant pas à l'équipe.

Cette équipe doit être capable d'avancer sur le projet en toute autonomie. Cette notion est souvent claire pour tout le monde bien qu'elle soit peu respectée dans les faits.

Equipe de développement

L'équipe de développement **se compose de professionnels qui fournissent un incrément « Fini » potentiellement publiable (Releasable) à la fin de chaque Sprint**. Un incrément « Fini » est requis à la revue de sprint. Seuls les membres de l'équipe de développement créent l'incrément.

Les équipes de développement sont structurées et habilitées par l'organisation à s'organiser et gérer leur propre travail. La synergie résultante optimise l'efficacité et l'efficacité globale de l'équipe de développement.

Scrum master

Le Scrum Master est **chargé de promouvoir et supporter Scrum** tel que défini dans le Guide Scrum. Les Scrum Masters remplissent leur rôle en aidant tout le monde à comprendre la théorie, les pratiques, les règles et les valeurs de Scrum.

Le Scrum Master est **un leader-serviteur de l'équipe Scrum**. Le Scrum Master assiste les personnes externes à l'équipe Scrum pour identifier quelles sont les interactions bénéfiques avec elle. Le Scrum Master aide tout le monde à adapter leurs interactions avec l'équipe Scrum pour maximiser la valeur créée par cette équipe.

Le scrum master se met au service du product owner tout en le guidant dans la bonne direction. Il n'est pas rare que le rôle de product owner soit très mal compris.

Il devra ainsi guider le product owner à mieux prendre son rôle et à accepter que ses responsabilités sont très différentes des rôles fonctionnels que nous pouvons trouver par exemple dans les équipes cycle en V.

Le scrum master aidera et guidera le product owner dans la création et le maintien du product backlog. Ceci passera par :

- l'organisation du product backlog
- rendre transparent ce product backlog
- la clarté des items de celui-ci pour l'équipe de développement
- l'aider à ordonner les items pour maximiser la valeur livrée

Product owner

Le Product Owner est **responsable de maximiser la valeur du produit résultant du travail de l'équipe de développement**. La façon de jouer ce rôle peut varier grandement selon les organisations, les équipes Scrum et les individus.

Le product owner est le « seul » responsable de la gestion du backlog. Il peut déléguer certaines tâches de gestion de son backlog à l'équipe de développement mais gardera toujours la responsabilité des résultats.

Le product owner sera ainsi responsable de :

- fournir des items clairs, complets et compris de tous
- d'organiser et de prioriser le contenu de son backlog
- d'optimiser la valeur que pourra délivrer l'équipe de développement
- rendre visible le backlog, la vision du produit et le travail imaginé pour les prochaines itérations

3.1.4 - Événements Scrum :

Sprint

L'itération ou le cycle de développement du scrum est nommé un « sprint » ; ce sprint scrum dure en général **deux ou trois semaines**. Des cycles courts permettent d'avoir des feedbacks plus fréquemment.

Mais ceux-ci ne doivent pas être trop courts non plus, pour laisser le temps à l'équipe de réaliser des développements permettant d'atteindre des objectifs réalistes. On appellera d'ailleurs ces objectifs, les objectifs du sprint.

Et pour finir, la durée de chaque sprint scrum doit être fixe. Il ne faut pas changer cette durée quelle que ce soit les raisons.

Un sprint scrum commence par **un sprint planning** et se terminera par **une sprint review** et **une sprint retrospective**. Tous les matins, l'équipe de développement s'harmonisera avec la **Daily** et l'équipe fera des **Product Backlog Refinement** pour aider le Product Owner à affiner le contenu du backlog.

Voici un petit schéma pour représenter un sprint scrum :



Planification des sprints (sprint planning)

La Sprint Planning est la cérémonie que l'équipe scrum réalise en ouverture de Sprint pour cadrer le sprint qui démarre. Commencer un Sprint sans même le préparer n'a que peu de valeur ; il est difficile de travailler sans avoir une vision du travail à réaliser.

- **L'objectif** : bien organiser le sprint scrum qui démarre.
- **La durée** : le temps qu'il faut pour finir la préparation du sprint. Mais c'est préférable de ne pas dépasser plus de 2h.
- **Les participants** : le Product Owner (PO), le Scrum Master (SM) et toute l'équipe de développement (des développeurs)
- **Les animateurs** : c'est Le Product Owner qui est le maître de cérémonie. Le Scrum Master sera plutôt celui qui va seconder le Product Owner pour intervenir en cas de besoin.

Le déroulement :

1. Le Product Owner définit les objectifs du sprint.
2. L'équipe va devoir ré-estimer le point d'effort (la complexité) de chacune des demandes du sprint précédent qui ont fini en « work in progress » (en cours).
3. Le Product Owner va alors proposer aux développeurs un ensemble de demandes qu'ils devront prendre en charge pendant le sprint. Ces demandes répondront dans l'ensemble aux objectifs fixés en début de cérémonie.

4. On n'estime plus des demandes pendant le sprint planning mais on réalise les estimations en amont du Sprint Planning. C'est lors de la Product Backlog Refinement que l'estimation est faite.
5. La dernière étape est de découper l'ensemble des demandes en sous-tâches techniques.

Chaque jour du sprint scrum, l'équipe participe au daily scrum.

Mêlée quotidienne (daily Scrum)

La Daily Scrum (daily meeting) est une réunion quotidienne qui rassemble l'ensemble de l'équipe de réalisation qui ne doit pas durer plus de 15 minutes.

- **L'objectif** : l'équipe de développement doit être claire sur l'avancement du projet et relever des points bloquants.
- **La durée** : 15 minutes maximum tous les jours du sprint scrum
- **Les participants** : l'équipe des développeurs, le Product Owner (en observateur toléré) et le Scrum Master (nécessaire si l'équipe n'est pas mature)
- **Les animateurs** : l'équipe de développement autonome

Le déroulement :

On fait un tour de l'équipe de développement et chaque participant doit :

1. Expliquer ce qu'il a fait depuis la dernière Daily
2. Dire ce qu'il pense faire jusqu'à la prochaine Daily
3. Lever une alerte (s'il y en a)

Revue de Sprint (Sprint review)

- **L'objectif** : faire un point sur ce qui a été réalisé en cours de sprint. Il est aussi intéressant de présenter le rendu afin d'améliorer l'échange entre l'équipe et les parties prenantes ; cela dans le but de récupérer un maximum de feedback constructifs.
- **La durée** : ne dure pas plus de 1 heure
- **Les participants** : l'équipe de développement, le Product Owner, le Scrum Master, les utilisateurs clés [optionnel], les parties prenantes [optionnel] et les sponsors [optionnel]
- **Les animateurs** : C'est Le Product Owner (PO) qui est le maître de cérémonie, Le Scrum Master (SM) sera le second animateur. C'est plutôt conseillé que le PO et le SM présentent en binôme.

Le déroulement :

1. Le Product Owner présente le travail parcourus pendant le Sprint
2. Des développeurs font une démonstration d'une nouvelle fonctionnalité terminée ou d'un nouveau produit terminé
3. Le Scrum Master présente quelques indicateurs révélateurs pour expliquer concrètement l'avancement du projet.
4. Questions/Réponses et feedback.

Rétrospective Sprint (Sprint retrospective)

- **L'objectif** : de travailler sur deux axes principaux soit l'amélioration continue et la santé de l'équipe
- **La durée** : une heure
- **Les participants** : l'équipe de développement, le Product Owner et le Scrum Master
- **Les animateurs** : C'est le Scrum Master qui prépare et anime à 100%

Le déroulement :

Le scrum Master va proposer des ateliers ludiques afin d'atteindre des objectifs de la rétrospective avec quelques règles d'or : chaque rétrospective doit être différente elle doit être animée et pas monotone

3.1.5 - Artéfacts Scrum :

Définition

Les artefacts Scrum sont des éléments permettant de faire fonctionner le cadre de travail Scrum, en accord avec les principes et les pratiques s'inscrivant dans l'ADN de l'agilité. Au nombre de 3, ils regroupent :

- le sprint backlog,
- le product backlog,
- l'incrément produit.

Suivant la méthode Scrum, ces artefacts prennent la forme de listes qui ont pour objet :

- de matérialiser l'avancement du travail,
- de représenter l'atteinte des objectifs,
- de décrire la progression de l'équipe,

Carnet de produit (backlog product)

Le product backlog représente **le regroupement de l'ensemble des choses à réaliser sur le produit pour le faire évoluer** ; il est de la responsabilité du product owner. Contrairement aux méthodes cycle en V/waterfall, le backlog évolue constamment pour prendre en compte les nombreux feedbacks obtenus.

En cours du sprint, une fois par semaine, l'équipe participe au Product Backlog.

Carnet de Sprint (Sprint backlog)

Le sprint backlog représente **l'ensemble des items qui sont pris en charge par l'équipe de développement lors du sprint en cours** ; il contiendra également le plan pour livrer ces items dans un environnement stable. Nous considérons que chaque sprint possède son propre sprint backlog.

Incrément (Increment)

L'incrément représente **l'ensemble des items qui sont « done » dans le sprint en cours ajouté à l'incrément du sprint précédent**. Pour faire simple, c'est le produit dans un statut stable avec l'ensemble des ajouts que nous avons à la fin de chaque sprint.

3.2 - L'outil de gestion de projet Agile (Scrum/Jira)

3.2.1 - Présentation de Jira

Jira est un système de suivi de bugs, de gestion des incidents et de gestion de projets développé par Atlassian et publié pour la première fois en 2002. Il propose des solutions à la fois à destination des développeurs et des intervenants non développeurs.

L'outil Jira est destiné aux sociétés désireuses de mettre en place un fonctionnement en méthode agile et facilite le travail des utilisateurs concernés dans leur organisation. Il permet notamment la création et la planification de tâches via un système de rédaction et de gestion des récits utilisateurs.

Le pack Jira Cloud comprend trois solutions :

- **Jira Software** à destination des équipes de développeurs pour la mise en place de méthode Kanban et d'organisation de type scrum. Il permet le découpage des projets en récits utilisateurs (tâches, composants) et leur affectation aux développeurs.
- **Jira Service Management** (anciennement Jira Service Desk) pour la gestion des incidents et des délais de résolution. Fonctionnellement l'outil propose un système de ticket numérique, permettant à l'utilisateur déclarant l'incident d'alerter la bonne personne ou le bon service.
- **Jira Work Management** (anciennement Jira Core) est un module destiné aux équipes hors développeurs (dites métiers) pour la gestion des demandes de types accès aux systèmes d'information (obtention de licence pour un utilisateur par exemple) ou l'enregistrement des nouveaux collaborateurs.

Jira Software

Jira pour la gestion des exigences et des cas de test

De nos jours, de plus en plus d'équipes développent de manière itérative, et Jira Software constitue le hub central pour les phases de programmation, de collaboration et de livraison. Pour la gestion des tests, Jira s'intègre avec diverses extensions afin d'inclure de façon transparente les tests de QA dans le cycle de développement. Les équipes peuvent réaliser des tests efficaces et itératifs. Les équipes de QA utilisent des tickets Jira, des écrans personnalisés, des champs et des workflows pour gérer les tests manuels et automatisés.

Jira pour les équipes Agile

Pour les équipes qui adoptent des méthodologies Agile, Jira Software fournit des tableaux Scrum et Kanban prêts à l'emploi. Les tableaux constituent des centres de gestion des tâches, dans lesquels les tâches sont mappées à des workflows personnalisables. Ils assurent la transparence du travail d'équipe et offrent une visibilité sur l'état d'avancement de chaque tâche. Des fonctions de suivi du temps et des rapports de performance en temps réel (graphiques Burnup/Burndown, rapports de sprint, graphiques de vélocité) permettent aux équipes de surveiller étroitement leur productivité au fil du temps.

Jira pour les équipes de gestion de projet

Vous pouvez configurer Jira Software de sorte qu'il s'adapte à tout type de projet. Les équipes peuvent se lancer avec un modèle de projet ou créer leur propre workflow personnalisé. Les tickets Jira, également appelés « tâches », suivent chaque travail qui doit passer par les différentes étapes du workflow jusqu'à son achèvement. Les autorisations personnalisables permettent aux administrateurs de déterminer les utilisateurs habilités à voir et à effectuer des actions.

Jira pour les équipes de développement

Jira Software fournit des outils de planification et de feuille de route pour permettre aux équipes de gérer les parties prenantes, les budgets et les exigences des fonctionnalités dès le démarrage du projet. Jira s'intègre avec divers outils de CI/CD pour renforcer la transparence tout au long du cycle de vie de développement logiciel (SDLC). Lorsque le code de production est prêt à être déployé, les informations sur l'état apparaissent en temps réel dans le ticket Jira. Des outils intégrés de feature flagging permettent aux équipes de déployer de nouvelles fonctionnalités progressivement et en toute sécurité.

Jira Software pour les équipes DevOps

DevOps est un ensemble de pratiques visant à automatiser et intégrer les processus entre les équipes de développement et informatiques, afin de leur permettre de développer, de tester et de publier des logiciels plus rapidement et de manière plus fiable. Pour les équipes qui pratiquent DevOps, Jira Software est l'épine dorsale de la chaîne d'outils ouverte et intégrée d'Atlassian appelée Open DevOps. Jira Software s'intègre à des outils internes et tiers tout au long du cycle de vie DevOps, y compris des outils de contrôle de code et de version (comme Bitbucket, GitHub et Gitlab), des outils de gestion de la documentation et des connaissances (comme Confluence), ainsi que des outils de surveillance et d'exploitation (comme Opsgenie).

Jira pour les équipes de gestion de produit

Dans Jira Software, les équipes peuvent élaborer une feuille de route associée à chaque projet. Elles sont ainsi en mesure d'esquisser la vision à long terme de leur travail, mais également de suivre et de partager leur avancement. Ajoutez plus de détails à vos feuilles de route en faisant apparaître les dépendances et les prévisions sur l'achèvement du travail. Créez une vue qui met en évidence les feuilles de route de plusieurs équipes « en temps réel » en intégrant la feuille de route Jira Software à Confluence.

Jira pour la gestion des tâches

Créez des tâches pour vous-même et pour les membres de votre équipe et renseignez ses détails, les dates d'échéance et les rappels. Utilisez des sous-tâches pour décomposer les tâches les plus importantes. Autorisez d'autres utilisateurs à suivre l'avancement de la tâche et à être avertis lorsqu'elle est terminée. Créez des sous-tâches au sein de la tâche parente afin de décomposer le travail en unités assimilables pour les différents membres de l'équipe. Affichez toutes les tâches sur le tableau afin de visualiser facilement l'état de chacune d'elles.

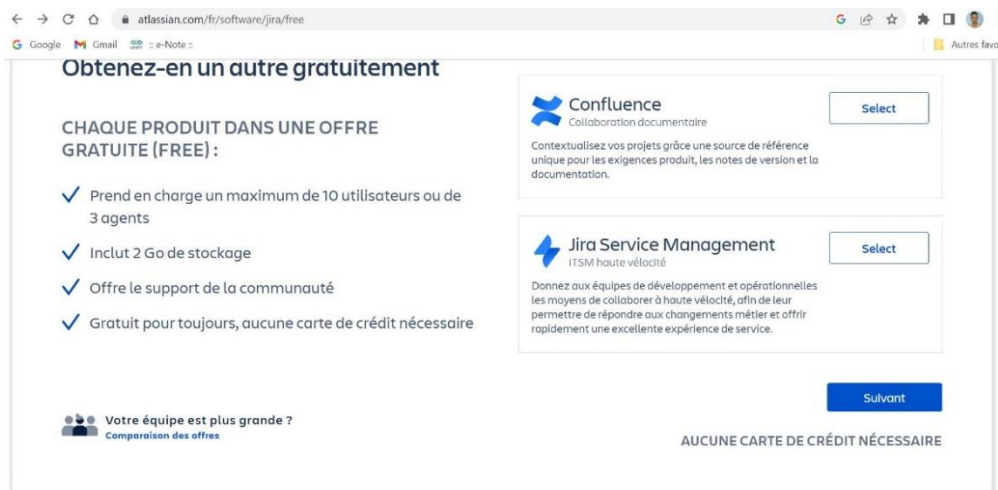
Jira pour le suivi des bugs

Les bugs ne sont rien de plus que l'appellation donnée aux tâches à faire, qui découlent de problèmes identifiés dans le logiciel en cours de développement. Il est important pour les équipes de visualiser l'ensemble des tâches et des bugs dans le backlog afin de pouvoir prioriser les objectifs globaux. Le puissant moteur de workflow de Jira garantit l'assignation et la hiérarchisation automatiques des bugs une fois ceux-ci capturés. Les équipes peuvent alors suivre un bug jusqu'à sa résolution.

3.2.2 - Installation et configuration d'un compte Jira

Étape 1 : Se rendre sur le site Atlassian. <https://www.atlassian.com/fr/software/jira/free>

Appuyer sur le bouton « Suivant »



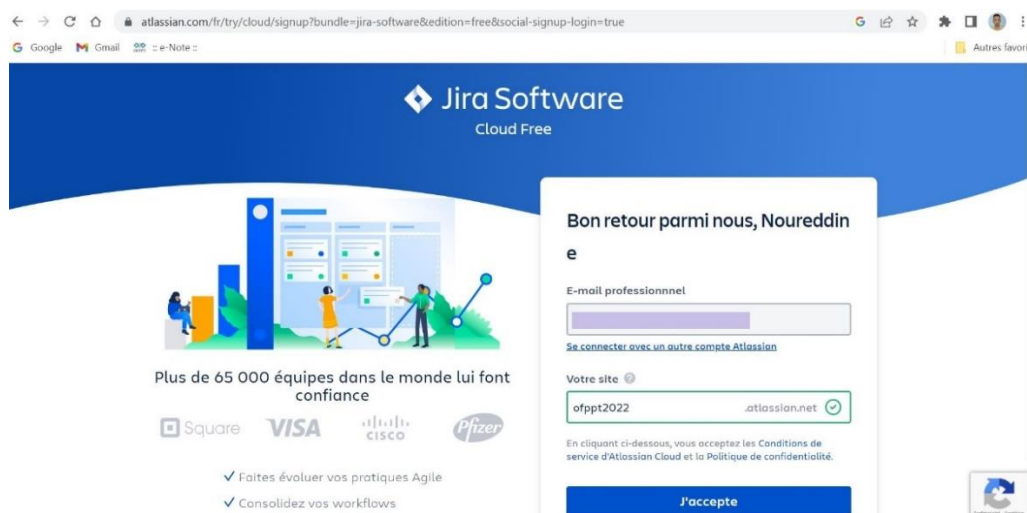
Étape 2 : Connecté vous !

Choisissez votre moyen de connexion.



Étape 3 : Votre site

Choisissez un nom pour votre site.



Étape 4 : Conditions d'utilisation

Acceptez les conditions d'utilisation.

Obtenez-en un autre gratuitement

CHACQUE PRODUIT DANS UNE OFFRE GRATUITE (FREE) :

- ✓ Prend en charge un maximum de 10 utilisateurs ou de 3 agents
- ✓ Inclut 2 Go de stockage
- ✓ Offre le support de la communauté
- ✓ Gratuit pour toujours, aucune carte de crédit nécessaire

**Confluence**
Collaboration documentaire
[Select](#)
Contextualisez vos projets grâce une source de référence unique pour les exigences produit, les notes de version et la documentation.

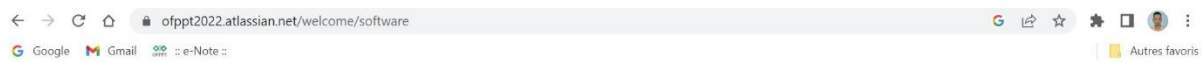
**Jira Service Management**
ITSM haute vélocité
[Select](#)
Donnez aux équipes de développement et opérationnelles les moyens de collaborer à haute vélocité, afin de leur permettre de répondre aux changements métier et offrir rapidement une excellente expérience de service.

[Skip](#) [Agree and continue](#)

BY SIGNING UP, YOU AGREE TO THE ATlassian CLOUD TERMS OF SERVICE AND PRIVACY POLICY

Étape 5 : Configuration

Configurez votre compte Jira



Recommander un projet

Répondez à quelques questions, et nous vous suggérerons le type de projets qui convient le mieux à vous et à votre équipe.

Aidez-nous à configurer votre compte Jira

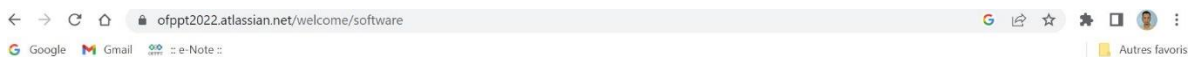
Je découvre [Jira](#).

Nous avons des délais [flexibles](#) pour effectuer notre travail.

[Ignorer](#) [Suivant](#)

Étape 6 : Création de projet

Créez votre premier projet KANBAN



Ajouter les informations du projet

Vous pouvez modifier ces informations à tout moment dans les paramètres de votre projet.

Nom

Application web

Cle 

AW

Modèle

Changer de modèle



Kanban RECOMMANDÉ

Affichez et faites avancer votre projet grâce à des tickets sur un tableau performant.

Type

Changer de type



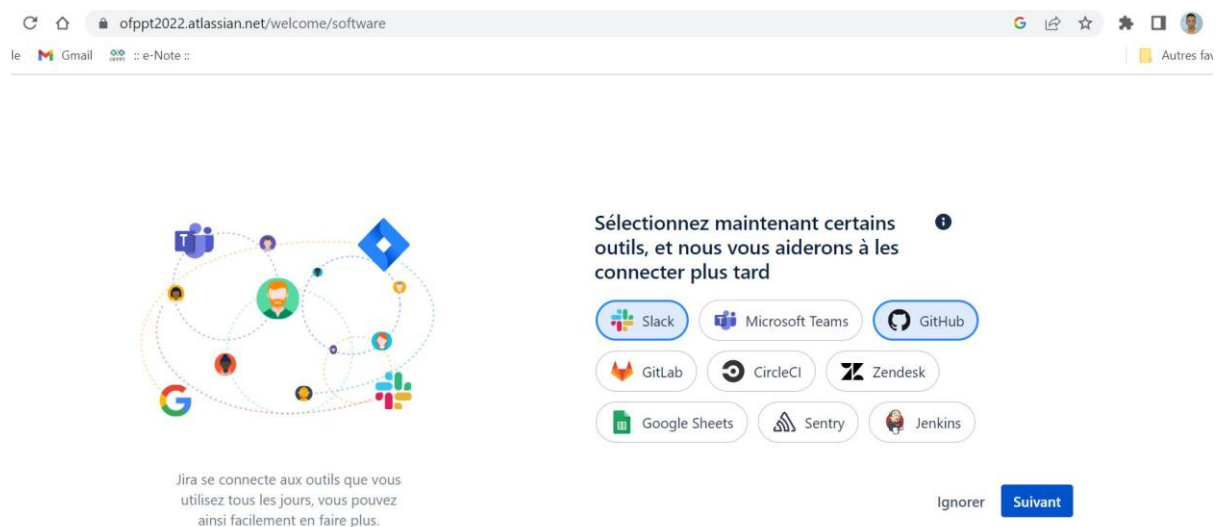
Géré par l'équipe RECOMMANDÉ

Contrôlez vos propres processus et pratiques de travail dans un espace autonome.

[Retour](#) [Créer un projet](#)

Étape 7 : Outils

Sélectionnez les outils que vous souhaitez les connecter plus tard



Jira se connecte aux outils que vous utilisez tous les jours, vous pouvez ainsi facilement en faire plus.

Sélectionnez maintenant certains outils, et nous vous aiderons à les connecter plus tard

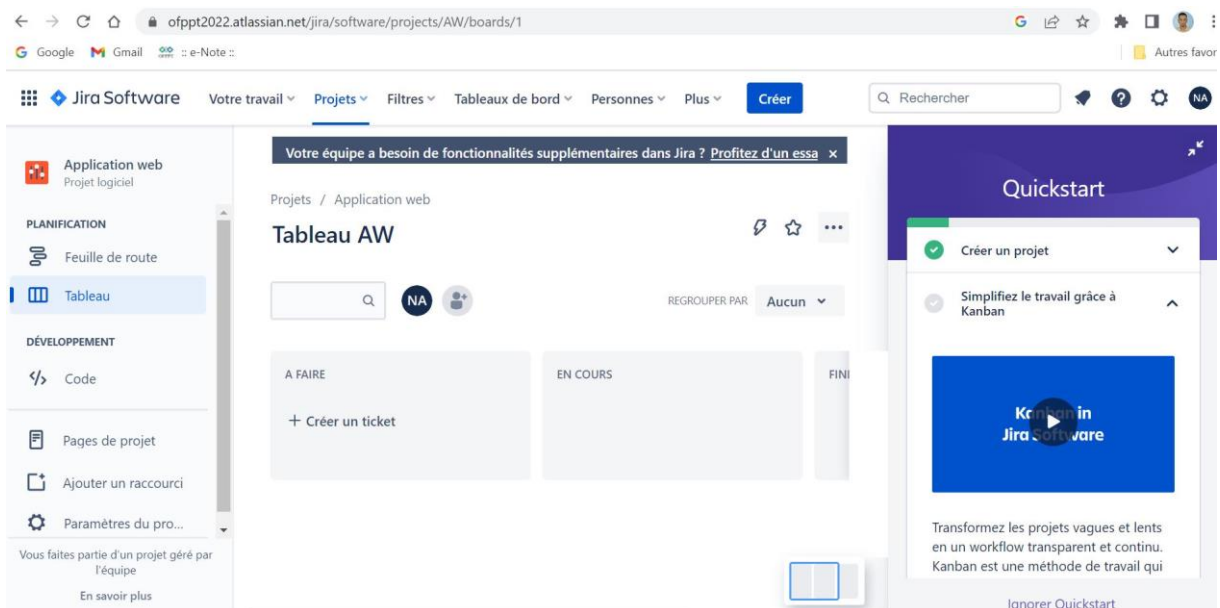
Slack Microsoft Teams GitHub
GitLab CircleCI Zendesk
Google Sheets Sentry Jenkins

Ignorer Suivant

Étape 8 : Félicitation !

Votre compte JIRA et projet ont été bien créés.

Découvrir le tableau de bord.



Jira Software

Votre travail Projets Filtres Tableaux de bord Personnes Plus Créer

Rechercher

Application web
Projet logiciel

PLANIFICATION
Feuille de route
Tableau

DÉVELOPPEMENT
Code

Pages de projet
Ajouter un raccourci
Paramètres du pro...

Vous faites partie d'un projet géré par l'équipe
En savoir plus

Votre équipe a besoin de fonctionnalités supplémentaires dans Jira ? Profitez d'un essai

Projets / Application web

Tableau AW

REGROUPER PAR Aucun

A FAIRE
+ Créer un ticket

EN COURS

FINI

Quickstart

Créer un projet

Simplifiez le travail grâce à Kanban

Kanban in Jira Software

Transformez les projets vagues et lents en un workflow transparent et continu. Kanban est une méthode de travail qui

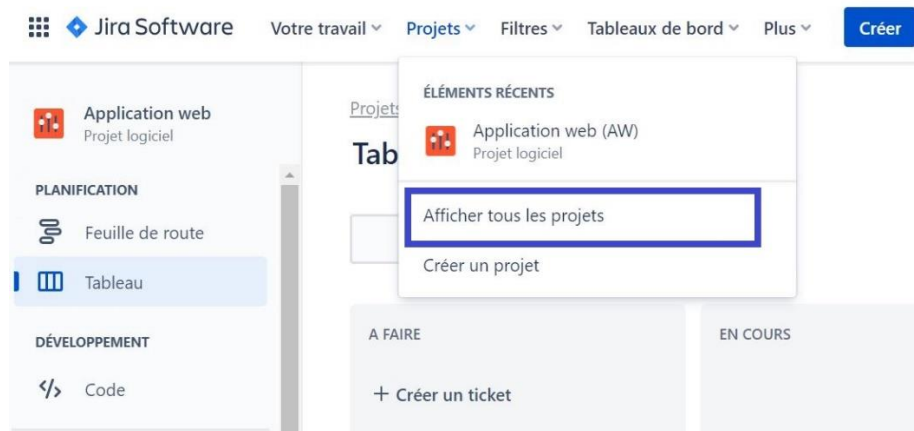
Ignorer Quickstart

3.2.3 - Création de projet SCRUM

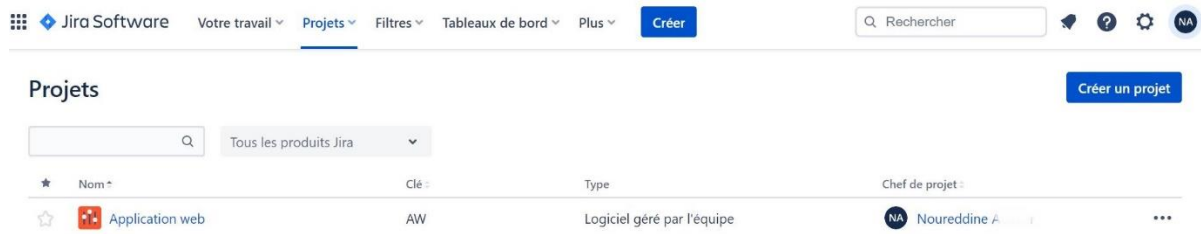
Pour débuter sur Jira en Scrum, il faut déjà créer votre premier projet.

Étape 1 : Créer un projet

Allez dans l'onglet **Projets**



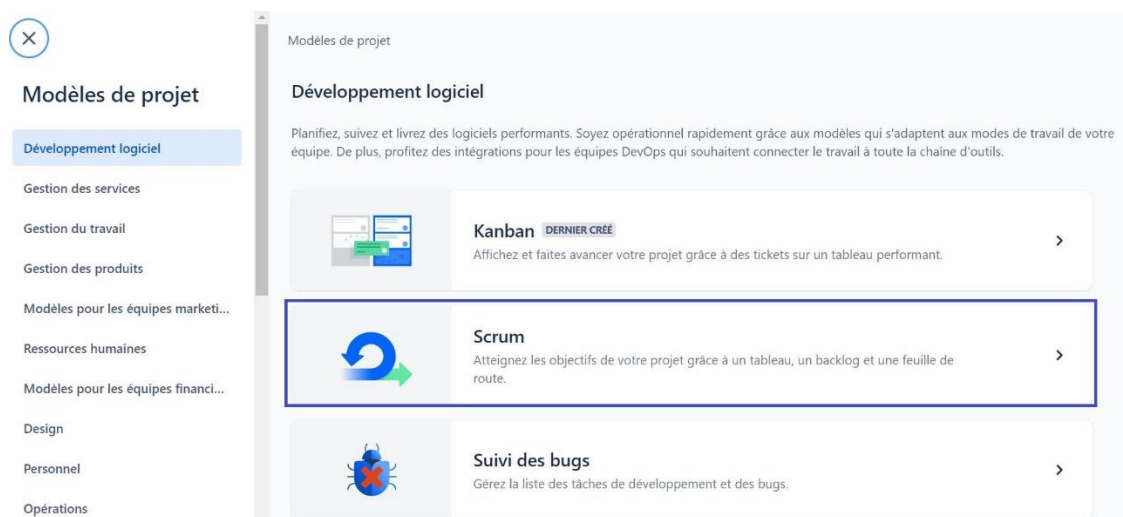
Cliquez sur **Créer un projet**



Étape 2 : Choisir le modèle de projet

La bibliothèque de modèles Jira contient des dizaines de modèles répartis dans différentes catégories et est conçue pour permettre à votre équipe de se lancer rapidement et sans le moindre accroc. Aujourd'hui, Jira Software propose trois modèles :

- **SCRUM** pour les équipes Agile qui utilisent un backlog, planifient et estiment leur travail en sprints, et livrent régulièrement.
- **KANBAN** pour les équipes qui surveillent le travail en continu (plutôt que dans des sprints) et mettent l'accent sur la gestion du travail en cours.
- **SUIVI DES BUGS** pour les équipes qui n'ont pas besoin de tableaux et préfèrent gérer les tâches de développement et les bugs dans un affichage en liste.



Sélectionnez le modèle de projet **SCRUM**

Cliquez sur **Utilisez un modèle**

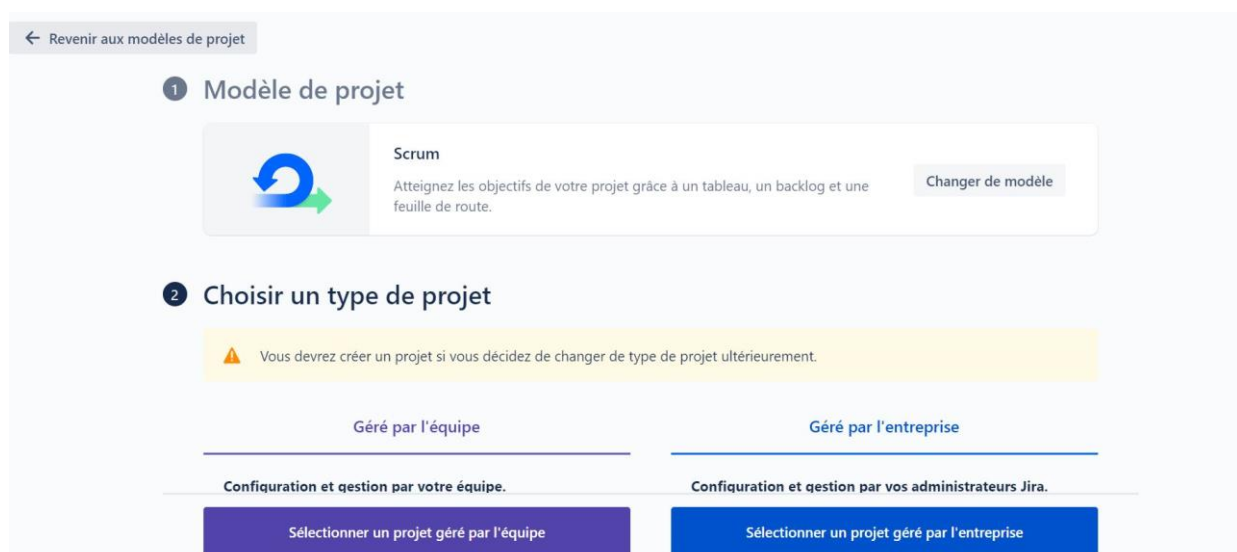


Étape 3 : Choisir le type de projet

Pour les modèles Scrum et Kanban uniquement, vous serez également invité à choisir un type de projet.

- **Les projets gérés par l'équipe** sont adaptés aux équipes indépendantes qui souhaitent contrôler leurs propres processus et pratiques de travail dans un espace autonome.
- **Les projets gérés par l'entreprise** sont configurés et maintenus par les administrateurs Jira. Ils sont conçus pour les entreprises qui souhaitent standardiser une méthode de travail entre de nombreuses équipes, comme le partage d'un workflow.

La différence fondamentale entre les deux types de projet réside dans la manière dont ils sont administrés et si l'administration a lieu au niveau de l'équipe ou au niveau de l'entreprise/l'administrateur Jira. Vous souhaitez en savoir plus sur les projets gérés par l'entreprise et par l'équipe ?



Cliquez sur **Sélectionner un projet géré par l'équipe**

Étape 4 : Ajouter les informations du projet

← Revenir aux types de projets

Ajouter les informations du projet

Vous pouvez modifier ces informations à tout moment dans les paramètres de votre projet.

Nom *

Test Jira Scrum

Accès Toutes les personnes ayant accès à ofppt2022 peuvent accéder à ce projet et l'administrer. [Passez à la version supérieure](#) pour personnaliser les autorisations du projet.

Clé ⓘ *

TJS

- ☐ **Connectez des dépôts, des documents et plus encore**
Synchronisez le travail de votre équipe à partir d'autres outils avec ce projet pour améliorer la visibilité, l'accès et l'automatisation.

Modèle

Changer de modèle



Scrum

Atteignez les objectifs de votre projet grâce à un tableau, un backlog et une feuille de route.

Type

Changer de type



Géré par l'équipe

Contrôlez vos propres processus et pratiques de travail dans un espace autonome.

Annuler

Créer un projet

Cliquez sur **Créer un projet**

Jira Software

Votre travail

Projets

Filtres

Tableaux de bord

Personnes

Plus

Créer

Rechercher

NA

Test Jira Scrum

Project logiciel

PLANIFICATION

Feuille de route

Backlog

Tableau

DÉVELOPPEMENT

Code

Pages de projet

Ajouter un raccourci

Vous faites partie d'un projet géré par l'équipe

En savoir plus

Projets / Test Jira Scrum

Tableau TJS

NA

REGROUPER PAR

Aucun

A FAIRE

EN COURS

FINI

+

Vous n'avez pas encore démarré de sprint

Vous ne pouvez pas utiliser votre tableau car vous n'avez pas encore démarré de sprint.

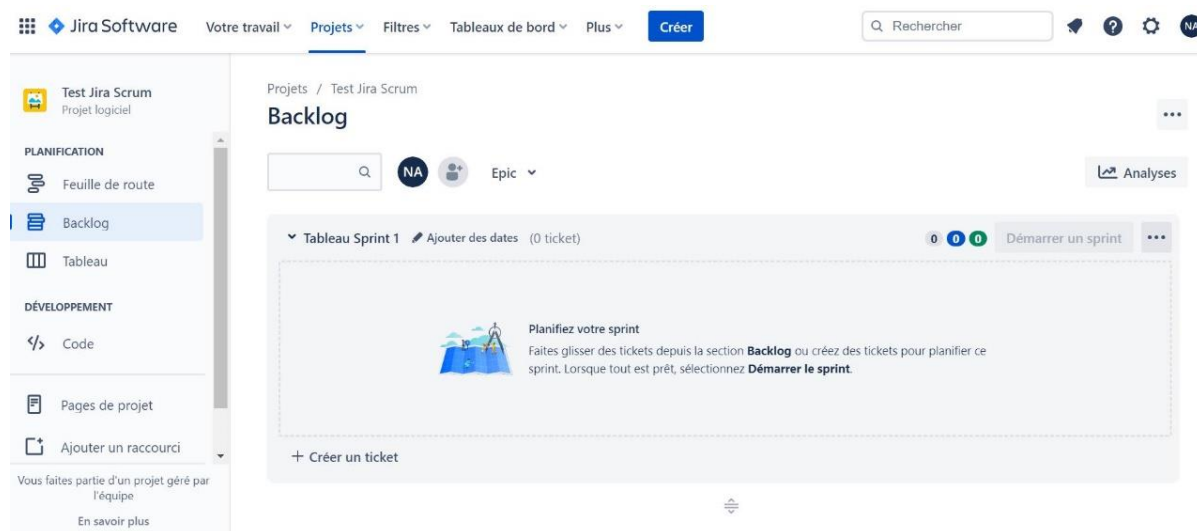
3.2.4 – Gérer votre projet

3.2.4.1 - Créer un backlog

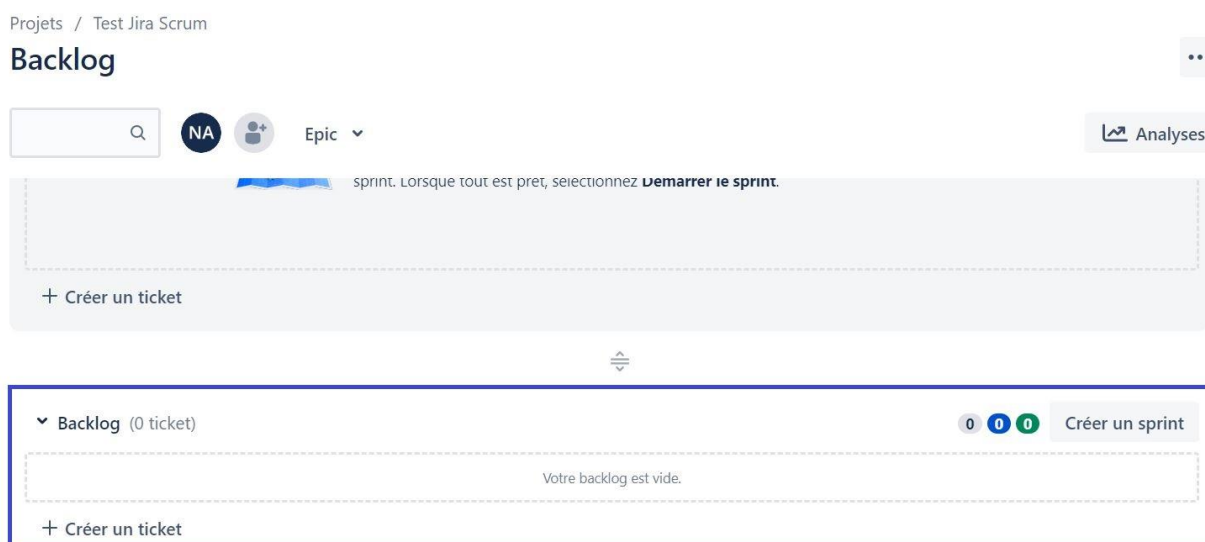
Un backlog produit est une liste hiérarchisée de fonctionnalités ou d'éléments de travail ou tâches destinées à l'équipe de développement.

En Jira, Un backlog contient la liste des épics ou tâches avec leurs tickets et leurs prépositions (durées ou points) et la personne assignée de faire le ticket.

Après la création du projet, un backlog vide est créé automatiquement, il suffit de cliquer dans le menu latéral pour y'accéder :

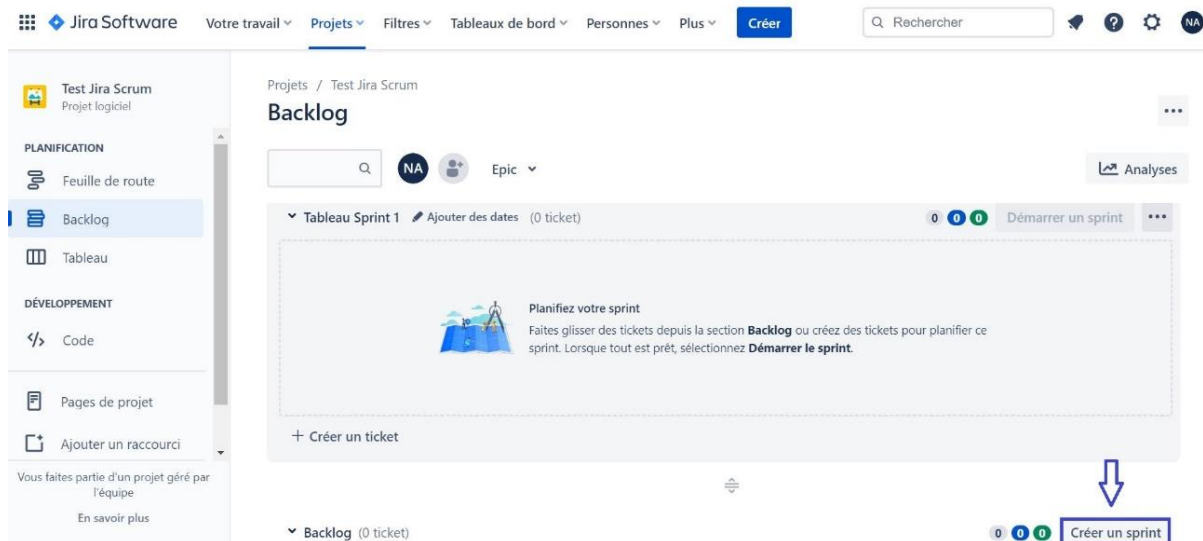


Vous pouvez maintenant définir toutes les tâches et fonctionnalités dans votre backlog en cliquant sur **créer un ticket** et par la suite designer le sprint en cliquant sur « ... » de tâche.



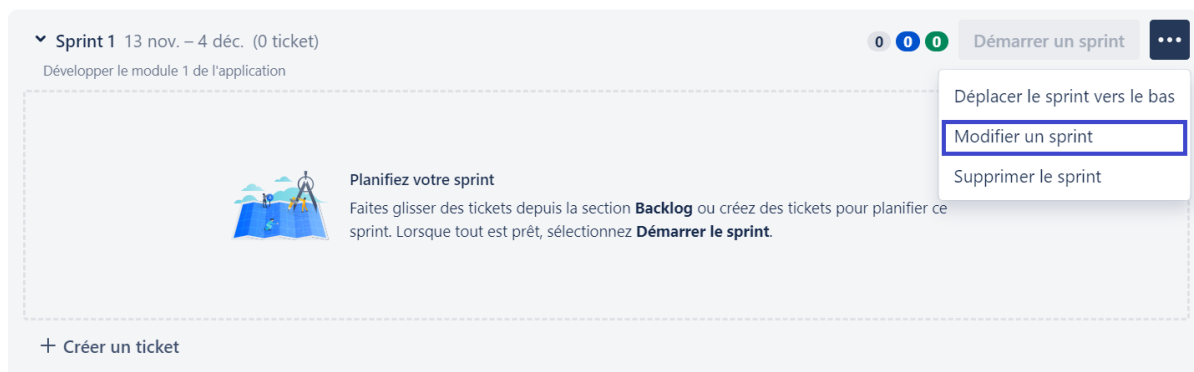
3.2.4.2 - Créer un sprint :

Par défaut, backlog du projet contient un sprint vide. Pour ajouter un nouveau sprint il suffit de cliquer sur le bouton « créer un sprint »

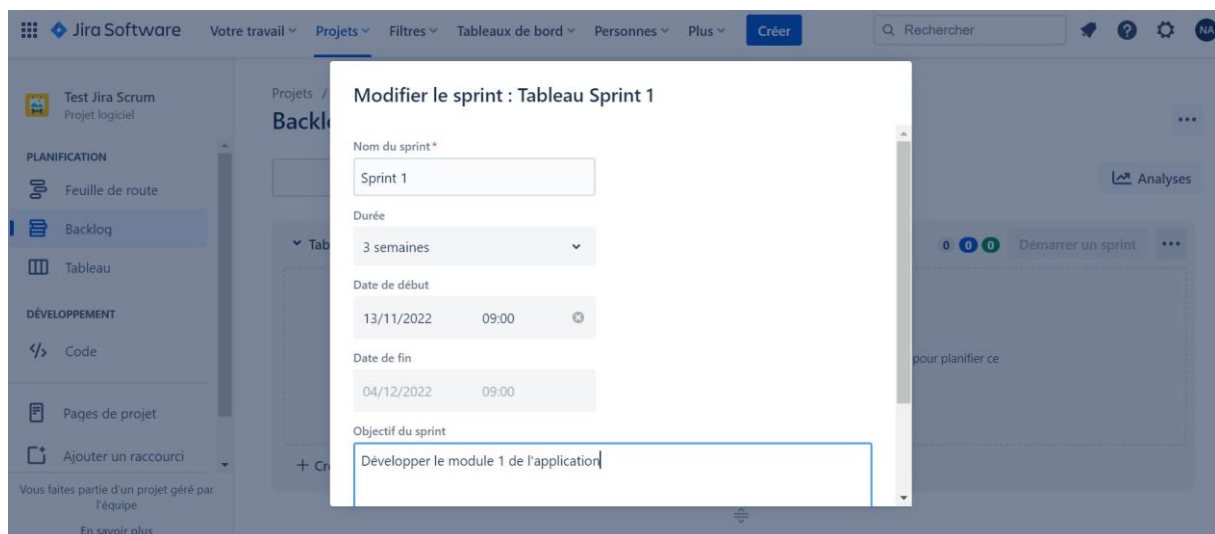


Configurer votre sprint :

Après avoir cliqué sur **Modifier un sprint**



Vous pouvez modifier les informations de votre sprint : Nom, durée, Date de début, date de fin et l'objectif du sprint :

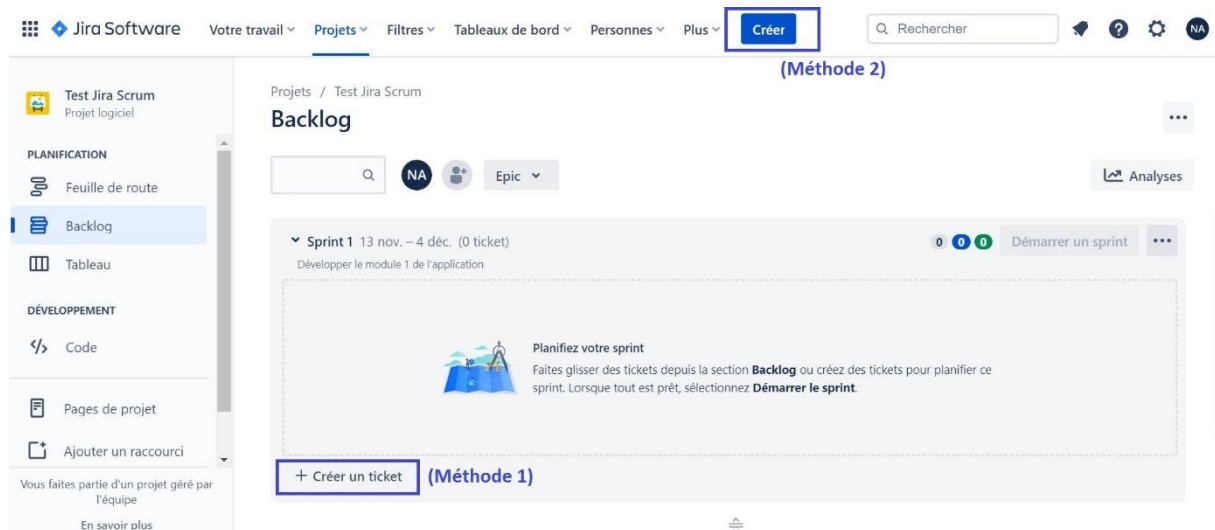


3.2.4.3 - Créer une tâche :

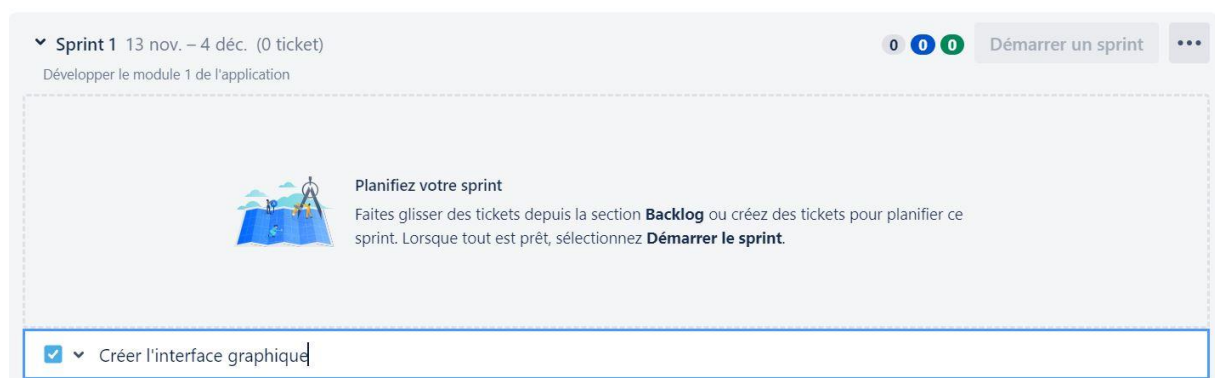
Un ticket est une tâche à effectuer, c'est un élément de travail. Un ticket suit plusieurs étapes : à faire, puis en cours, puis terminé. On peut dire que le ticket passe dans chaque étape du workflow – flux de travail. Les tickets dans Jira Software peuvent être de plusieurs types. Dans un projet de développement logiciel, on retrouve les types de ticket suivants :

- **Une epic** est considérée comme un grand objectif devant être simplifié et divisé en plusieurs tâches afin d'organiser le travail des équipes agiles
- **Un bug** désigne un problème à corriger
- **Une story** représente une fonctionnalité à réaliser
- **Une tâche** est généralement une tâche technique à effectuer
- **Les sous-tâches** sont de petites unités de travail qui font partie d'une tâche plus importante.

Vous pouvez créer un ticket en utilisant plusieurs méthodes :



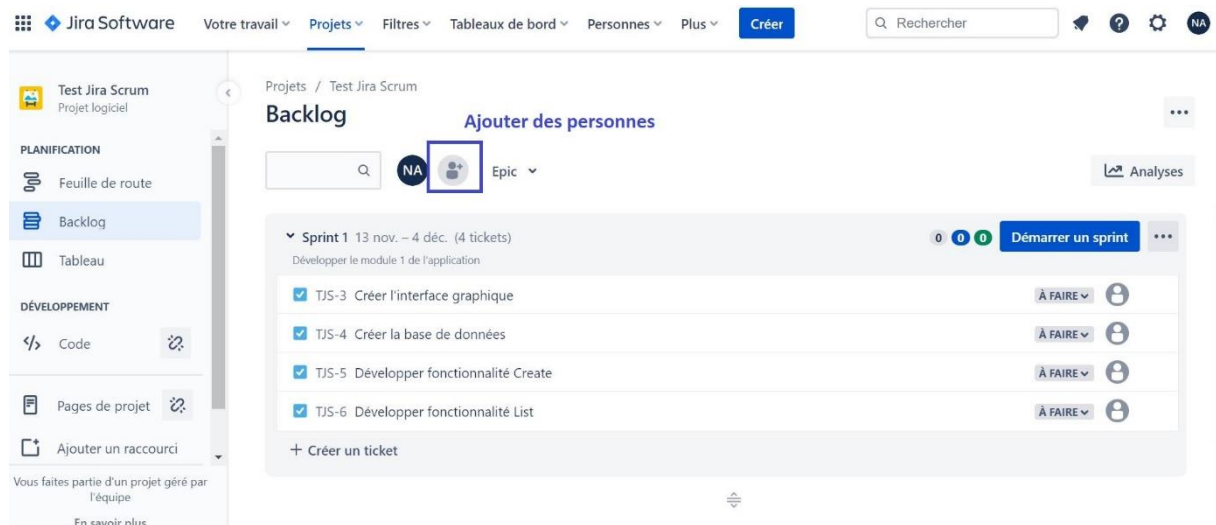
Méthode 1 : Créer un ticket pour Sprint 1



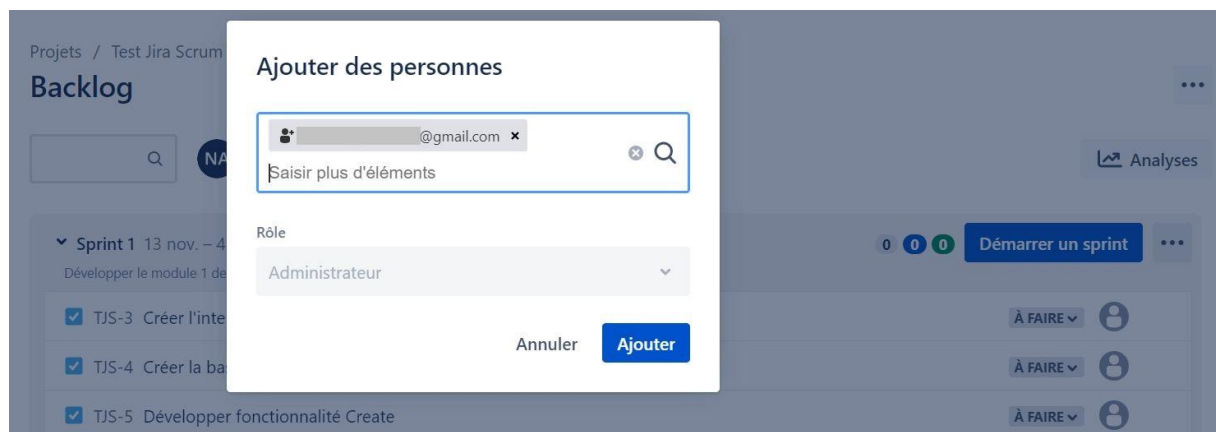
Méthode 2 : Créer un ticket en choisissant le sprint

3.2.4.4 - Ajouter une personne à votre projet (un collaborateur) :

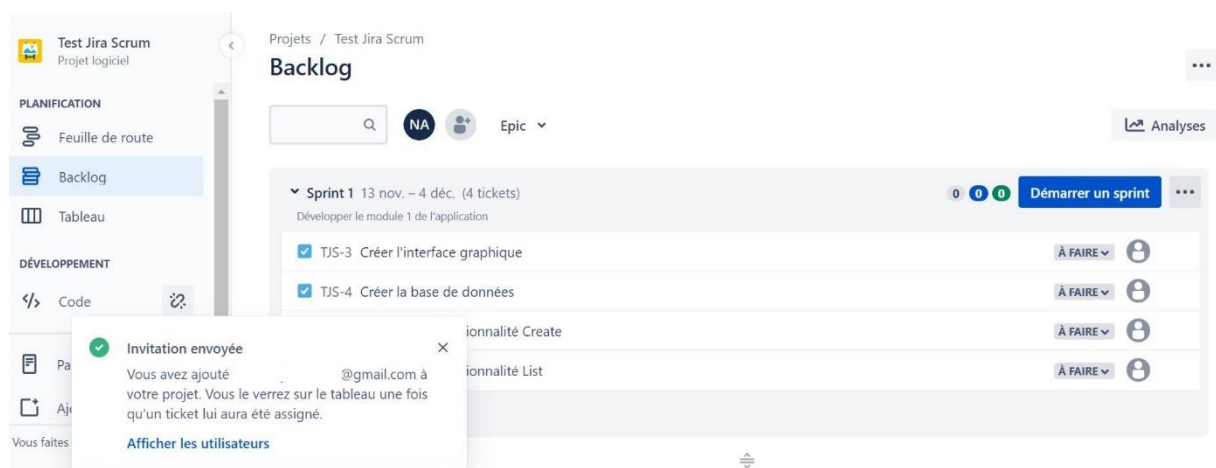
Pour ajouter une personne à votre projet, cliquez sur le bouton « Ajouter des personnes »



Ajouter l'email de la personne invitée



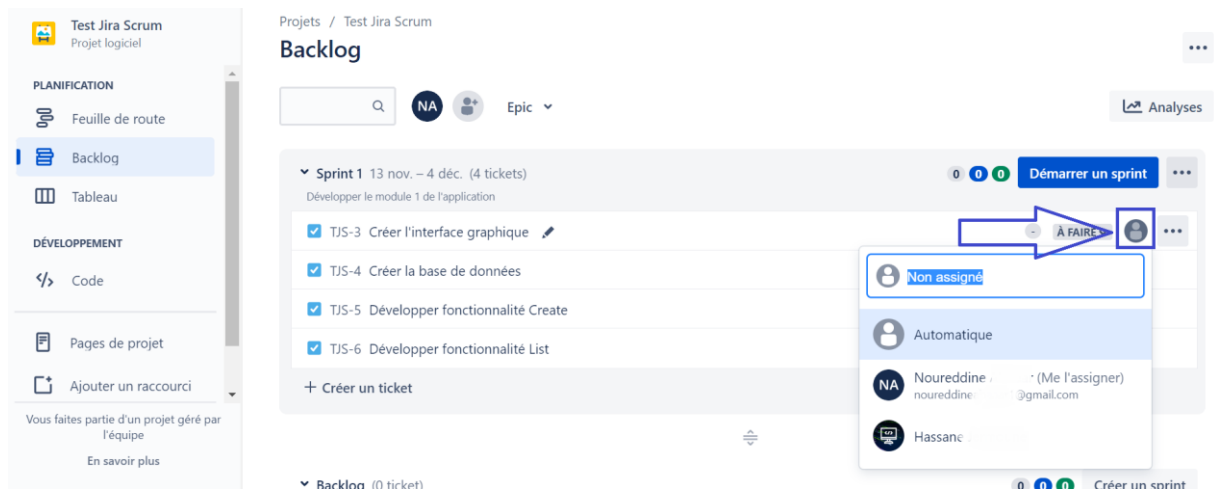
La personne doit accepter une invitation envoyée via son email et créer son compte JIRA s'il n'a pas de compte.



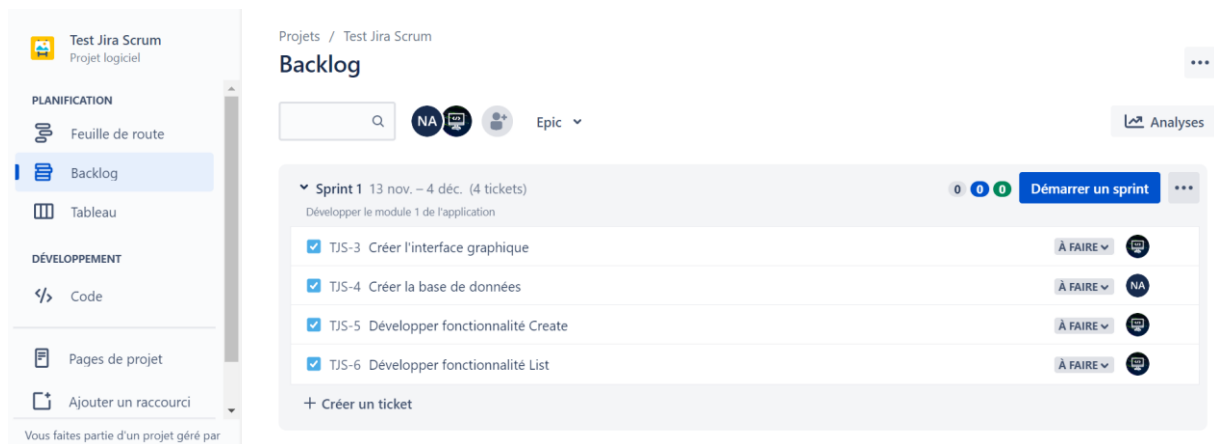
Maintenant, vous aurez le droit d'affecter les tâches à cette personne.

3.2.4.5 - Affecter une tâche à une personne :

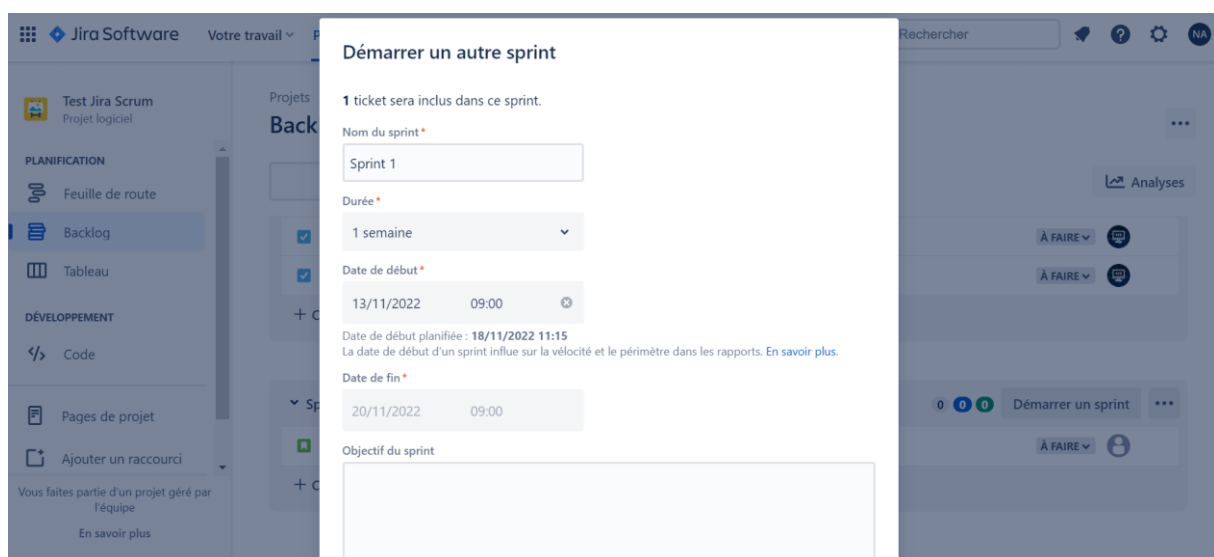
Pour affecter une tâche à une personne, cliquez sur



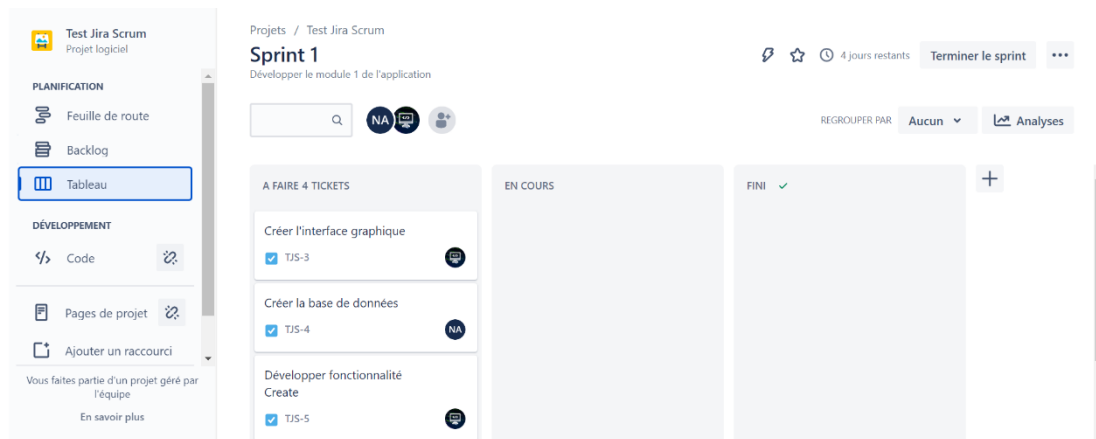
Après l'affectation des tâches, vous pouvez lancer votre sprint en cliquant sur « **Démarrer un sprint** »



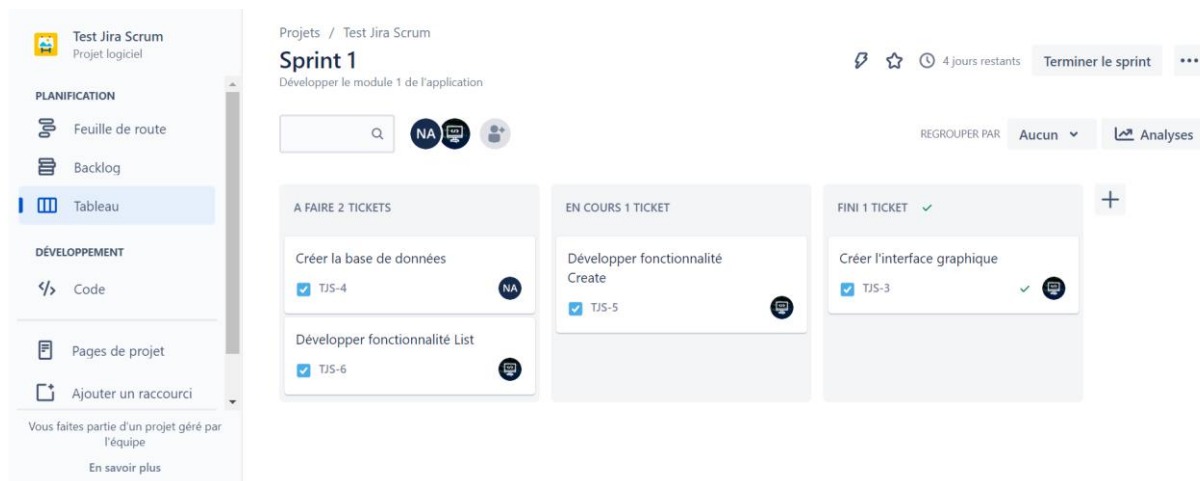
Après avoir lancé votre sprint



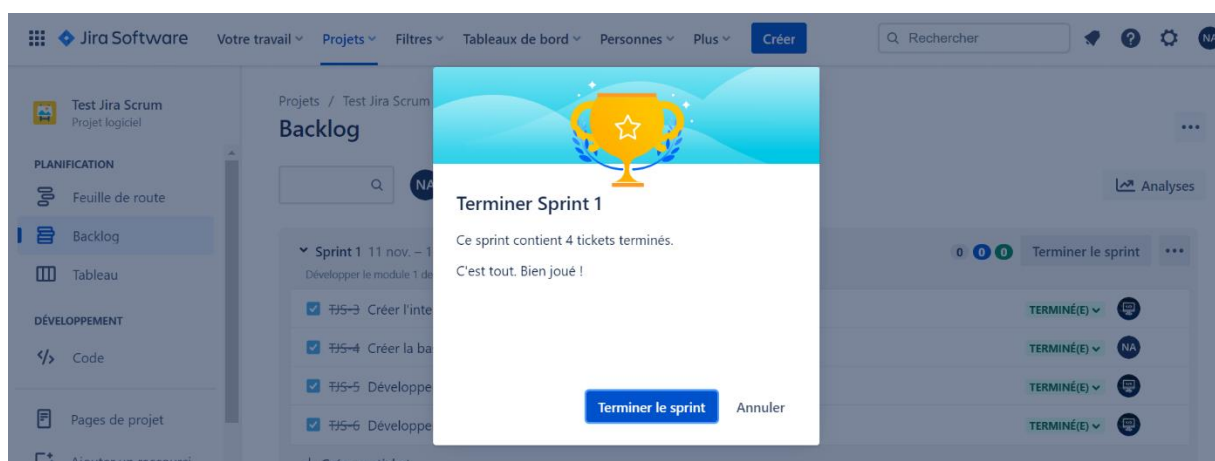
Les tâches seront affichées dans le tableau de projet et la personne peut changer l'état de tâche.



Si la personne a changé l'état d'une tâche, la modification sera partagée avec tous les membres du projet.



Si toutes les tâches sont terminées, vous pouvez Terminer le sprint en cours et commencer le sprint suivant



4- Outils de gestion de versions et de mesure de la qualité du code

4.1 - Les outils de gestion de versions (git/gitlab)

4.1.1 - Introduction

Outil de gestion de version est un outil (logiciel) permettant d'enregistrer, de suivre et de gérer plusieurs versions d'un fichier ou d'un code source. Il permet d'établir un historique de toutes les modifications effectuées sur un élément, pour ainsi avoir la possibilité de récupérer une version antérieure selon la date et l'heure de la sauvegarde, et ce en cas d'erreur ou de problème sur une version actuelle.

Le gestionnaire de version est :

- Un outil qui garde un historique des différentes mises à jour d'une application ou d'un logiciel.
- Un meilleur travail collaboratif : gestion de plusieurs versions du code source.

Exemples :

- **Git** est un logiciel de gestion de versions décentralisé. C'est un logiciel libre et gratuit, créé en 2005 par Linus Torvalds, auteur du noyau Linux, et distribué selon les termes de la licence publique générale GNU version 2.
- **Mercurial** est un logiciel de gestion de versions décentralisé disponible sur la plupart des systèmes Unix et Windows.

4.1.2 - Présentation de git et gitlab

C'est quoi Git ?

Git est un outil de gestion de version ou VCS en anglais (version control system) qui permet de stocker un ensemble de fichiers en conservant la chronologie de toutes les modifications qui ont été effectuées dessus.

Il fait parti de la famille des VCS dit décentralisés car dans son fonctionnement chaque développeur va avoir en local une copie complète de l'historique de son code source.

C'est quoi GitLab?

GitLab est un logiciel libre de forge basé sur git proposant les fonctionnalités de wiki, un système de suivi des bugs, l'intégration continue et la livraison continue.

Fonctionnalités

- Un **système de suivi des bugs** est un logiciel qui permet d'effectuer un suivi des bugs signalés dans le cadre d'un projet de développement de logiciel.
- **L'intégration continue (CI)** est un ensemble de pratiques utilisées en génie logiciel consistant à vérifier à chaque modification de code source que le résultat des modifications ne produit pas de régression dans l'application développée.
- **La livraison continue (CD)** est une approche d'ingénierie logicielle dans laquelle les équipes produisent des logiciels dans des cycles courts, ce qui permet de le mettre à disposition à n'importe quel moment. Le but est de construire, tester et diffuser un logiciel plus rapidement.
- Un **wiki** est une application web qui permet la création, la modification et l'illustration collaboratives de pages à l'intérieur d'un site web.

4.1.3 - Fonctionnement de base de Git

Git est un système de gestion de version décentralisé. Cela signifie que les données du dépôt Git ne se trouvent pas sur un serveur distant mais bel et bien sur votre machine.

Le mode décentralisé présente beaucoup d'avantages :

- Git est extrêmement rapide à mettre en place : en 3 secondes, on a créé un dépôt sans avoir besoin de tripatouiller un serveur quelconque ;
- le dépôt n'étant pas dépendant du réseau, les opérations sont très rapides et vous pouvez travailler n'importe où, e.g dans le train ;
- il est possible de créer autant de dépôts que l'on veut sur une même machine.

Un dépôt Git (repository)

Un “dépôt” correspond à la copie et à l’importation de l’ensemble des fichiers d’un projet dans Git. Il existe deux façons de créer un dépôt Git :

- On peut importer un répertoire déjà existant dans Git ;
- On peut cloner un dépôt Git déjà existant.

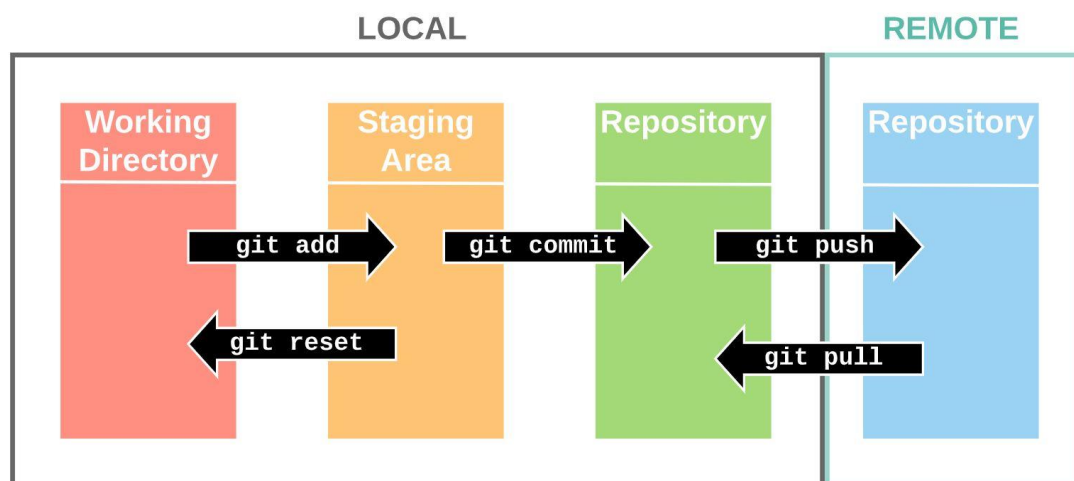
Les états des fichiers

Git gère trois états dans lesquels les fichiers peuvent résider : modifié, indexé et validé.

- **Modifié (“modified”)** signifie que vous avez modifié le fichier mais qu’il n’a pas encore été validé en base.
- **Indexé (“staged”)** signifie que vous avez marqué un fichier modifié dans sa version actuelle pour qu’il fasse partie du prochain instantané du projet.
- **Validé (“committed”)** signifie que les données sont stockées en sécurité dans votre base de données locale.

Les zones de travail

Les états de fichiers sont liés à des zones de travail dans Git. En fonction de son état, un fichier va pouvoir apparaître dans telle ou telle zone de travail. Tout projet Git est composé de trois sections : le répertoire de travail, la zone d’index et le répertoire Git.



Le **répertoire de travail (working tree)** correspond à une extraction unique d'une version du projet. Les fichiers sont extraits de la base de données compressée située dans le répertoire Git et sont placés sur le disque afin qu'on puisse les utiliser ou les modifier.

La **zone d'index (staging area)** correspond à un simple fichier, généralement situé dans le répertoire Git, qui stocke les informations concernant ce qui fera partie du prochain instantané ou du prochain "commit".

Le **répertoire Git (repository)** est l'endroit où Git stocke les méta-données et la base de données des objets de votre projet. C'est la partie principale ou le coeur de Git.

4.1.4 - Manipulation des commandes de base de Git

CONFIGURATION DES OUTILS

Configurer les informations de l'utilisateur pour tous les dépôts locaux

git config

Pour configurer les informations de l'utilisateur pour tous les dépôts locaux. Le flag -global rend ces changements persistants sur votre machine plutôt qu'uniquement pour le projet.

```
git config --global user.name <votre_nom_d'utilisateur>
git config --global user.email <votre_adresse_email>
```

CRÉER DES DÉPÔTS

Démarrer un nouveau dépôt ou en obtenir un depuis une URL existante

git init

Cette commande est utilisée pour créer un nouveau dépôt GIT :

```
git init
```

git clone

La commande git clone est utilisée pour la vérification des dépôts et le téléchargement d'un projet et tout son historique de versions. Si le dépôt se trouve sur un serveur distant, utilisez

```
git clone git@github.com:name/chemin/vers/dépôt
```

Inversement, si une copie de travail d'un dépôt local doit être créée, utilisez:

```
git clone /chemin/vers/dépôt
```

EFFECTUER DES CHANGEMENTS

Consulter les modifications et effectuer une opération de commit

git add

La **commande git add** peut être utilisée pour ajouter des fichiers à l'index. Par exemple, la commande suivante ajoutera un fichier nommé temp.txt dans le répertoire local de l'index :

```
git add temp.txt
```

git status

La **commande git status** affiche la liste des fichiers modifiés ainsi que les fichiers qui doivent encore être ajoutés ou validés. Usage :

```
git status
```

git commit

La **commande git commit** permet de **valider les modifications apportées** au HEAD. Notez que tout commit ne se fera pas dans le dépôt distant.

```
git commit -m "Description du commit"
```

git reset

Pour réinitialiser l'index et le répertoire de travail à l'état du dernier commit, la **commande git reset** est utilisée :

```
git reset --hard HEAD
```

git diff

La **commande git diff** permet de lister les conflits. Pour visualiser les conflits d'un fichier :

```
git diff --base <nom-fichier>
```

La commande suivante est utilisée pour afficher les conflits entre les branches à fusionner:

```
git diff <branche-source> <branche-cible>
```

Pour simplement énumérer tous les conflits actuels, utilisez :

```
git diff
```

GROUPER DES CHANGEMENTS

Nommer une série de commits et combiner les résultats de travaux terminés

git branch

La **commande git branch** peut être utilisée pour répertorier, créer ou supprimer des branches. Pour répertorier toutes les branches présentes dans le dépôt, utilisez :

```
git branch
```

Pour supprimer une branche :

```
git branch -d <nom-branche>
```

git checkout

La **commande git checkout** peut être utilisée pour créer des branches ou pour basculer entre elles. Par exemple nous allons créer une branche :

```
git checkout -b <nom-branche>
```

Pour passer simplement d'une branche à une autre, utilisez :

```
git checkout <nom-branche>
```

git merge

La **commande git merge** est utilisée pour fusionner une branche dans la branche active.

```
git merge <nom-branche>
```

SYNCHRONISER LES CHANGEMENTS

Référencer un dépôt distant et synchroniser l'historique de versions

git push

Git push est une autre commandes GIT de base. Un simple push envoie les modifications locales apportées à la branche principale associée :

```
git push origin master
```

git pull

Pour fusionner toutes les modifications présentes sur le dépôt distant dans le répertoire de travail local, la commande pull est utilisée.

```
git pull
```

VÉRIFIER L'HISTORIQUE DES VERSIONS

Suivre et inspecter l'évolution des fichiers du projet

git log

L'exécution de cette commande montre l'historique des versions pour la branche courante.

```
git log
```

git show

L'exécution de cette commande montre les modifications de métadonnées et de contenu incluses dans le commit spécifié.

```
git show [commit]
```

CHANGEMENTS AU NIVEAU DES NOMS DE FICHIERS

Déplacer et supprimer des fichiers sous suivi de version

git rm

Git rm peut être utilisé pour supprimer des fichiers de l'index et du répertoire de travail.
Usage :

```
git rm nomfichier.txt
```

git mv

L'exécution de cette commande renomme le fichier et prépare le changement pour un commit :

```
git mv [fichier-nom] [fichier-nouveau-nom]
```

4.1.5 - Manipulation des dépôts avec gitlab

Installation de Git

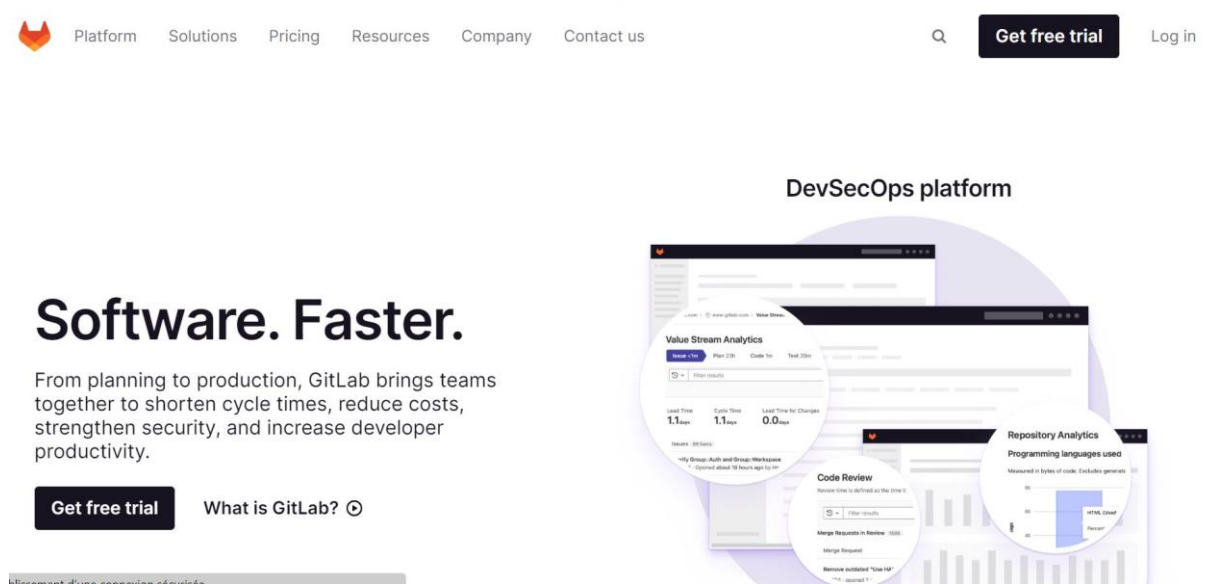
Installer git via le lien <https://git-scm.com/downloads>

Maintenant que vous avez installé Git sur votre système, vous voudrez personnaliser votre environnement Git en exécutant les deux commandes suivantes :

```
git config --global user.email "YOUR_EMAIL"
git config --global user.name "YOUR_NAME"
```

Création de compte sur gitlab

Allez sur <https://gitlab.com/> et suivez les instructions pour la création et la configuration de votre compte.

The image shows the GitLab homepage. At the top, there's a navigation bar with the GitLab logo, links for Platform, Solutions, Pricing, Resources, Company, and Contact us, a search icon, a 'Get free trial' button, and a 'Log in' link. The main section features the headline 'Software. Faster.' followed by a paragraph: 'From planning to production, GitLab brings teams together to shorten cycle times, reduce costs, strengthen security, and increase developer productivity.' Below this is another 'Get free trial' button and a link 'What is GitLab?'. To the right, there's a 'DevSecOps platform' section with a collage of screenshots showing various GitLab features like Value Stream Analytics, Repository Analytics, and Code Review.

Ajouter vos informations ou se connecter via Gmail/GitHub :



Free 30-day trial

No credit card required.

- ✓ Increase Operational Efficiencies
- ✓ Deliver Better Products Faster
- ✓ Reduce Security & Compliance Risk

Used by more than 100,000 organizations from around the globe:

SIEMENS CH@IRUS KnowBe4

Wish hotjar

Register with:



or

First name

Last name

Username

Email

We recommend a work email address.

Password

Minimum length is 8 characters.

☐

Je ne suis pas un robot



Acceptez les conditions d'utilisation

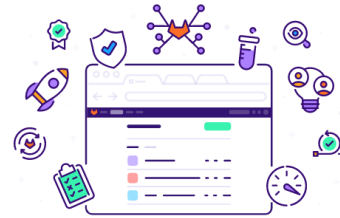
Ajouter l'entreprise, exemple : votre établissement.



About your company

To activate your trial, we need additional details from you.

Company Name	Number of employees
ISTA TAFRAOUT	1 - 99
Country	
Morocco	
Telephone number (optional)	
Allowed characters: +, 0-9, -, and spaces.	
Website (optional)	
Continue	



Créer votre premier projet



Create or import your first project

Projects help you organize your work. They contain your file repository, issues, merge requests, and so much more.

Create	Import
Group name	
istatafraout	
Project name	
tp-gitlab	
Your project will be created at:	
https://gitlab.com/istatafraout/tp-gitlab	
You can always change your URL later	
☒ Include a Getting Started README	
Recommended if you're new to GitLab	
Create project	

Cliquer sur **Ok, let's go** pour accéder à votre premier projet.

The screenshot shows the GitLab 'Learn GitLab' tutorial page. The left sidebar contains a navigation menu with items like 'Project information', 'Learn GitLab' (highlighted with a 22% progress bar), 'Repository', 'Issues', 'Merge requests', 'CI/CD', 'Security & Compliance', 'Deployments', 'Packages and registries', 'Infrastructure', 'Monitor', and 'Analytics'. The main content area is titled 'Learn GitLab' and includes a progress bar showing '22% completed'. Below the progress bar, there are three main sections: 'Set up your workspace', 'Plan and execute', and 'Deploy'. Each section contains a list of tasks and links to guide the user through the setup process.

Learn GitLab

Ready to get started with GitLab? Follow these steps to set up your workspace, plan and commit changes, and deploy your project.

22% completed

- Set up your workspace**
 - Complete these tasks first so you can enjoy GitLab's features to their fullest:
 - [Invite your colleagues](#)
 - [Create a repository](#)
 - [Set up your first project's CI/CD](#)
 - [Start a free trial of GitLab Ultimate](#)
- Plan and execute**
 - Create a workflow for your new workspace, and learn how GitLab features work together:
 - [Create an issue](#)
 - [Submit a merge request \(MR\)](#)
- Deploy**
 - Use your new GitLab workflow to deploy your application, monitor its health, and keep it secure:
 - [Run a Security scan using CI/CD](#)

Félicitation ! Votre projet a été bien créé.

The screenshot shows the GitLab project page for 'tp-gitlab'. The left sidebar is the same as in the previous screenshot. The main content area displays the project details, including the project ID (41268226), storage size (72 KB), and a list of commits. The 'Initial commit' by Nouredine ABABAR is highlighted, showing the commit hash (71411d75) and the time (35 minutes ago). Below the commit list, there are buttons for 'README', 'Add LICENSE', 'Add CHANGELOG', 'Add CONTRIBUTING', 'Add Kubernetes cluster', and 'Set up CI/CD'. A table at the bottom shows the commit history.

tp-gitlab

Project ID: 41268226

1 Commit 1 Branch 0 Tags 72 KB Project Storage

main tp-gitlab / +

Find file Web IDE Clone

Initial commit
Nouredine ABABAR authored 35 minutes ago

71411d75

[README](#) [Add LICENSE](#) [Add CHANGELOG](#) [Add CONTRIBUTING](#) [Add Kubernetes cluster](#) [Set up CI/CD](#)

[Configure Integrations](#)

Name	Last commit	Last update
README.md	Initial commit	35 minutes ago

Cloner un repository distant (projet) en local :

Méthode 1 : Cloner avec HTTPS

Exécuter la commande pour cloner le projet en local :

```
git clone https://gitlab.com/istatafraout/tp-gitlab-repo.git
```

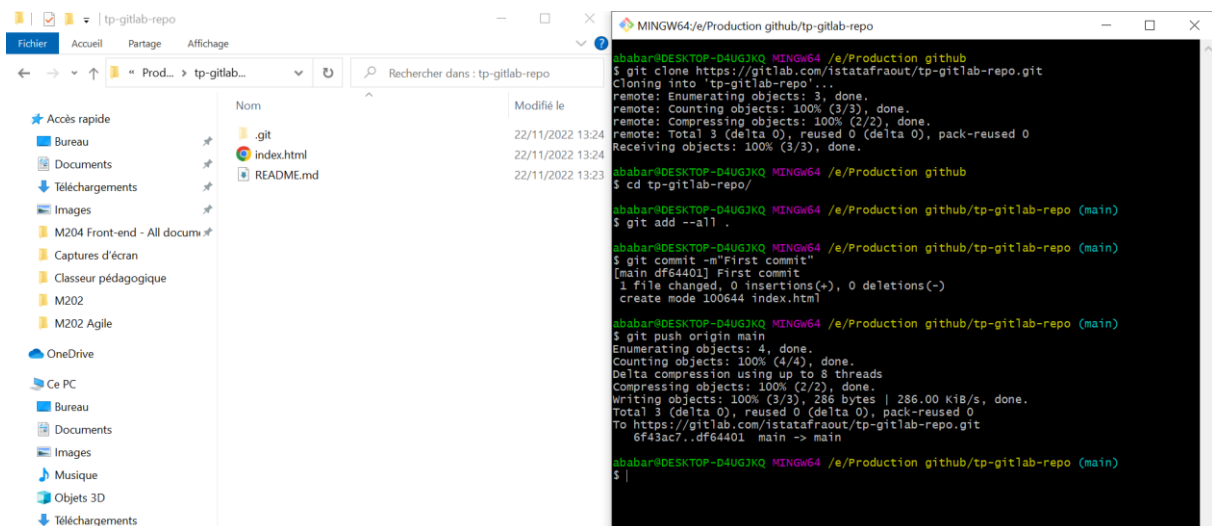
Accéder au dossier de travail :

```
cd tp-gitlab-repo/
```

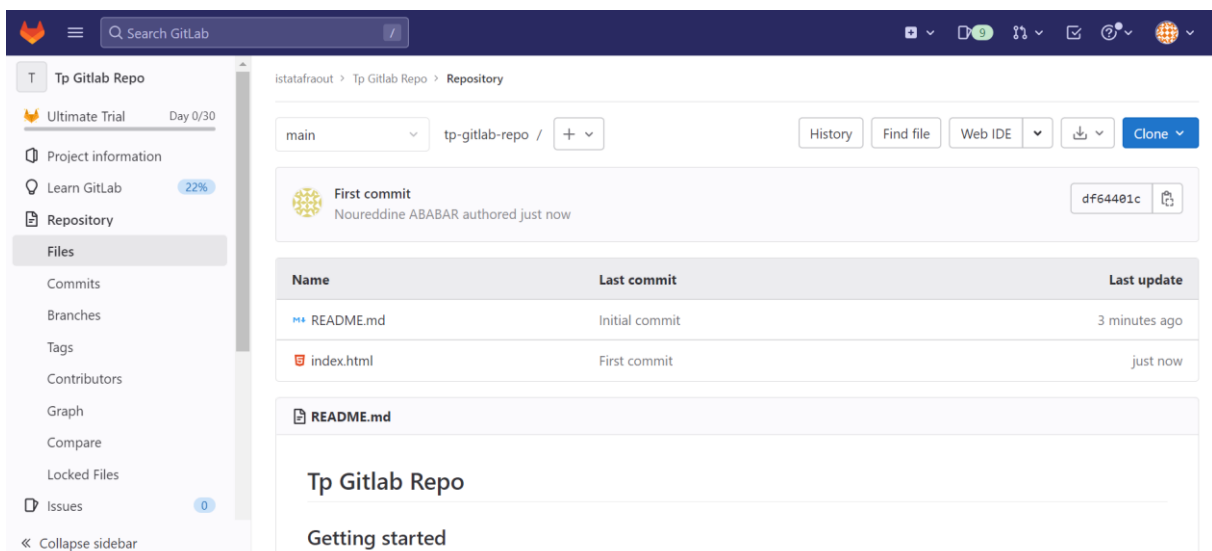
Ajouter un fichier, exemple **index.html**

Envoyer les fichiers en repository distant

```
git add --all .  
git commit -m "First commit"  
git push origin main
```



Vérifier sur GitLab que le fichier a été envoyé.



Méthode 2 : Cloner avec SSH

Sur votre git bush, lancer les commandes suivantes pour générer un code SSH :

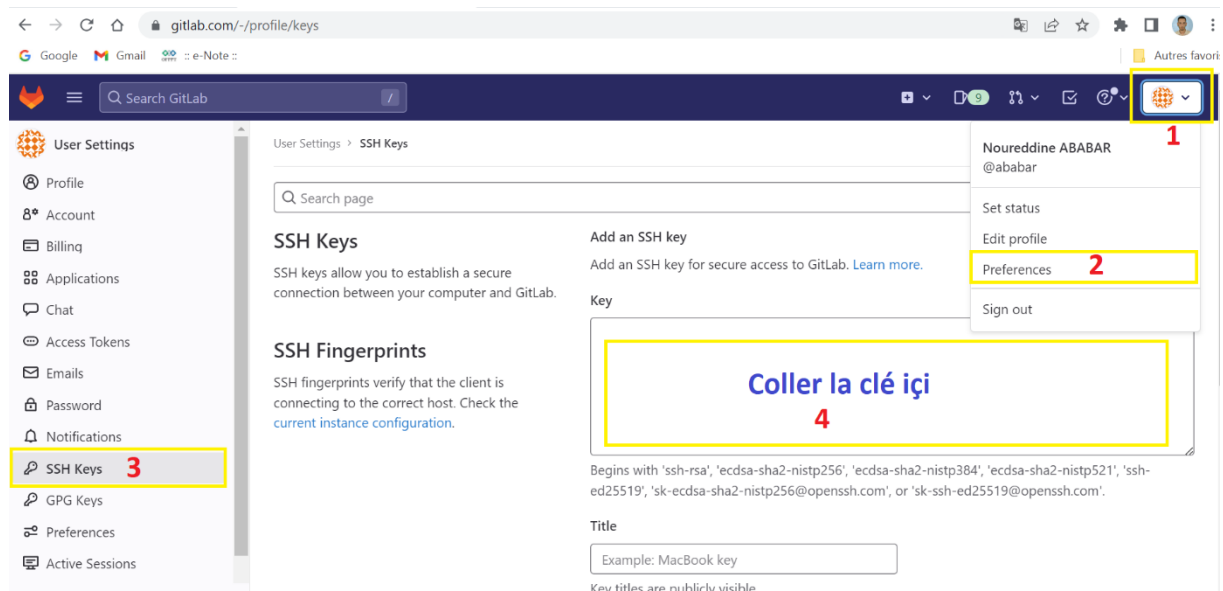
```
cd ~/.ssh  
ssh-keygen -t ed25519 -C "Votre email"  
cat id_ed25519.pub
```

Copier la clé SSH



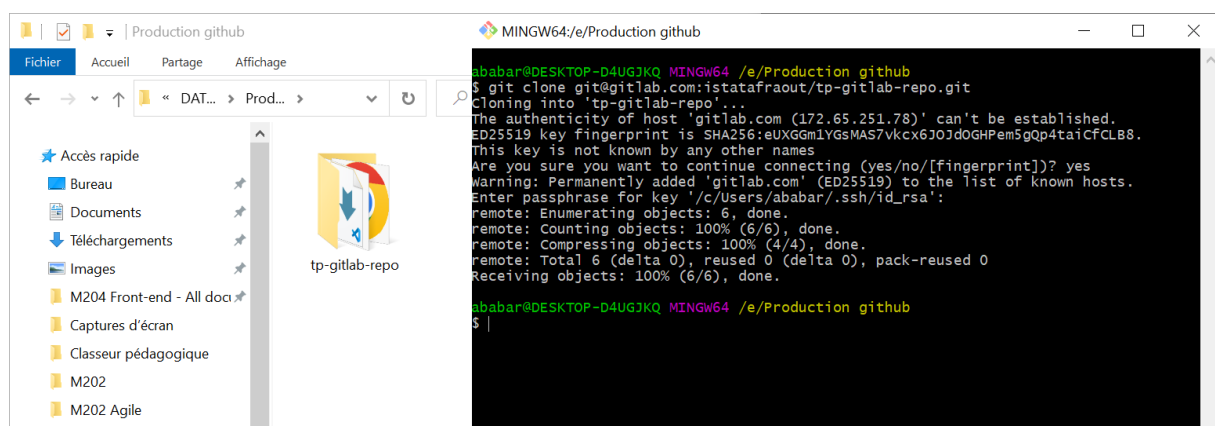
```
MINGW64:/c/Users/ababar/.ssh  
ababar@DESKTOP-D4UGJKQ MINGW64 ~/.ssh  
$ cd ~/.ssh  
ababar@DESKTOP-D4UGJKQ MINGW64 ~/.ssh  
$ cat id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIA49KfKLpcOF0p34YXG5fVmiSOq  
@gmail.com
```

Coller la clé SSH copiée dans GitLab



Exécuter la commande pour cloner le projet en local :

```
git clone git@gitlab.com:istatafraout/tp-gitlab-repo.git
```



Accéder au dossier de travail :

```
cd tp-gitlab-repo/
```

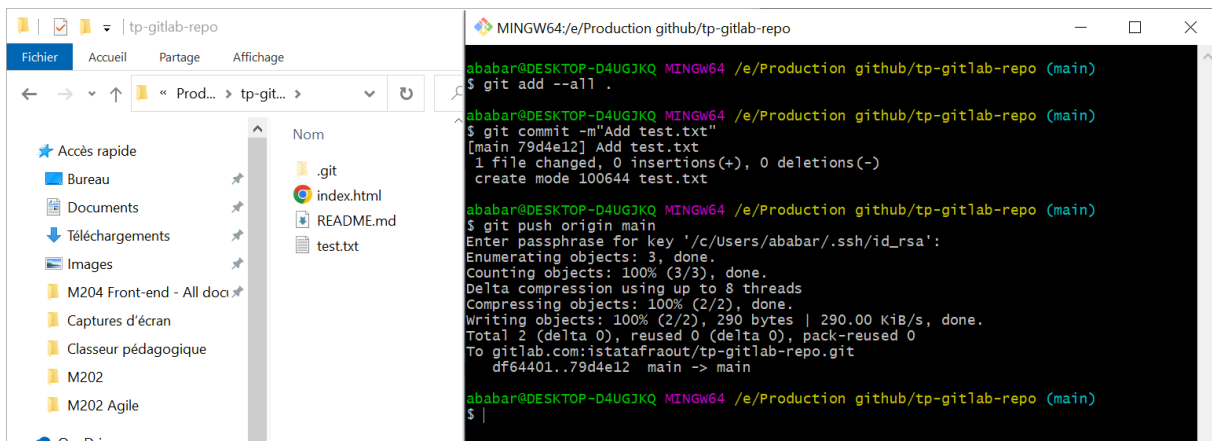
Ajouter un fichier, exemple **test.txt**

Envoyer les fichiers en repository distant

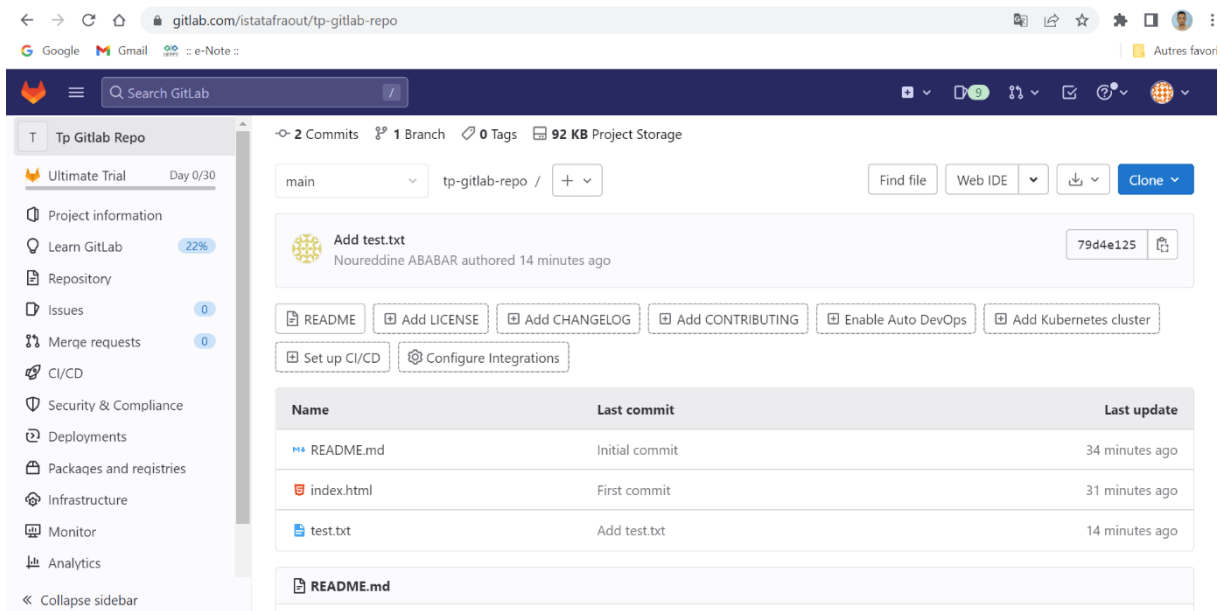
```
git add --all .
```

```
git commit -m "Add test.txt"
```

```
git push origin main
```



Vérifier sur GitLab que le fichier a été envoyé.

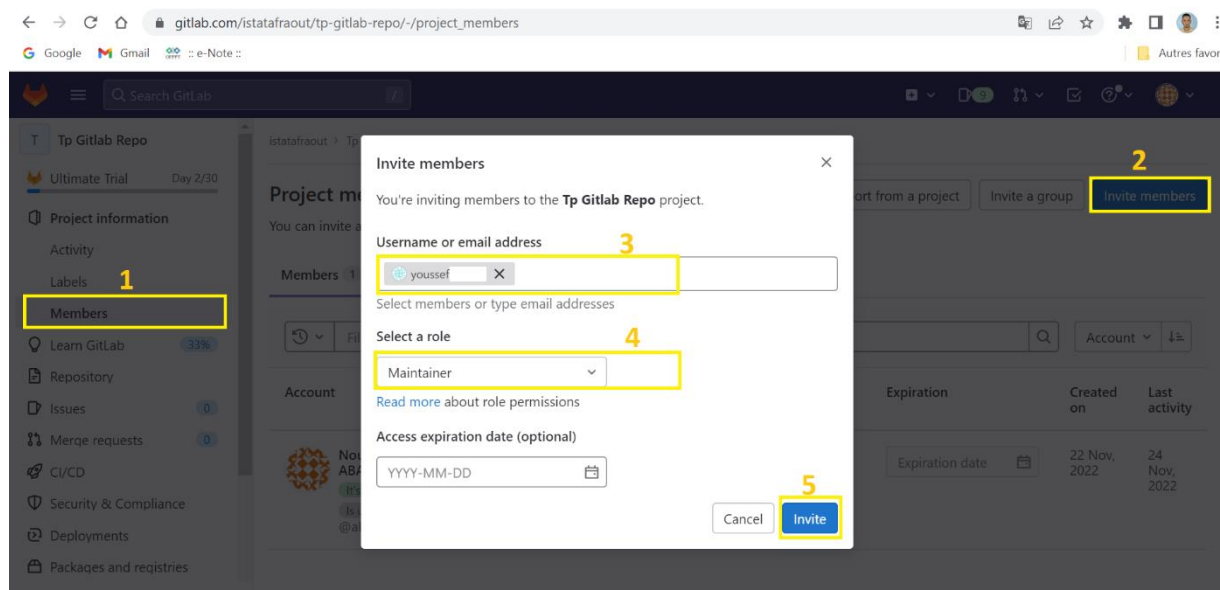


4.1.6 - Collaboration en gitlab

Pour ajouter un développeur dans une équipe Gitlab :

1. Depuis la barre latérale de la page du dépôt, cliquer sur “**Projet information**”
2. Cliquer sur “**Membres**” et ensuite “**Invite membres**”
3. Taper le ou les noms d'utilisateurs (ou adresse email) du/des développeurs à ajouter
4. Sélectionner le rôle (ou niveau de permissions) à donner à ce(s) développeur(s)
5. Vérifier que le(s) développeur(s) est/sont bien capables d'ajouter et pusher un commit dans la branche de travail du dépôt

Note : pour qu'un développeur ait le droit de pusher des commits dans votre dépôt, il faut lui donner le rôle de “Maintainer”. Le rôle “Developer” ne suffit pas.



4.1.7 - Manipulation des branches

Presque tous les VCS proposent une certaine forme de gestion de branches. Créer une branche signifie diverger de la ligne principale de développement et continuer à travailler sans impacter cette ligne. Pour de nombreux VCS, il s'agit d'un processus coûteux qui nécessite souvent la création d'une nouvelle copie du répertoire de travail, ce qui peut prendre longtemps dans le cas de gros projets.

Une branche dans Git est simplement un pointeur léger et déplaçable vers un de ces commits. La branche par défaut dans Git s'appelle **main**. Au fur et à mesure des validations, la branche main pointe vers le dernier des commits réalisés. À chaque validation, le pointeur de la branche main avance automatiquement.

La commande git branch permet en fait bien plus que la simple création et suppression de branches.

Si vous la lancez sans argument, vous obtenez la liste des branches courantes :

```
git branch
```

```
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git branch
* main
```

Créer une nouvelle branche

Que se passe-t-il si vous créez une nouvelle branche ? Eh bien, cela crée un nouveau pointeur pour vous. Supposons que vous créez une nouvelle branche nommée dev. Vous utilisez pour cela la commande git branch :

```
git branch dev
```

```
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git branch dev

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git branch
  dev
* main
```

Basculer entre les branches

Pour basculer sur une branche existante, il suffit de lancer la commande git checkout. Basculons sur la nouvelle branche dev :

```
git checkout dev
```

```
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git checkout dev
Switched to branch 'dev'

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (dev)
$ |
```

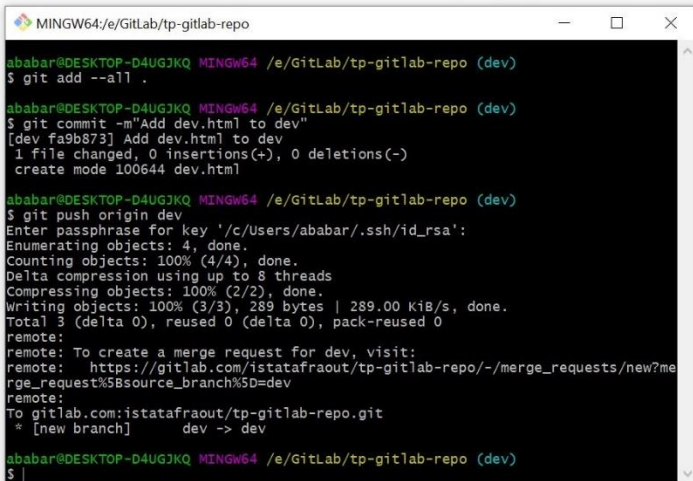
Explication :

Maintenant, nous avons créé deux branches et chaque modification effectuée sur la branche dev ne concerne pas la branche main et vice-versa.

Supposons que j'ai ajouté un fichier dev.html dans la branche dev

DATA (E) > GitLab > tp-gitlab-repo





La branche main contiendra

main	tp-gitlab-repo /	+	History	Find file	Web IDE	↓	Clone
Name	Last commit	Last update					
M+ README.md	Initial commit	52 minutes ago					

Et la branche dev contiendra

dev tp-gitlab-repo / + History Find file Web IDE Clone

Name	Last commit	Last update
 README.md	Initial commit	52 minutes ago
 dev.html	Add dev.html to dev	4 minutes ago

4.1.8 - Fusion et gestion des conflits

Pour fusionner deux branches ici on va à nouveau se placer dans la branche dans laquelle on souhaite fusionner puis effectuer un git merge.

Pour fusionner la branche dev avec la branche main, il faut se placer dans la branche main (get checkout main) et exécuter la commande

```
git merge dev
```

Cette commande va télécharger toutes les modifications effectuées dans la branche dev vers la branche main.



```
MINGW64:/e/GitLab/tp-gitlab-repo
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (dev)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git merge dev
Updating c2a313e..fa9b873
Fast-forward
 dev.html | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 dev.html

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git push origin main
Enter passphrase for key '/c/Users/ababar/.ssh/id_rsa':
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To gitlab.com:istatafraout/tp-gitlab-repo.git
 c2a313e..fa9b873  main -> main

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$
```

Gestion des conflits :

Sur la branche main	Sur la branche dev
Le développeur a modifié h1 du fichier dev.html et envoyer les modifications à dépôt distant (push)	Le développeur a modifié h1 du fichier dev.html et envoyer les modifications à dépôt distant (push)
Hello My Friend	Hello World!

Si vous avez se placer dans la branche main et essayer de fusionner la branche main avec dev, vous aurez un message de conflit

```
MINGW64:/e/GitLab/tp-gitlab-repo
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git merge dev
Auto-merging dev.html
CONFLICT (content): Merge conflict in dev.html
Automatic merge failed; fix conflicts and then commit the result.
```


Allez vers le fichier concerné et gérer ce conflit

```
<<<<<< HEAD
      <h1>Hello My Friend</h1>
=====
      <h1>Hello World!</h1>
>>>>>> dev
```

Dans ce cas, vous pouvez laisser le bon code et envoyer les modifications au dépôt distant

```
ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main|MERGING)
$ git add --all .

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main|MERGING)
$ git commit -m'conflict res-1'
[main 71ab290] conflict res-1

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ git push origin main
Enter passphrase for key '/c/Users/ababar/.ssh/id_rsa':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 221 bytes | 221.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To gitlab.com:istatafraout/tp-gitlab-repo.git
   a1860ea..71ab290  main -> main

ababar@DESKTOP-D4UGJKQ MINGW64 /e/GitLab/tp-gitlab-repo (main)
$ |
```


4.2 - L'outil de mesure de la qualité du code (SonarQube)

4.2.1 - Notions des métriques de la qualité du code

La qualité du code

La qualité du code, aussi appelée qualité structurelle du logiciel, est différente de la qualité d'un logiciel, aussi appelée qualité fonctionnelle d'un logiciel, dans le sens où la première concerne la manière avec laquelle une fonctionnalité est implémentée et la seconde le résultat final.

La mesure de la qualité du code peut en partie être mesurée automatiquement par des outils d'analyse statique du code source. Ces outils sont disponibles pour la plupart des langages et proposent de **nombreuses métriques** et conventions standards.

Définition d'une métrique :

Métrique logicielle : mesure d'une propriété d'un logiciel (par exemple le nombre de lignes de codes), c'est une approche quantitative : extraire une mesure de la qualité d'un logiciel à partir de l'analyse statistique du code source.

Métriques standards :

Complexité cyclomatique

C'est la mesure du nombre de chemins linéaires possibles dans une fonction. Plus simplement, il s'agit du nombre de points de décision de la méthode (if, case, while, ...) + 1 (le chemin principal).

Une méthode trop complexe ou trop longue poserait les problèmes suivants : manque de lisibilité du code, Perte d'informations pendant le débogage, Tests unitaires moins efficaces.

Un code source qui présenterait trop de méthodes avec une complexité cyclomatique importante verrait sa valeur sensiblement réduite étant donné sa maintenabilité moindre.

SLOC (ligne de code source)

C'est une métrique logicielle servant à mesurer la taille d'un programme informatique en dénombrant le nombre de lignes de son code source.

On différencie principalement deux manières de calculer : la mesure « physique » et la mesure « logique ». La première se base sur le nombre de lignes présentes dans le fichier sans autre traitement. La seconde ne prend que les lignes qui seront effectivement exécutées, ce qui permet d'éliminer l'impact du style différent d'un programmeur à l'autre.

Densité des commentaires

En fonction de la formule de calcul employée, il s'agit du rapport entre le nombre de lignes de code (SLOC) et le nombre de commentaires (CLOC). La formule la plus simple étant $DC = CLOC / LOC$.

La qualité du code source dépend souvent de la manière avec laquelle il a été commenté. L'absence partielle ou totale de commentaires pourrait être un signe de mauvaise qualité.

Pour les raisons suivantes : le nombre de commentaires nécessaires dépend grandement du type de langage, le code généré n'inclut pas souvent les commentaires, la nécessité de commentaire dépend directement de la clarté du code, il n'est pas recommandé d'imposer un tel niveau de commentaire mais d'instaurer un système de contrôle lors des revues de code. Il serait également intéressant de former les développeurs à écrire de meilleurs commentaires.

Couverture de code

La couverture de code représente la proportion du code source qui est couverte par des tests, généralement automatisés. Le calcul est exprimé en pourcentage et peut être obtenu sur base du nombre d'instructions, de méthodes ou de classes parcourues. Cela inclut les tests unitaires (unit tests) mais aussi les tests fonctionnels et de validation.

Un taux de couverture élevé par des tests unitaires présente de nombreux avantages.

Duplication de code

La duplication de code (ou code clone) consiste en la répétition d'instructions similaires ou identiques dans un code source.

Le code dupliqué pose des problèmes de maintenance dont l'importance augmente avec la quantité de code dupliqué.

La détection du code dupliqué est donc un bon moyen d'augmenter la qualité du code en permettant d'identifier les parties à retravailler.

Afferent & efferent coupling

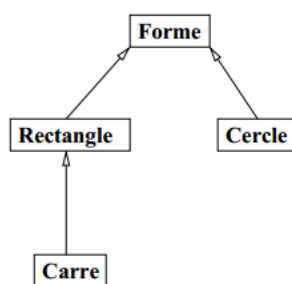
L'Afferent coupling représente le nombre de références vers la classe mesurée. Ces références doivent être externes à cette classe. Les références internes ne comptent pas. Cette métrique donne une bonne indication de l'importance de la classe dans le code.

L'Efferent coupling est la mesure du nombre de types que la classe « connaît ». Cela comprend : l'héritage, l'implémentation d'interfaces, les types des paramètres, les types de variable, les exceptions levées et capturées. En bref, tous les types mentionnés dans le code source de la classe mesurée.

L'instabilité

L'instabilité d'un module est définie par son niveau de résistance au changement. Plus un module est stable, moins il est facile de le changer. Elle est obtenue en calculant le rapport entre l'efferent coupling au couplage total. Cette valeur est donc obtenue en divisant l'efferent coupling à la somme de l'efferent coupling et l'afferent coupling.

Prenons un exemple avec le principe de l'héritage en orienté objet.



Classe	Afferent coupling	Efferent coupling	Instability ($C_e / (C_e + C_a)$)
Forme	2	0	$0 / (0 + 2) = 0$
Rectangle	1	1	$1 / (1 + 1) = 0.5$
Cercle	0	1	$1 / (1 + 0) = 1$
Carre	0	1	$1 / (1 + 0) = 1$

Ainsi Forme est très stable car le rapport entre ses dépendances efférentes sont nulles par rapport à ses dépendances afférentes. Le cas de Rectangle démontre une instabilité moyenne. Par contre Cercle et Carre sont très instables.

Abstractness

L'abstractness d'un module indique son niveau d'abstraction par rapport aux autres classes présentes dans le code. Le niveau d'abstraction d'un paquetage est simplement le rapport entre le nombre d'interfaces et le nombre total de types de ce paquetage :

Cette métrique, à elle seule, n'est pas utile. Combinée avec la métrique Instability, elle permet de calculer la Distance from main sequence.

Distance from main sequence

La distance from main sequence indique l'équilibre du module entre l'abstractness et l'instability. Cet indicateur est toujours compris entre 0 et 1.

$$\text{Formule de calcul} = | \text{Abstractness} + \text{Instability} - 1 |$$

Il représente en fait l'équilibre qui doit résider entre le niveau d'abstraction et l'indice d'instabilité. Plus la valeur est proche de 0, plus le design du paquetage est considéré comme bon.

Nombre de paramètres d'une méthode

Les méthodes avec trop de paramètres peuvent rendre le code moins performant mais aussi en réduire la visibilité. Il est recommandé d'avoir un nombre de paramètres réduit.

Il existe principalement deux manières de réduire les paramètres d'une méthode. La première est de privilégier l'usage de propriétés ou champs sur la classe pour traiter les différents états. La seconde consiste à fournir l'instance d'une classe ou structure dont le rôle est de contenir les différents paramètres.

Nombre de variables d'une méthode

Un nombre trop élevé peut avoir un impact sur la lisibilité de celle-ci. Comme pour le nombre de paramètres d'une méthode, il peut être contreproductif de calculer cette valeur sur du code généré. Il est conseillé d'avoir un nombre maximum de 8 variables dans le corps d'une méthode.

Nombre de surcharges

Le nombre de surcharges (number of overloads) permet de déterminer le nombre de méthodes du même nom. Si une méthode n'est pas surchargée, la valeur sera égale à 1. Un nombre trop important de surcharges peut réduire la lisibilité du code et réduire la qualité.

Critères supplémentaires

Architecture

L'architecture d'un logiciel peut être comparée à celle d'un bâtiment à certains égards. Il ne s'agit pas vraiment de la qualité de la maçonnerie, du plafonnage ou des électros de la cuisine, mais de l'agencement des pièces et tous les autres aspects fonctionnels globaux. La qualité de cette architecture est également très importante pour évaluer la qualité du code car elle impacte de nombreux facteurs de qualités : **Maintenabilité, Evolutivité, Performance, Pertinence.**

Style & lisibilité

Le style est la première chose qu'on voit quand on regarde le code. Un effet de halo est inévitable. Cela pourrait jouer en défaveur d'une évaluation humaine. Il existe des outils d'analyse de style statique qui ne sont évidemment pas affectés par ce biais.

Enfin bonne nouvelle, beaucoup des mises en forme peuvent être automatisées dans des éditeurs de code.

Documentation Technique

En général, les outils modernes permettent de documenter les différentes classes et méthodes à l'intérieur de celles-ci, mais plus rapidement l'architecture en général. La présence d'un document décrivant cette dernière et justifiant les choix effectués est un très bon critère de qualité.

A cela s'ajoute les documents qui doivent permettre au développeur de configurer son environnement de travail afin de pouvoir commencer à travailler sur le code.

Fiabilité

La fiabilité correspond au nombre de défaillances rencontrées dans un intervalle de temps donné. Plus cette valeur est élevée, moins le logiciel est fiable. On peut mesurer cette fiabilité en procédant à des tests manuels mais aussi avec des tests de montée en charge automatisés.

Portabilité

La portabilité est la capacité d'un logiciel à fonctionner sur des systèmes différents. Les langages de haut niveau de type Java permettent généralement d'obtenir une portabilité importante.

La dépendance à une plateforme pourrait dans certains cas être un facteur influençant l'évaluation de la qualité du code.

Sécurité

La sécurité est l'un des aspects clés dans la mesure de la qualité du code. Souvent négligée, elle devient de plus en plus importante avec la montée en puissance des applications en ligne. On peut détecter certains problèmes par l'intermédiaire de logiciel d'analyse statique du code, mais cela est très limité. De plus, les exigences en matière de sécurité varient fortement d'un projet à l'autre. Elles seront très élevées pour une application bancaire, mais probablement beaucoup moins pour les logiciels dont l'usage est local ou qui traitent des informations disponibles publiquement.

Nombre de bugs

Il s'agit du nombre de problèmes perturbant l'usage normal du logiciel. Ils sont extrêmement difficiles à identifier en lisant le code ou par analyse statique et sont généralement découverts au moment où ils se manifestent. De ce fait, le nombre de bugs réel est rarement connu. Ces bugs peuvent correspondre à des violations de règles, mais la plupart du temps, il s'agit de problèmes fonctionnels très difficilement identifiables par des outils automatisés.

Tableau récapitulatif

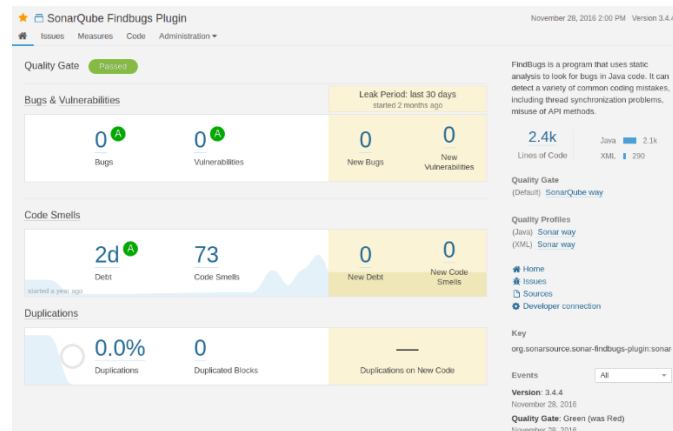
Légende : AS = Analyse statique | EX = Expert | AU = Audit | A = Application | MO = Module | C = Classe | ME = Méthode | OO = Orienté Objet

Métrique	Méthode	Langage	Localisation	Valeur repère	Intérêt
Complexité cyclomatique	AS	Tous	A-MO-C-ME	Max 10	Elevé
SLOC	AS	Tous	A-MO-C-ME	-	Faible
Densité des commentaires	AS	Tous	A-MO-C-ME	Entre 20% & 40%	Modéré
Couverture de code	AS	Tous	A-MO-C-ME	80%	Elevé
Duplication de code	AS	Tous	A-MO-C-ME	0	Faible
Afferent coupling	AS	Tous	MO-C	-	Elevé
Efferent coupling	AS	Tous	MO-C	Max 20	Elevé
Instability	AS	Tous	MO	Entre 0.0 & 0.3 et entre 0.7 et 1.0	Elevé
Abstractness	AS	Tous	MO	-	Elevé
Distance from main sequence	AS	OO	MO	Max 0.7	Elevé
Number of variables	AS	Tous	A-MO-C-ME	Max 5	Faible
Number of methods	AS	OO	C	Max 8	Faible
Nombre de surcharges	AS	Tous	C	Max 6	Faible
Architecture	EX	Tous	A-MO-C-ME	-	Elevé
Style & lisibilité	EX	Tous	A-MO-C-ME	-	Modéré
Documentation	EX	Tous	A-MO-C-ME	-	Modéré
Fiabilité	EX	Tous	A-MO-C-ME	-	Elevé
Portabilité	EX	Tous	A-MO-C-ME	-	Modéré
Sécurité	EX	Tous	A-MO-C-ME	-	Modéré
Nombre de bugs	EX	Tous	A-MO-C-ME	-	Modéré

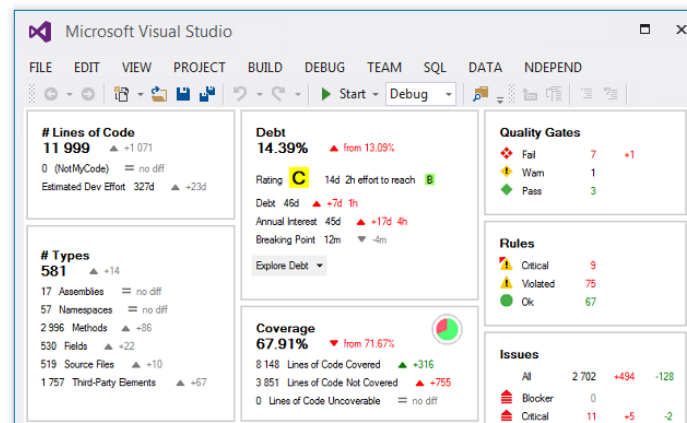
4.2.2 - Présentation des outils existants de mesure de la qualité du code

Ces outils permettent d'obtenir des métriques sur un code source de logiciel :

SonarQube est un outil de contrôle de qualité logicielle pour les développements dans plus de 25 langages dont Java, C/C++, C#, PHP, Python, COBOL.



NDepend est un outil de contrôle de qualité logicielle pour les développements .NET.



Testwell CMT++ est un outil de mesure de complexité pour C et C++

Testwell CMTJava est un outil de mesure de complexité pour Java

4.2.3 - Présentation de SonarQube

SonarQube est l'outil **d'analyse de la qualité du code et de la sécurité** le plus populaire du marché. Avec le soutien de la communauté open source, Sonarqube peut actuellement analyser et produire des sorties pour plus de 25 langages de programmation, ce qui est supérieur à la plupart des outils du marché.

Il est disponible dans une édition communautaire gratuite et d'autres éditions payantes premium.

SonarQube comprend en fait "deux sous-applications" :

- un **moteur d'analyse (le scanner)** : installé localement sur la machine du développeur.
- un **serveur centralisé** : pour la conservation des enregistrements et la création de rapports.

Une seule instance de **serveur SonarQube est capable de prendre en charge plusieurs scanners**, ce qui vous permet de centraliser les rapports sur la qualité du code provenant de nombreux développeurs en un seul endroit.

Les principaux avantages de l'utilisation de SonarQube sont :

- Donne la possibilité de générer plusieurs rapports de la qualité de logiciel et celle de niveau de concept de qualités dans le projet (Fiabilité, sécurité, maintenabilité).
- S'intègre facilement dans les pipelines CI / CD avec une seule commande de ligne.
- Peut également être intégré au cycle de construction Maven et Gradle
- Vérifie presque tout : qualité du code, formatage, déclarations de variables, gestion des exceptions et bien plus encore

SonarQube classe les défauts logiciels selon 3 catégories :

- Les **bugs** : anomalies évidentes du code. Ils impactent la fiabilité (reliability) de l'application.
- Les **vulnérabilités** : faiblesses du code pouvant nuire au système. Elles impactent la sécurité de l'application.
- Les « **code smells** » : anti-patterns (ou anti-patterns). Ils impactent la maintenabilité de l'application.

Cela vous aide à vous assurer que votre code est de la qualité attendue avant qu'il ne soit fusionné !

4.2.4 - Installation de SonarQube

Note : Le serveur SonarQube nécessite Java version 11 et les scanners SonarQube nécessitent Java version 11 ou 17.

On peut installer serveur SonarQube avec les bases de données : PostgreSQL, Microsoft SQL Server, Oracle.

Etape 1 : Installer JAVA 11

<https://www.oracle.com/java/technologies/javase/jdk11-archive-downloads.html>

Ajouter %JAVAHOME%\jdk-11.0.16\bin dans les variables de l'environnement.

Vérifier la version de java dans votre machine :

```
ababar@DESKTOP-D4UGJKQ MINGW64 ~/Desktop
$ java --version
java 11.0.16 2022-07-19 LTS
Java(TM) SE Runtime Environment 18.9 (build 11.0.16+11-LTS-199)
Java HotSpot(TM) 64-Bit Server VM 18.9 (build 11.0.16+11-LTS-199, mixed mode)
```

Etape 2 : Installer la base de données PostgreSQL et PgAdmin

Installer postgresql 13 :

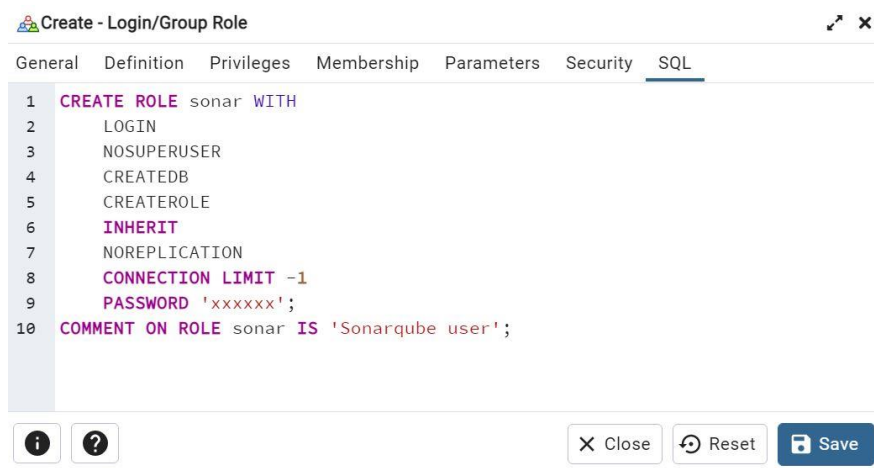
<https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>

Installer pgAdmin :

<https://www.pgadmin.org/download/>

Après l'installation, ouvrez le "pgAdmin4" et connecter à l'instance par défaut.

Créer un nouvel utilisateur "sonar" avec le mot de passe "sonar"



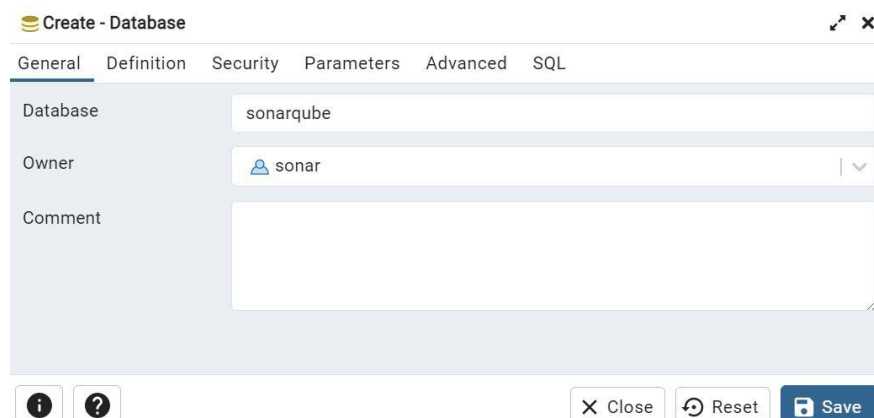
Create - Login/Group Role

General Definition Privileges Membership Parameters Security **SQL**

```
1 CREATE ROLE sonar WITH
2 LOGIN
3 NOSUPERUSER
4 CREATEDB
5 CREATEROLE
6 INHERIT
7 NOREPLICATION
8 CONNECTION LIMIT -1
9 PASSWORD 'xxxxxx';
10 COMMENT ON ROLE sonar IS 'Sonarqube user';
```

Close Reset Save

Créer la base de données "sonarqube"



Create - Database

General Definition Security Parameters Advanced SQL

Database sonarqube

Owner sonar

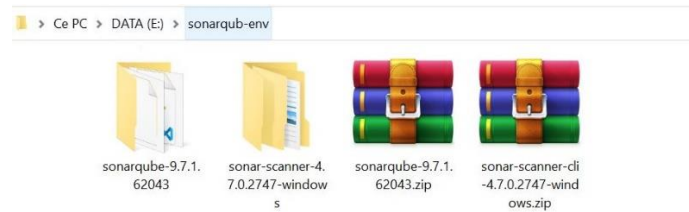
Comment

Close Reset Save

Etape 3 : Installer serveur SonarQube et SonarScanner à partir des fichiers ZIP

Serveur SonarQube : <https://www.sonarqube.org/downloads/>

SonarScanner : <https://docs.sonarqube.org/latest/analysis/scan/sonarscanner/>



Ajouter `$SonarQubeHOME\bin\windows-x86-64` et `$SonarScannerHOME\bin` dans les variables de l'environnement

Etape 4 : Configuration de serveur SonarQube

Définir l'accès à la base de données en modifiant le fichier `$SonarQubeHOME\conf\sonar.properties`

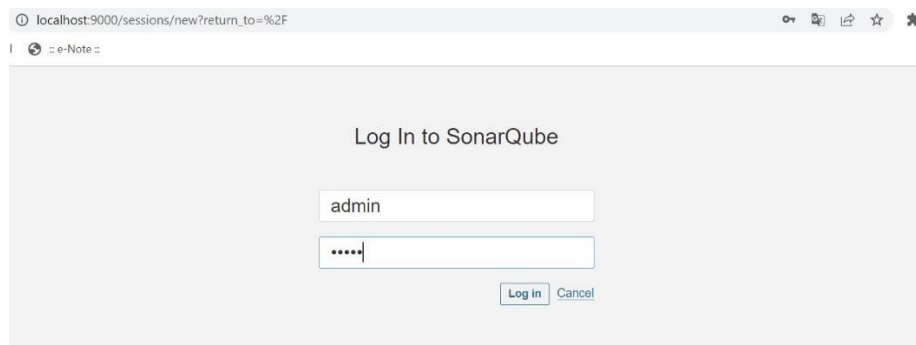
```
sonar.jdbc.username=sonar
sonar.jdbc.password=sonar
sonar.jdbc.url=jdbc:postgresql://localhost/sonarqube?currentSchema=public
```

Etape 5 : Créer un projet

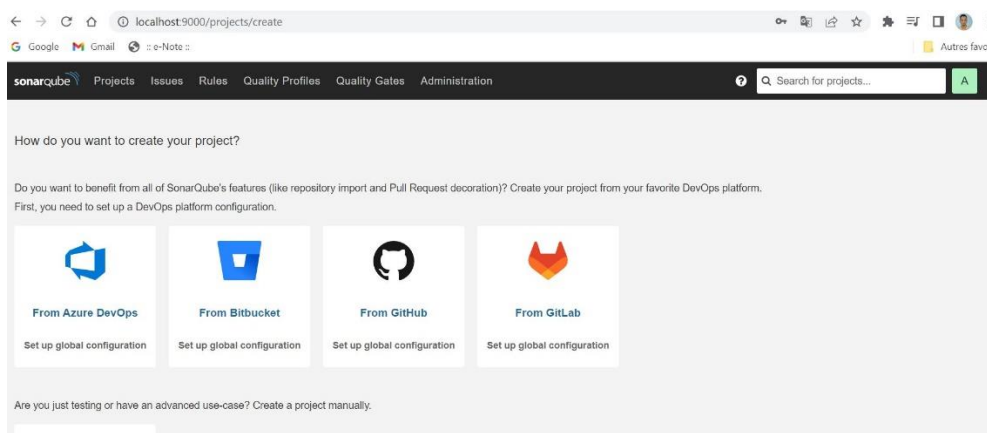
Lancer la commande : `StartSonar.bat`

```
ababar@DESKTOP-D4UGJKQ MINGW64 ~/Desktop
$ StartSonar.bat
Starting SonarQube...
2022.11.29 17:53:58 INFO app[][o.s.a.AppFileSystem] Cleaning or creating temp d
```

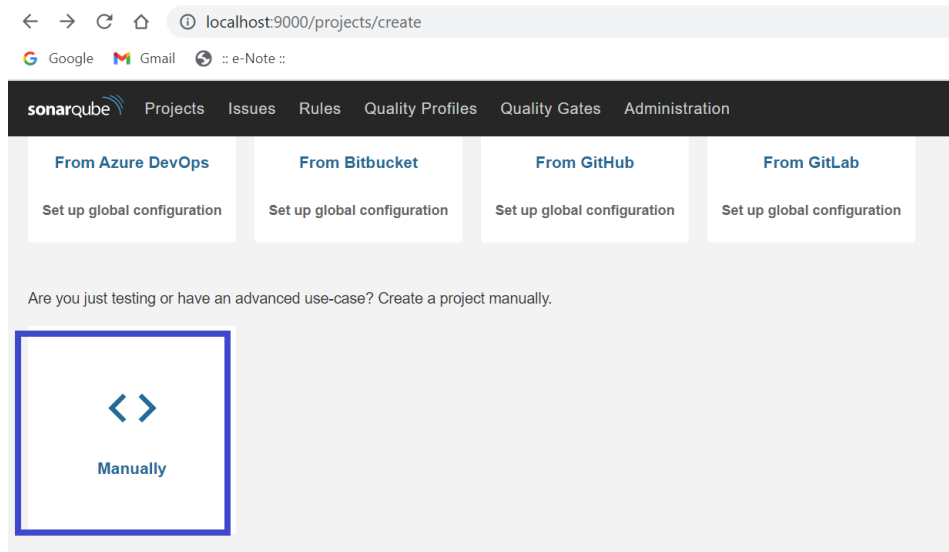
Vérifier que le serveur SonarQube est lancé



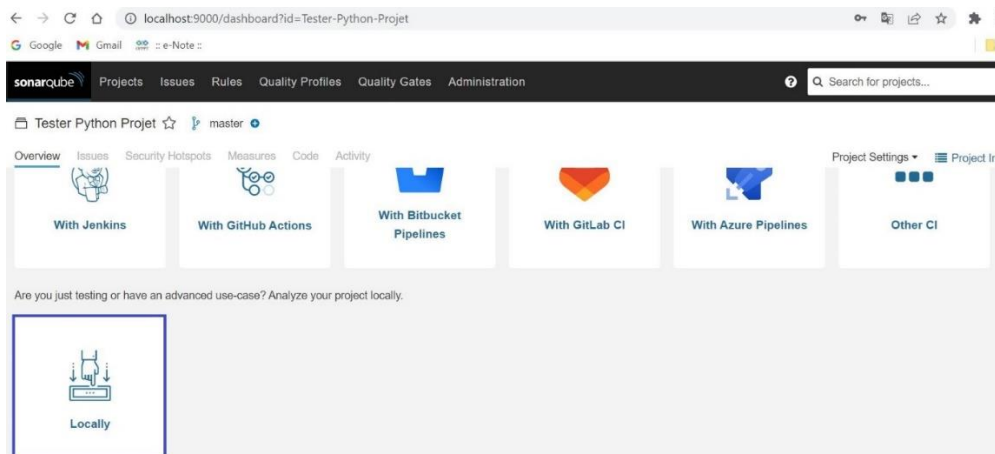
Utiliser **admin/admin** pour se connecter et changer le mot de passe



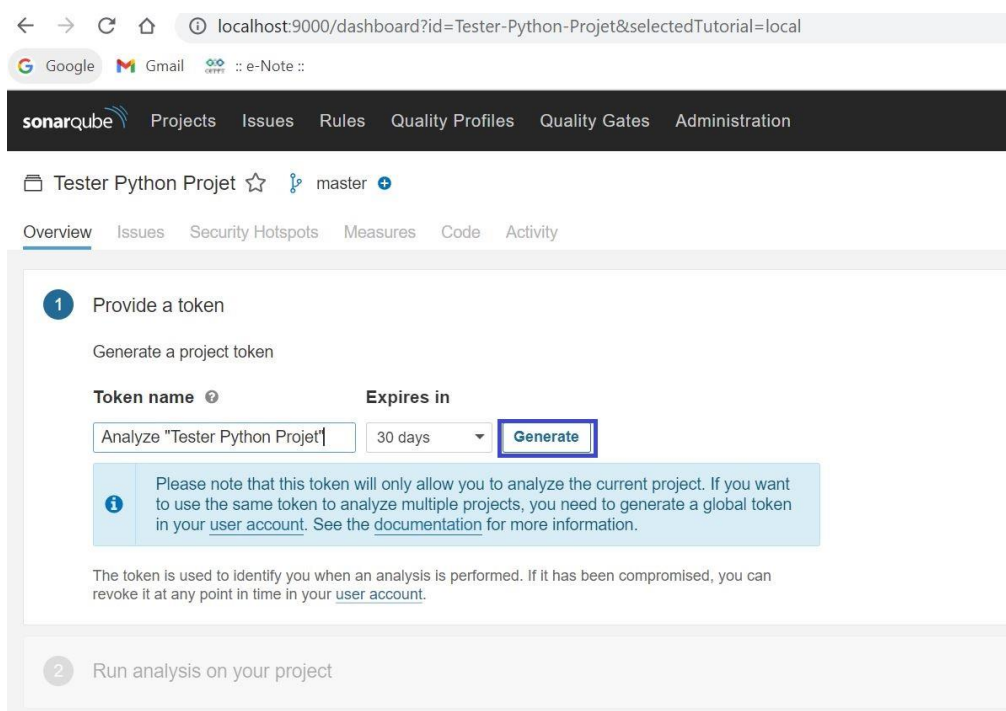
Pour créer un projet manuellement, accéder au « Projects » et cliquer sur le bouton «Manually »



Pour tester un projet en local, cliquer sur « Locally »



Générer un token



Cliquer sur « Continue »

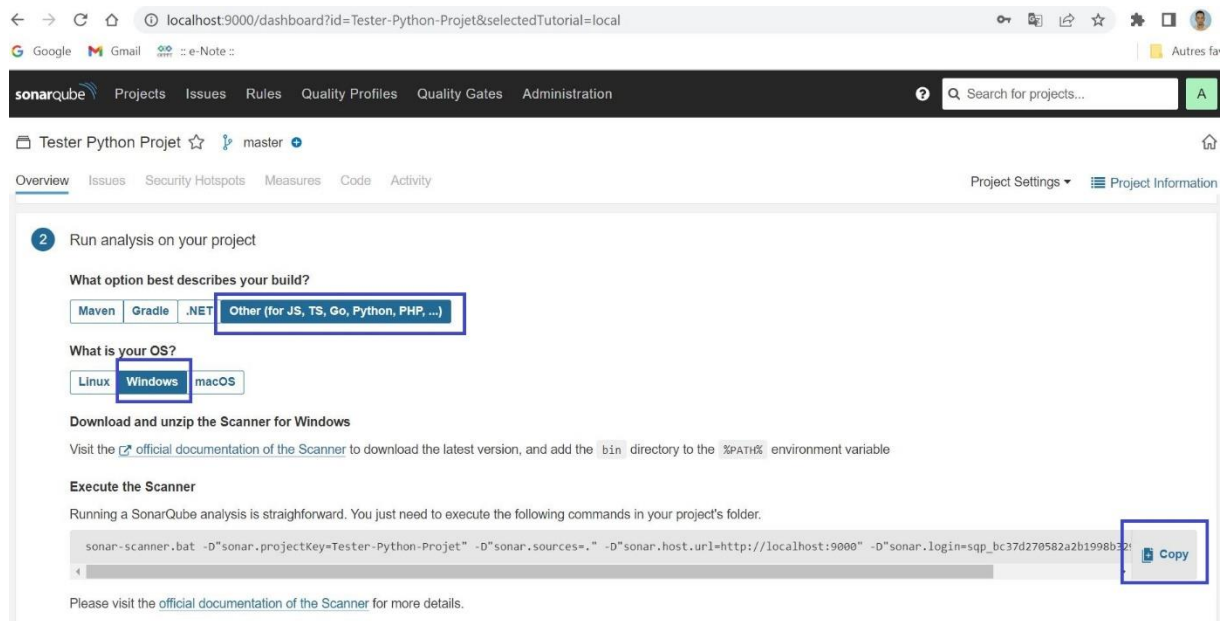
1 Provide a token

Analyze "Tester Python Projet": `sqp_bc37d270582a2b1998b3290d45ad7e43d52ecd05` 

The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your [user account](#).

Continue

Choisir le type de build ou le langage et type de votre système d'exploitation



localhost:9000/dashboard?id=Tester-Python-Projet&selectedTutorial=local

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Tester Python Projet master

Overview Issues Security Hotspots Measures Code Activity

Project Settings Project Information

2 Run analysis on your project

What option best describes your build?

Maven Gradle .NET Other (for JS, TS, Go, Python, PHP, ...)

What is your OS?

Linux Windows macOS

Download and unzip the Scanner for Windows

Visit the [official documentation of the Scanner](#) to download the latest version, and add the `bin` directory to the `%PATH%` environment variable

Execute the Scanner

Running a SonarQube analysis is straightforward. You just need to execute the following commands in your project's folder.

```
sonar-scanner.bat -D"sonar.projectKey=Tester-Python-Projet" -D"sonar.sources=." -D"sonar.host.url=http://localhost:9000" -D"sonar.login=sqp_bc37d270582a2b1998b3290d45ad7e43d52ecd05"
```

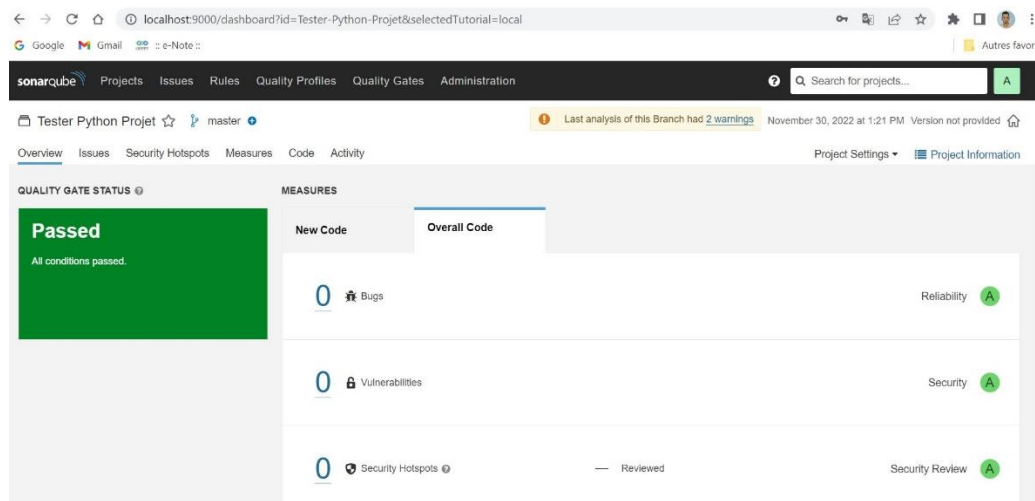
Copy

Please visit the [official documentation of the Scanner](#) for more details.

Exécuter la commande dans le dossier de votre projet



Une fois que le scanner à terminer l'analyse, les données seront envoyées automatiquement au serveur SonarQube.



localhost:9000/dashboard?id=Tester-Python-Projet&selectedTutorial=local

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration

Tester Python Projet master

Overview Issues Security Hotspots Measures Code Activity

Project Settings Project Information

QUALITY GATE STATUS

Passed

All conditions passed.

MEASURES

New Code Overall Code

0 Bugs Reliability A

0 Vulnerabilities Security A

0 Security Hotspots Security Review A

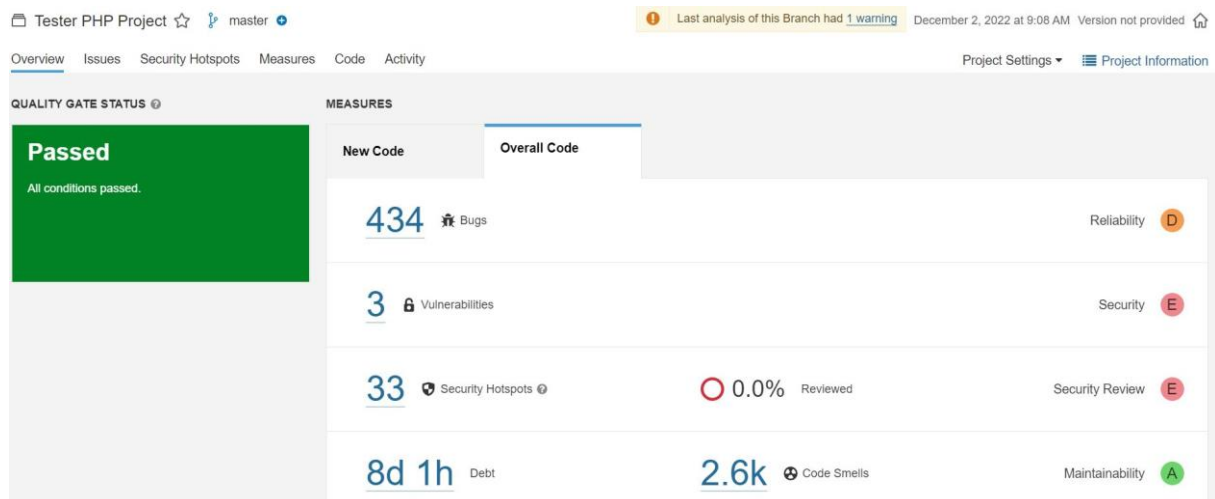
4.2.5 - Les fonctionnalités de SonarQube

4.2.5.1 - Analyse des résultats SonarQube

En supposant que SonarQube soit en local, les résultats de l'analyse seront consultables sur <http://localhost:9000/sonar/>. Chaque projet dispose d'un espace d'analyse dédié.

Ecran d'accueil de SonarQube « Overview »

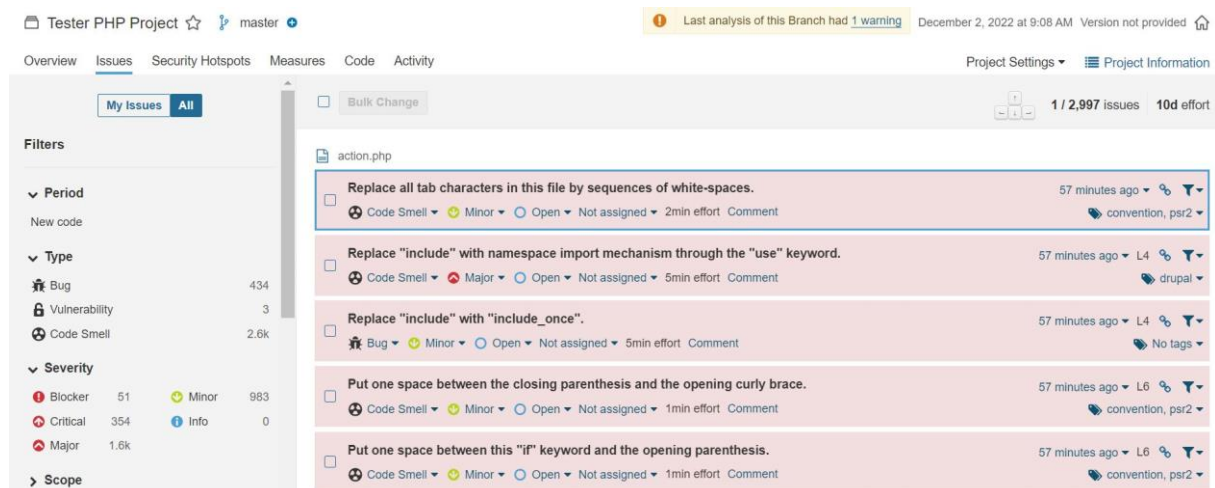
Sur l'écran d'accueil d'un projet analysé par SonarQube se trouve un récapitulatif des grandes familles de défauts :



La mention « Passed » signifie que le projet satisfait les exigences minimum définies dans l'espace « Barrières qualité ». Les paramètres par défaut peuvent être adaptés.

Onglet « Défauts » | « Issues »

Dans cet onglet, chaque bug, vulnérabilité ou code smell est répertorié et détaillé dans une liste.



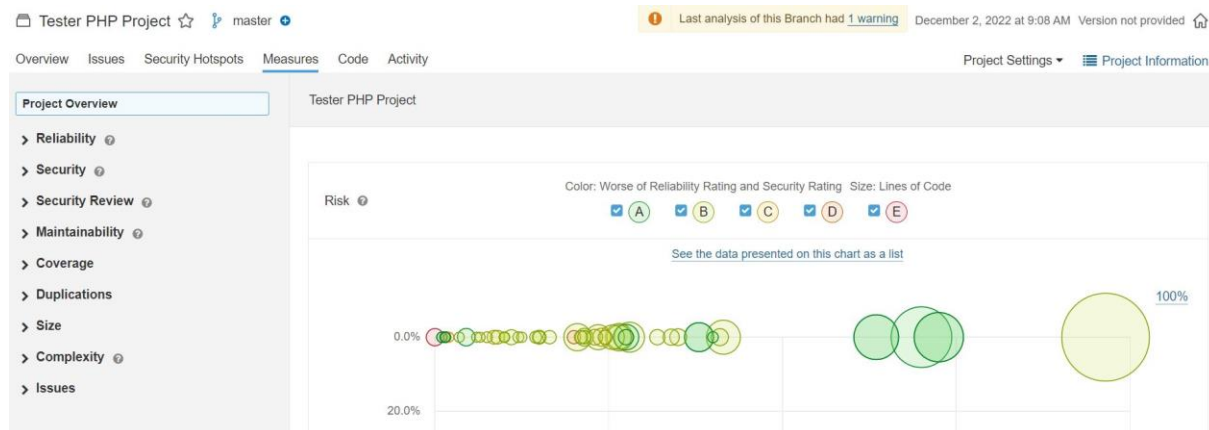
L'intérêt immédiat, orienté projet : tel un outil de bugtracking ou de gestion de tickets, cette liste permet de prioriser les points à traiter, d'évaluer leur temps de correction et de les affecter à un développeur. Le statut de chaque défaut est affiché. Les options des projets, tels que l'utilisateur affecté par défaut, peuvent être configurés (onglet « Configuration » -> « Configuration du projet »).

En se positionnant en vue « Effort », on peut également voir le temps total que prendrait la correction de tous les défauts du code, ce qui peut faciliter la planification du travail d'un développeur lors d'un sprint par exemple.

Le second intérêt, orienté humain : comme chaque défaut comporte une explication détaillée et des pistes d'amélioration, le développeur est susceptible de monter en compétence et d'améliorer à long terme la qualité de son code. C'est donc un véritable outil d'amélioration continue !

Onglet « Mesures »

On y trouve des infos supplémentaires, plus générales, telles que la complexité du projet et le pourcentage de commentaires (20% étant habituellement reconnu comme un bon pourcentage). Contrairement à l'écran précédent, celui-ci n'est pas très bavard : à l'utilisateur de trancher si le taux de complexité est acceptable, par exemple.



Les classes d'évaluation

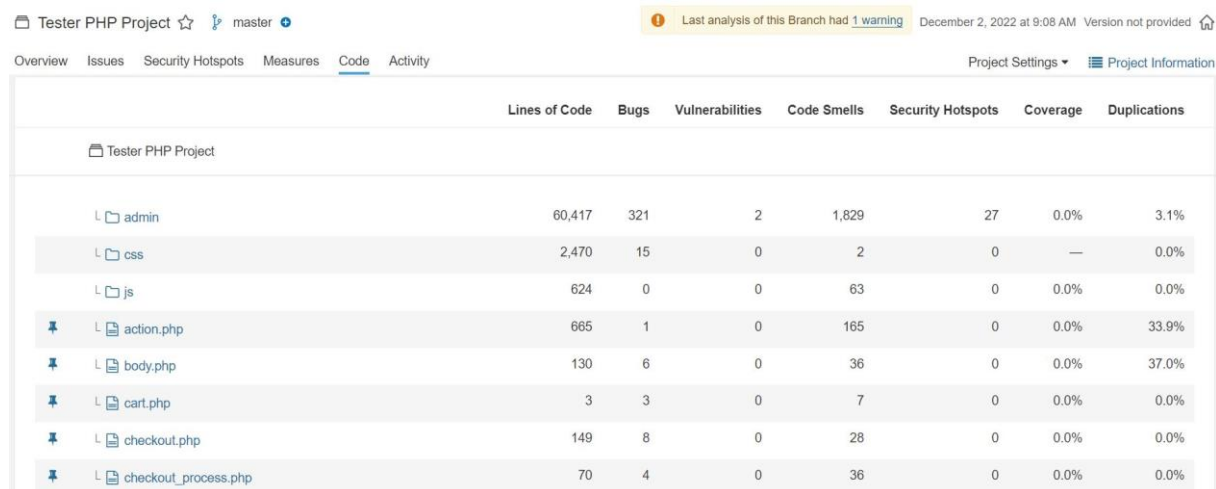
Classe	Maintenabilité	Bugs	Sécurité
A	$\leq 5\%$	0	$\geq 80\%$
B	6 à 10%	1 mineur	$\geq 70\%$ à $< 80\%$
C	11 à 20%	1 majeur	$\geq 50\%$ à $< 70\%$
D	21 à 50%	1 critique	$\geq 30\%$ à $< 50\%$
E	$> 50\%$	1 bloqueur	$< 30\%$

Onglet « Security Hotspot »

Security Hotspot met en évidence un morceau de code sensible à la sécurité que le développeur doit examiner. Après examen, vous constaterez soit qu'il n'y a pas de menace, soit que vous devez appliquer un correctif pour sécuriser le code.

Onglet « Code »

Cet onglet est particulièrement pratique. Il permet en effet de visualiser, fichier par fichier, les problèmes identifiés par SonarQube. Le code est intégralement affiché, avec des icônes au niveau des lignes posant problème.



The screenshot shows the SonarQube interface for a project named 'Tester PHP Project'. The 'Code' tab is selected, displaying a table with columns: Lines of Code, Bugs, Vulnerabilities, Code Smells, Security Hotspots, Coverage, and Duplications. The table lists various files and directories, including 'admin', 'css', 'js', and several PHP files like 'action.php', 'body.php', 'cart.php', 'checkout.php', and 'checkout_process.php'. Each row shows the corresponding metrics for that file.

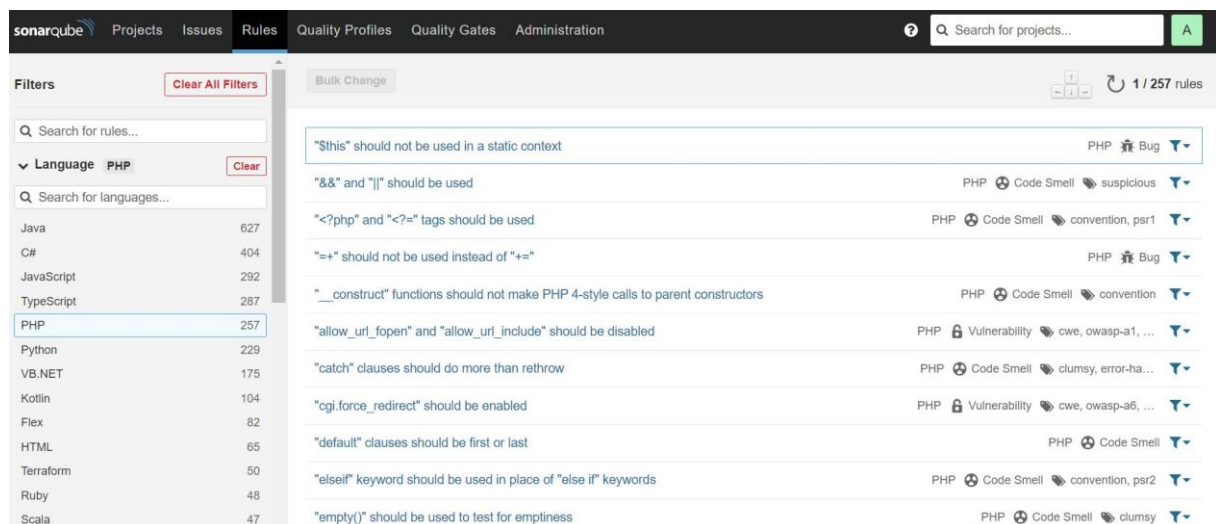
	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
admin	60,417	321	2	1,829	27	0.0%	3.1%
css	2,470	15	0	2	0	—	0.0%
js	624	0	0	63	0	0.0%	0.0%
action.php	665	1	0	165	0	0.0%	33.9%
body.php	130	6	0	36	0	0.0%	37.0%
cart.php	3	3	0	7	0	0.0%	0.0%
checkout.php	149	8	0	28	0	0.0%	0.0%
checkout_process.php	70	4	0	36	0	0.0%	0.0%

4.2.5.2 - Gestion des règles SonarQube

Liste des règles

La liste exhaustive des règles associées aux défauts est disponible à cette adresse : http://localhost:9000 /coding_rules.

Dans cet espace, il est possible de modifier la sévérité que SonarQube attribue à chaque défaut. La détection de certains défauts peut aussi être désactivée. Lors des analyses suivantes, ils seront donc ignorés.



The screenshot shows the SonarQube 'Rules' tab. On the left, there's a 'Filters' sidebar with a search bar and a list of languages. The 'Language' filter is set to 'PHP'. The main area displays a list of rules with their descriptions, categories, and severity levels. The rules are sorted by the number of issues found (1 / 257 rules).

Rule Description	Category	Severity
"\$this" should not be used in a static context	PHP Bug	High
"&&" and " " should be used	PHP Code Smell	suspicious
"<?php" and "<?=" tags should be used	PHP Code Smell	convention, psr1
"==" should not be used instead of "==="	PHP Bug	High
"__construct" functions should not make PHP 4-style calls to parent constructors	PHP Code Smell	convention
"allow_url_fopen" and "allow_url_include" should be disabled	PHP Vulnerability	cwe, owasp-a1, ...
"catch" clauses should do more than rethrow	PHP Code Smell	clumsy, error-ha...
"cgi.force_redirect" should be enabled	PHP Vulnerability	cwe, owasp-a6, ...
"default" clauses should be first or last	PHP Code Smell	convention, psr2
"elseif" keyword should be used in place of "else if" keywords	PHP Code Smell	clumsy
"empty()" should be used to test for emptiness	PHP Code Smell	clumsy

Travaux dirigés et pratiques – M202 Adopter l'approche agile

TD1 : Planifier un projet

Exercice 1 :

Soit la liste des **tâches** suivante :

- A : choisir le menu (30 min)
- B : acheter les ingrédients (90 min)
- C : préparer l'apéritif (30 min)
- D : nettoyer la table (10 min)
- E : mettre la table (10 min)
- F : préparer les ingrédients (30 min)
- G : cuisiner les plats (60 min)
- H : servir le repas (10 min)

Soit la liste des **contraintes** suivante :

- B doit être après A
- C et F doivent être après B
- E doit être après D
- G doit être après F
- H doit être après C, G et E

1. Compéter le tableau suivant :

Tâche	Durée	Antécédent	Successeur

2. Tracer le diagramme de GANTT
3. Identifier le chemin critique

Exercice 2 :

Pour la réalisation d'un projet, le découpage réalisé par la méthode du « brainstorming » a donné le tableau d'ordonnancement suivant :

Unité temps = semaine

Tâche	Durée	Antécédent	Successeur
A	4	-	
B	2	-	
C	8	B	
D	4	B	
E	4	-	
F	2	A, C, D, E	
G	2	F	

1. Compléter le tableau d'ordonnancement du projet (colonne successeur)
2. Tracer le diagramme de GANTT
3. Identifier le chemin critique
4. Donner, sous forme de tableau, les dates (début/fin) au plutôt, au plus tard et les marges totales des tâches.

TD2 : Adopter une méthode agile SCRUM

Question 1

En scrum, qui valide les user-stories pour qu'elles passent de « test » à « done » ?

- A. le scrum master
- B. le product owner
- C. l'équipe de développement
- D. toute l'équipe

Question 2

L'équipe de développement peut-être composée :

- A. de développeurs front et de développeurs back
- B. de développeurs, d'un business analyst et d'un testeur
- C. d'un testeur et de développeurs
- D. de développeurs, d'un PMO et d'un testeur

Question 3

Le product backlog refinement est :

- A. le nouveau nom de la backlog grooming
- B. une cérémonie scrum
- C. une pratique d'affinage de user-stories et d'estimation en poker planning
- D. ce n'est pas officiel en scrum

Question 4

Quels artefacts existent en scrum ?

- A. le definition of done
- B. le definition of ready
- C. le sprint backlog
- D. l'increment backlog

Question 5

Le product owner est responsable :

- A. de valider que les items sont bien « done »
- B. de la gestion du product backlog
- C. que l'équipe de développement comprenne bien les items
- D. de rendre transparent le sprint backlog

Question 6

Le scrum master doit :

- A. avoir des connaissances avancées sur la gestion d'un product backlog
- B. faciliter les cérémonies scrum
- C. travailler avec les autres scrum master de l'entreprise
- D. coacher l'équipe de développement si nécessaire

Question 7

Quelles sont les phrases qui sont vraies autour du sprint review

- A. le product owner y invite l'équipe scrum et les principales parties prenantes
- B. l'équipe discute des problèmes rencontrés et comment ils ont été résolus
- C. l'équipe y démontre le travail « fini »
- D. la revue des délais et des budgets doit être fait

Question 8

Le scrum :

- A. est un cadre de travail
- B. est une méthode agile
- C. ne s'occupe pas de la maintenance applicative
- D. est un cadre léger

Question 9

Le scrum permet dans des cas exceptionnels d'annuler un sprint. Quelles phrases sont vraies ?

- A. l'équipe de développement peut demander au product owner d'annuler le sprint
- B. seul le product owner est autorisé à annuler un sprint
- C. un sprint recommence la semaine suivante pour ne pas perturber les dates des cérémonies
- D. seul le scrum master est autorisé à annuler un sprint

Question 10

Quelle(s) affirmation(s) sont vraies concernant la sprint planning ?

- A. tous les items choisis en sprint planning ne sont pas forcément finalisés.
- B. seule l'équipe de développement valide sa capacité de faire (le nombre d'items du sprint)
- C. un sprint planning peut durer jusqu'à 8h
- D. l'équipe de développement doit définir comment elle va réaliser les items

TP1 : Créer un projet SCRUM

Commençons par constituer des groupes de 5 personnes qui jouent les rôles ci-dessous



Chapeau Product
Owner



Chapeau
Designer



Chapeau
Développeur
front-end



Chapeau
Développeur
back-end



Chapeau Testeur

Quand vous portez un chapeau, vous devez jouer le rôle jusqu'à la fin de l'exercice.

A tour de rôle, vous devrez jouer au moins le rôle du Product Owner.

Dans cet exercice, on souhaite appliquer la méthode SCRUM via l'application JIRA Software.

Pendant la réalisation de TP, prenez des captures d'écran qui vont vous aider lors de création de rapport.

Etape 1 : Créer un projet SCRUM « Gestion d'établissement » (Capture 1)

Etape 2 : Créer votre premier Sprint « Paramétrage de l'application ». (Capture 2)

Vous trouvez ci-dessous la liste des tâches à réaliser durant ce sprint :

Numéro	Tâche	Affecter à
1	Créer la charte graphique	Designer
2	Intégrer l'interface de CRUD (Bootstrap)	Développeur front-end
3	Créer la base de données	Développeur back-end
4	Développer l'interface de CRUD – Stagiaire	Développeur back-end
5	Développer l'interface de CRUD – Formateur	Développeur back-end
6	Développer l'interface de CRUD – Filière	Développeur back-end
7	Développer l'interface de CRUD – Module	Développeur back-end
8	Tester les interfaces créées	Testeur

Etape 3 : Démarrer votre Sprint (Capture 3)

La durée de Sprint est une semaine

Etape 4 : Réalisation des tâches

- Etat 1 : Après 5 min de lancement de sprint, le designer et le développeur back-end doivent commencer les tâches : 1 et 3. (Capture 4)
- Etat 2 : Après que le designer termine la tâche 1, le développeur front-end doit commencer la tâche 2. (Capture 5)
- Etat 3 : Après que le développeur front-end termine la tâche 2 et le développeur back-end termine la tâche 3, le développeur back-end doit commencer les tâches 4, 5, 6 et 7 (Capture 6)
- Etat 4 : Après que le développeur back-end termine les tâches 4, 5, 6 et 7, le testeur doit commencer la tâche 8 (Capture 7)
- Etat 5 : Après que le testeur termine la tâche 8, le product owner doit terminer Sprint 1. (Capture 8)

Etape 5 :

Organiser sprint review et sprint rétrospective

Décrire les objectifs de chaque réunion dans le rapport qui sera livré dans la fin de TP.

Etape 6 : Terminer Sprint 1 (Capture 9)**Etape 7 :**

Organiser Sprint planning pour le module gestion des inscriptions

Découper ce module en tâches et désigner le réalisateur de chaque tâche.

Etape 8 :

Créer le deuxième Sprint « module gestion des inscriptions » et lancer votre sprint. (Capture 10)

Etape 9 :

Organiser sprint review et sprint rétrospective

Décrire les objectifs de chaque réunion dans le rapport qui sera livré dans la fin de TP.

Etape 10 : Terminer Sprint 2

TP2 : Créer un dépôt git sur GitLab

Installation de Git

Installer git via le lien <https://git-scm.com/downloads>

Maintenant que vous avez installé Git sur votre système, vous voudrez personnaliser votre environnement Git en exécutant les deux commandes suivantes :

```
git config --global user.email "YOUR_EMAIL"  
git config --global user.name "YOUR_NAME"
```

Création de compte sur gitlab

Allez sur <http://gitlab.com> et suivez les instructions pour la création et l'activation de votre compte.

Création d'un dépôt

Nous allons maintenant créer un dépôt distant. Sur <https://gitlab.com/dashboard/projects/> :

- Cliquez sur **New project** puis **Create blank project**.
- Donnez un nom à votre dépôt, par exemple **PremierDepot**. Par défaut, votre dépôt est privé Terminez en cliquant sur **Create project**.

Cloner votre dépôt en utilisant SSH

Créer une clé SSH et ajouter cette clé dans SSH Keys

Premiers commits

1. Ajouter des fichiers dans votre répertoire où le dépôt git a été initialisé.
2. Utilisez les commandes git status et git diff en vous aidant de la documentation au besoin. Que constatez-vous ?
3. Indexez l'ensemble des fichiers puis vérifiez l'indexation avec git status.
4. Poussez les modifications sur votre dépôt distant.

TP3 : Collaborer sur un dépôt gitlab

Commençons par constituer des groupes de binôme qui jouent les rôles ci-dessous



Chapeau **propriétaire de projet**



Chapeau **développeur**

Travaillant sur le même dépôt **PremierDepot**.

A tour de rôle, vous devrez jouer au moins le rôle "**propriétaire de projet**".

Etape 1 :

Le **propriétaire** doit inviter le **développeur** pour collaborer dans votre projet.

Etape 2 :

Le **développeur** doit importer le dépôt de **propriétaire** (git clone).

Le **développeur** va ajouter un fichier dans le dépôt distant.

Etape 3 :

Le **propriétaire** doit récupérer en local les modifications que le développeur a envoyées.

TP4 : Les branches et gestion de conflits

Un conflit de fusion intervient lorsque l'on tente de fusionner deux branches qui modifient la même partie d'un même fichier.

Procédure pour causer un conflit de fusion

Étapes à suivre, une par une :

1. Créer un nouveau dépôt sur GitLab
2. Cloner ce dépôt localement avec **git clone**
3. Dans le répertoire du dépôt local, créer un fichier **README.md** contenant les paroles d'une chanson de votre choix
4. Créer un commit initial sur la branche **main** et l'envoyer sur GitLab avec **git add**, **git commit** puis **git push**
5. Créer une branche **branche1** à partir de **main**, avec **git checkout -b**
6. Dans le **README.md** de cette branche, modifier à votre guise la première phrase des paroles, créer un commit, puis envoyer les modifications de cette branche sur GitLab
7. Revenir à la branche **main** avec **git checkout**
8. Créer une branche **branche2** à partir de **main** (comme dans l'étape 5)
9. Dans le **README.md** de cette branche, modifier également la première phrase des paroles avec un texte différent de celui saisi à l'étape 6, créer un commit, puis envoyer les modifications de cette branche sur GitLab
10. Revenir à la branche **main**
11. Fusionner **branche1** dans **main**, avec **git merge**
12. Fusionner **branche2** dans **main**

Si vous avez bien suivi cette procédure, vous devriez obtenir un conflit de fusion :

```
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
```

Procédure pour résoudre un conflit de fusion

Pour résoudre ce conflit, il va falloir :

1. ouvrir le fichier **README.md** dans son éditeur de code,
2. Constater comment git représente le conflit, et la source de chaque version,
3. éditer le fichier pour ne conserver que la version finale souhaitée,
4. puis créer un commit.

Bonus

Supprimer les branches **branche1** et **branche2**, non seulement dans votre dépôt local, mais aussi dans le dépôt distant associé (sur GitLab).

TP5 : Analyser votre code avec SonarQube

L'objectif de TP est d'analyser le code d'un projet.

Etape 1 : Installer JDK 11

Etape 2 : Installer la base de données PostgreSQL 13 et PgAdmin 4

Etape 3 : Créer un utilisateur et une base de données.

Etape 4 : Installer serveur SonarQube et SonarScanner

Etape 5 : Configurer votre serveur SonarQube (Modifier le fichier sonar.properties)

```
sonar.jdbc.username=user_name  
sonar.jdbc.password=user_password  
sonar.jdbc.url=jdbc:postgresql://localhost/database_name?currentSchema=public
```

Etape 6 : Lancer serveur SonarQube et se connecter via le compte admin/admin

Etape 7 : Créer un nouveau projet via <http://localhost:9000/projects>

Etape 8 : Télécharger n'importe quel projet public sur github

Etape 9 : Lancer la commande d'analyse dans le dossier de projet téléchargé

Références

- <https://www.manager-go.com/gestion-de-projet/>
- <https://openclassrooms.com/fr/courses/4296701-gerez-un-projet-digital-avec-une-methodologie-en-cascade>
- <https://www.appvizer.fr/magazine/collaboration/gestion-de-taches>
- <https://blog.myagilepartner.fr/index.php/category/blog-scrum/>
- <https://docs.gitlab.com/>
- <https://docs.sonarqube.org/latest/>
- Métrique et critères d'évaluation de la qualité du code source d'un logiciel | Pierre Mengal